AI-Based Adaptive VR Meditation System

INDIVIDUAL THESIS

Badda Liyanage Don Praveen Devinda Perera

IT22191038

Individual component: Cosinor-Enhanced Multi-level Stress Inference from Wearable RR Intervals under Real-Time Deployment Constraints

Group ID: R26-DS-006

Report submitted in partial fulfilment of the requirements for the
B.Sc. (Hons) Degree in Information Technology
Specialising in Data Science

Department of Computer Science

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Badda Liyanage Don Praveen Devinda Perera

Student ID: IT22191038

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Mr. Samadhi Rathnayake		
Co-supervisor: Ms. Malithi Nawarathne		

ABSTRACT

Heart rate variability (HRV) measured using wearable devices is widely used for stress detection. However, many reported performance results come from offline evaluations where the complete recording is available. This research develops a multi-level stress inference component for an adaptive virtual reality (VR) meditation system and investigates whether improvements observed offline remain valid during real-time operation. Four stress levels were estimated using RR intervals and skin temperature. Models were evaluated using leave-one-subject-out cross-validation on WESAD, with additional experiments conducted on an independent wrist-PPG dataset. Adding 24-hour Cosinor features increased macro-F1 from 0.551 to 0.642. However, further analysis showed that this improvement was not caused by circadian correction. The recordings covered only 6.2% of a day, making reliable estimation of a 24-hour rhythm impossible. Residual features contributed more to performance than the fitted baseline, while different assumed periods produced similar results. The pattern replicated across datasets and with protocol-defined labels. These results indicate that the improvement mainly comes from within-session baseline normalisation. Midpoint labelling increased macro-F1 by 0.071 to 0.084 across four model configurations, producing overly optimistic offline estimates. Reducing the window length to 60 beats lowered prediction delay to approximately 45 seconds without reducing accuracy. A causal baseline seeded from the population mean improved macro-F1 by 0.068 over no baseline, while personalisation and a dedicated causal architecture brought no measurable gain and forecasting did not outperform persistence. The deployed two-way ensemble combines gradient-boosted trees with a causal multi-scale neural network. It processes PPG data through a Python backend and reports stress levels with confidence information and uncertainty bands. Under real-time constraints, it achieved a macro-F1 of 0.5923 and quadratic weighted kappa of 0.7525. Each 15-second signal frame was processed in less than half a second.

Keywords: heart rate variability, stress detection, real-time inference, baseline normalisation, wearable sensors

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, Mr. Samadhi Rathnayake, and my co-supervisor, Ms. Malithi Nawarathne, for their guidance, encouragement and constructive feedback throughout this research. Their questions about the evidence behind each result shaped the direction of this work.

I am grateful to the members of project group R26-DS-006, whose work on the wearable device, the mobile application, the VR environment and the voice-based component made the complete system possible, and whose clients were built against the interface of this component.

I also thank the authors of the WESAD dataset and of the wearable stress and exercise dataset of Hongn et al. for making their data publicly available, and the Sri Lanka Institute of Information Technology for providing the academic environment in which this research was carried out.

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 Overall project context 1

1.2 Individual component context 2

1.3 Problem statement 4

1.4 Research gap 6

1.5 Aim and objectives 7

1.5.1 Aim 7

1.5.2 Objective 8

1.6 Research questions or hypotheses 8

1.7 Scope limitations and assumptions 9

1.8 Individual contribution 11

1.9 Thesis structure 13

2 LITERATURE REVIEW AND RELATED WORK 15

2.1 Review method and scope 15

2.2 Domain and theoretical foundations 17

2.2.1 Wearable Physiological Stress Inference 17

2.2.2 RR Intervals and Heart Rate Variability 17

2.2.3 Skin Temperature 18

2.2.4 Multi-Level Stress Representation 18

2.2.5 Cosinor Rhythmometry 19

2.2.6 Machine Learning and Deep Learning 19

2.2.7 Causality in Real-Time Stress Inference 20

2.3 Related research 20

2.4 Existing systems products and techniques 23

2.4.1 Wearable and Consumer Monitoring 23
2.4.2 HRV Feature-Based Stress Detection 23
2.4.3 Deep-Learning Techniques 24
2.4.4 Temporal Baseline Modelling 24
2.4.5 Multi-Level Stress Inference 25
2.4.6 Real-Time and Deployable Techniques 25
2.5 Gap synthesis and conceptual framework 26
3 RESEARCH METHODOLOGY 29
3.1 Research design 29
3.2 Data participants or study materials 31
3.2.1 WESAD Dataset 31
3.2.2 Stress-Level Construction 31
3.2.3 Secondary Wearable Dataset 32
3.3 Experimental or study procedure 33
3.3.1 Physiological Data Preparation 33
3.3.2 HRV and Temporal Feature Construction 34
3.3.3 Temporal Mechanism Experiments 34
3.3.4 Window Labelling and Causal Evaluation 36
3.3.5 Causal Baseline Estimation 36
3.3.6 Model Comparison 39
3.3.7 Deployment Experiments 39
3.4 Measures and evaluation metrics 40
3.4.1 Macro-F1 Score 40
3.4.2 Quadratic Weighted Kappa 41
3.4.3 Precision, Recall and Confusion Matrix 41
3.4.4 Ordinal Error Measures 42

3.4.5 Prediction Latency 42
3.4.6 Model Stability 42
3.4.7 Subject-Independent Evaluation 43
3.4.8 Confidence and Coverage 43
3.4.9 Forecasting Performance 43
3.5 Data analysis 44
3.5.1 Paired comparisons 44
3.5.2 Effect sizes 45
3.5.3 Multiple comparisons 45
3.5.4 Confidence intervals 46
3.5.5 Correlation analyses 46
3.5.6 Reporting conventions 47
3.6 Ethics privacy security and responsible AI 47
3.6.1 Data and Consent 47
3.6.2 Privacy 48
3.6.3 Security 49
3.6.4 Responsible Use of the Predictions 49
3.6.5 Fairness and Generalisation 50
3.6.6 Integrity of Reporting 50
3.7 Reproducibility and research artefacts 51
3.7.1 Experimental Environment 51
3.7.2 Controlling Randomness 51
3.7.3 Built-in Verification Checks 52
3.7.4 Traceability of Reported Figures 52
3.7.5 Research artefacts 53
3.7.6 Code and data availability 55

4 PRODUCT AND SOLUTION DESIGN 56
4.1 Requirements 56
4.1.1 Stakeholders 57
4.1.2 Functional requirements 58
4.1.3 Non-functional requirements 60
4.1.4 Changes from the proposed requirements 62
4.1.5 Constraints and Exclusions 64
4.2 Component architecture within the complete solution 66
4.2.1 Position in the Complete System 66
4.2.2 External interfaces 68
4.2.3 Internal Architecture 70
4.2.4 Ingestion and Signal Processing 72
4.2.5 Causal State 72
4.2.6 Windowing and Features 72
4.2.7 Inference 73
4.2.8 Output 74
4.3 Design alternatives and decisions 74
4.3.1 Labelling Convention 76
4.3.2 Baseline Estimator 77
4.3.3 Window Length 78
4.3.4 Neural Architecture 78
4.3.5 Ensemble Composition and Personalisation 79
4.3.6 Cold-Start Seeding 80
4.3.7 Output Form 80
4.3.8 Forecasting 81
4.4 Detailed Design 81

4.4.1 Signal Processing 82
4.4.2 Causal State 83
4.4.3 Feature Vector for the Gradient-Boosted Trees 84
4.4.4 Sequence Input for the Neural Network 85
4.4.5 MS-CGCA Network 85
4.4.6 Gradient-Boosted Trees 87
4.4.7 Ensemble and Confidence Gate 87
4.4.8 Output Message 88
4.5 Standards and professional practice 89
5 IMPLEMENTATION INTEGRATION AND TESTING 91
5.1 Development environment and technology stack 91
5.2 Component Implementation 93
5.2.1 Single Source of Truth 94
5.2.2 Streaming Inference Engine 94
5.2.3 Server Layer 95
5.2.4 Model Artefacts 95
5.3 Integration with the Other Components 96
5.4 Verification and software testing 99
5.5 Deployment and operation 102
6 RESULTS AND VALIDATION 104
6.1 Research results 104
6.1.1 Temporal-Basis Audit 104
6.1.2 Mechanism Decomposition 105
6.1.3 Timescale Invariance 107
6.1.4 Statistical Validation and Per-Subject Analysis 108
6.1.5 Cross-Dataset Replication and Label Sensitivity 110

6.1.6 Ordinal Error Structure 112
6.1.7 Cost of the Labelling Convention 113
6.1.8 Window Length and Model Configuration 114
6.1.9 Cold-Start Seeding 115
6.1.10 Confidence-Gated Output and Forecasting 117
6.2 Product and technical results 119
6.2.1 Deployed Operating Point 119
6.2.2 Processing Performance 120
6.2.3 Requirement Verification 121
6.3 User or stakeholder validation 123
6.4 Integration results 124
6.5 Summary of findings 126
7 DISCUSSION 128
7.1 Interpretation of findings 128
7.2 Comparison with literature baselines and existing solutions 130
7.3 Research contribution 132
7.4 Product contribution 134
7.5 Limitations and threats to validity 135
7.5.1 Statistical Conclusion Validity 135
7.5.2 Internal Validity 136
7.5.3 Construct Validity 137
7.5.4 External Validity 137
7.5.5 Technical Limitations 138
7.6 Implications 139
8 CONCLUSION AND FUTURE DIRECTIONS 141
8.1 Conclusion 141

8.2 Achievement of objectives 142
8.3 Individual research and product contributions 145
8.3.1 Research Contributions 145
8.3.2 Product Development Contributions 145
8.3.3 Collaborative Contributions 146
8.4 Future directions 146
REFERENCES 149
GLOSSARY 153
APPENDICES 154
Appendix A Research instruments or extended results 154
Appendix B Academic integrity and AI use disclosure 155

LIST OF FIGURES

Figure 4.1: Position of the stress inference component within the complete system 66
Figure 4.2: Internal pipeline of Stress Inference Component 71

LIST OF TABLES

Table 3.1: Causal baseline trackers 38
Table 3.2: Research Artefacts 53
Table 4.1: Stakeholders of the stress inference component 57
Table 4.2: Functional requirements 58
Table 4.3: Non-functional Requirements 60
Table 4.4: Traceability from proposed to final requirements 62
Table 4.5: External interfaces of the stress inference backend 69
Table 4.6: Design alternatives and decisions 75
Table 4.7: Feature vector for the gradient-boosted trees 84
Table 4.8: Channels of the sequence input 85
Table 4.9: Layers of MS-CGCA Network 86
Table 4.10: Fields of the prediction message 88
Table 4.11: Standards and professional practices applied 89
Table 5.1: Technology stack of the component 91
Table 5.2: Main modules of the component 93
Table 5.3: Integration points with the other components 97
Table 5.4: Automated Test Suite 100
Table 6.1: Additive decomposition of the temporal features (XGBoost, WESAD) 105
Table 6.2: Performance against the assumed detrending period (XGBoost) 107
Table 6.3: Contribution of the fitted MESOR and the residual on the two datasets 110
Table 6.4: Decomposition with protocol-defined binary labels (XGBoost, WESAD) 111
Table 6.5: Inflation from midpoint labelling relative to endpoint labelling 113
Table 6.6: Deployable configurations (endpoint labelling, five seeds) 114
Table 6.7: Cold-start seeding of the causal baseline (endpoint labelling) 116
Table 6.8: Confidence gating at different coverage levels 117
Table 6.9: Forecasting against persistence (endpoint labelling) 118
Table 6.10: Processing performance of the deployed component 121
Table 6.11: Verification of the requirements 122
Table 6.12: Summary of findings against the research questions 126

Table 8.1: Achievement of the research objectives 142

LIST OF ABBREVIATIONS

Abbreviation	Meaning
BiLSTM	Bidirectional long short-term memory
BLE	Bluetooth Low Energy
BVP	Blood volume pulse
CNN	Convolutional neural network
CORN	Conditional ordinal regression for neural networks
ECG	Electrocardiogram
EDA	Electrodermal activity
EWMA	Exponentially weighted moving average
HF	High frequency (0.15–0.4 Hz)
HRV	Heart rate variability
JSON	JavaScript Object Notation
LF	Low frequency (0.04–0.15 Hz)
LOSO	Leave-one-subject-out
LSTM	Long short-term memory
MESOR	Midline estimating statistic of rhythm
MS-CGCA	Multi-Scale Circadian-Guided Cross-Attention
PPG	Photoplethysmography
RMSSD	Root mean square of successive differences
RR	Interval between successive R-peaks (heartbeats)

Abbreviation	Meaning
SD	Standard deviation
SDNN	Standard deviation of normal-to-normal intervals
TLS	Transport Layer Security
TSST	Trier Social Stress Test
VLF	Very low frequency (0.003–0.04 Hz)
VR	Virtual reality
WESAD	Wearable Stress and Affect Detection dataset
XGBoost	Extreme Gradient Boosting

1 INTRODUCTION

1.1 Overall project context

Stress can have implications for both the physical and mental aspects of well-being. In view of this, there is increasing attention being given to systems capable of providing continual detection of stress and support when the stress level rises. The usefulness of wearable sensors in this respect lies in their ability to monitor physiological information while the wearer goes about his normal activities. Machine learning techniques can be used to analyze this information and derive estimates of changes due to stress [1].

The overall research project, R26-DS-006, aims at designing an application that can in real-time detect the stress level of a user and assist him in reducing stress using an adaptive virtual reality environment. This research project utilizes wearable sensing, wireless communication, machine learning, virtual reality, and web-based technologies in order to gather physiological data from the user.

This process starts by collecting physiological signals of the user using a wearable sensor. Cardiovascular measurements constitute some of the primary inputs. Such measurements are used to obtain RR intervals and HRV characteristics. HRV is extensively applied in wearable stress research due to the ability of changes in intervals between heartbeats to provide insight into physiological stress response [1], [2].

The acquired data is sent to the mobile app using Bluetooth Low Energy technology. Next, the data is sent to the backend which is based on Python. The backend is responsible for all processing and stress detection operations. The information about the physiological state is used to determine the stress level.

The estimated stress level is then transmitted to the Meta Quest 2 virtual reality application. The VR environment can adjust depending on the estimated state of the user. The idea is to create an environment which will help reduce the stress of the user in case of high stress level detection. The mobile application also displays the physiological and stress-related data to the user.

The whole project is therefore based on a closed-loop system. The physiological data of the user are collected. The data collected are processed in order to estimate stress. Then the estimated stress level is utilized in order to adjust the VR environment. The goal of this system is not only to monitor stress level, but also to reduce stress by means of the environment.

In the framework of this complete project, this module is aimed at stress estimation task. The wearable RR interval and HRV data are used to estimate multiple stress levels. In addition, this research investigates the ability of the selected models to function in real-time mode. The next subsection provides a detailed description of this particular research.

1.2 Individual component context

This component is concerned with Multi-level Stress Inference in VR from Wearable RR Intervals under Real-time Deployments using Cosinor Enhancement. The primary objective of this component is to infer a user's level of stress based on his physiological signals. This prediction of the stress level will then be delivered to the whole system so that the VR world can adjust accordingly.

The main physiological signal utilized in this research is the RR interval. The RR interval is the duration of time between two successive beats. The RR interval series can be employed to extract features for heart rate variability (HRV). HRV has been extensively utilized in wearables for stress monitoring and machine learning for stress detection [1], [2]. It has also been reported in literature that HRV could be used to discriminate among multiple stress levels rather than just stress and non-stress conditions [3].

WESAD dataset was selected as the primary benchmark dataset in this research. WESAD includes physiological signals recorded from the subjects under baseline, stress, amusement, and meditation conditions [2]. The dataset has been extensively used for detecting stress from wearable sensors and is a well-known common dataset to compare various models.

In the first part of this research, the temporal aspect of HRV was considered to enhance stress inference. Physiological signals can vary depending on different factors. First, one of these factors might be the biological rhythm of an organism. Cosinor model is a popular tool for analyzing biological rhythms. It allows representing the rhythm by its MESOR (midline estimating statistic of rhythm), amplitude, and phase [4].

It has been reported that removing circadian trends from HRV can improve stress detection [5]. Based on this evidence, Cosinor features were utilized in the beginning of this research. The goal was to evaluate whether temporal information could help to infer multi-level stress from wearable RR intervals.

However, the study did not end at the observation of improved classification results. Additional research was conducted to explore the reasons behind the improvement. It was necessary since WESAD recordings cover only the short period of time within 24 hours cycle. Thus, the additional analysis was performed to establish whether the achieved improvement is actually caused by circadian correction or some other factor.

Then, the research proceeded from the field of feature engineering to the analysis of different models. Both machine learning and deep learning approaches were studied. The latter has become widely used in physiological stress detection due to its ability to learn complex time dependencies from physiological signals [6]. Within this research XGBoost, BiLSTM, and causal neural models were compared in different experimental setups.

Finally, the next step of the research was the analysis of differences between offline and real-time evaluation of models. Offline experiment allows processing of the entire recording. However, the real-time experiment cannot use the information that is not observed yet. Thus, every prediction should be generated based on past and present information.

This necessitated the execution of a number of experiments. They involved causal feature processing, window labelling, model architecture, cold-start, confidence-

aware, and short-term forecasting. These experiments were used to investigate the impact of deployment limitations on the performance of the stress inference system.

The study also explored the issue of correlation between model accuracy and usability. A model can score slightly better in terms of accuracy while being unsuitable for use in practice. For instance, a temporal model may rely on knowledge of subsequent points in a sequence, which is not available at the time of making predictions. Therefore, the choice of a model had to take into account both prediction accuracy and causality.

The final inference system was implemented in a Python-based backend. The backend collects the physiological data from the wearable system and uses a real-time inference pipeline to process it. The system classifies the user's stress into one of four stress levels: relaxed, mild, moderate, and high. A confidence value is also estimated. This data is then available for use by the adaptive VR application and other components of the overall system.

This research component is thus not limited to just the development of a stress classifier. It explores how issues relating to temporal processing, model design, labeling, and deployment considerations influence HRV-based stress detection. It also considers whether the improvements achieved in the offline study hold any ground in the case where the model is deployed in a real-time scenario. The study integrates the analysis of physiological characteristics, machine learning techniques, methodological appraisal, and practical applications in one research element.

1.3 Problem statement

Wearable stress detection has proven effective in experimental settings. HRV-based techniques have been shown to detect physiological changes due to stress [1], [2]. Multilevel stress classification techniques have also been used by combining HRV characteristics and machine learning models [3].

The problem is that many stress detection experiments are based on offline experiments. In this setting, the entire recording is already present while the data processing takes place. This does not mirror the environment of a live system, in

which the information available at the moment of prediction is limited to that which has already been recorded [1].

Such differences in environments may affect the validity of the reported results. Some feature extraction techniques may require access to information from the entire recording. The same applies to temporal models, which depend on latter part of information from the sequence. These methods may perform well during offline testing but may not be suitable for live inference.

The interpretation of temporal information also raises problems. Some research has found an increase in HRV prediction accuracy after eliminating the circadian tendencies [5]. However, a short recording may not have enough information to be representative of a full-day cycle. Consequently, any improvements found may not be due to the mechanism that was thought responsible.

The construction of windows and their labelling might also be a source of errors. A label placed for the midpoint of a window may correspond to a previous period compared to some of the physiological values used in training the model. In that case, there will be some additional information which would not be available in a real-time system. That difference may cause the model to perform better in offline mode than online.

A real-time stress inference system faces practical considerations too. It should generate predictions fast enough. It should work even when a new user has no historical data at all. It should also deal with uncertain predictions and changing physiological patterns.

Hence, the issue is not just about finding out how to achieve maximum accuracy in stress classification. Instead, the issue is about how to create a multi-level inference algorithm based on HRV that still holds its validity in the case of real-time deployment.

The problem studied in this work can be formulated in the following way:

How can multi-level stress be inferred from wearable RR interval data so that it retains its predictive value and the validity of the inference process in the case of real-time deployment?

The proposed solution to the problem includes investigation of temporal feature processing, model architecture, labelling, causal inference, cold-start behaviour, confidence-aware output, and forecasting. The objective is to find out the method that is sufficiently accurate for stress assessment and can be used in the entire real-time system.

1.4 Research gap

Detection of stress through wearable devices has been extensively researched with the use of physiological data and machine learning. Nonetheless, there are some limitations in the transition of such methods from an offline experimental framework to a real-time system [1].

Firstly, one of the limitations pertains to the issue of deployment validity. The majority of studies assess their models based on recordings that are already complete. Such practice makes all the data available for processing. A real-time system cannot make any use of future data. Lack of attention to the practical aspects of deployment in wearable stress detection research was noted in previous reviews [1]. Hence, even good offline performance does not guarantee real-time applicability of a model.

Secondly, the limitation pertaining to the processing of temporal features should be mentioned. Earlier studies showed better performance of stress classification when circadian trend was removed from HRV data [5]. Nonetheless, modeling of a circadian rhythm requires a certain degree of daily cycle coverage [4]. It is unclear if short laboratory recordings give enough data for proper detection of a circadian rhythm.

Third, there is a problem related to window labeling and alignment in time. Physiological data are often split into windows prior to classification, and where in the window the label is defined influences the evaluation. In case the label corresponds to a time that is earlier within the window, the later physiological data may, in fact, influence the prediction made concerning the earlier time. This will lead to a positive bias in the performance estimation.

Another research gap involves the relation between model accuracy and its readiness for deployment. Deep learning can take into account the temporal structure of the physiological data [6]. Yet some temporal structures may allow the data to influence predictions in both directions. This is alright when the system is analyzed off-line but poses problems for causal usage.

Another problem is the behavior of the system while being used. A newly registered user may not possess enough history to calibrate the system for their needs. In addition, the predictions may have varying confidence. Finally, a stress mitigation system can use predictions of the change of stress in the future. Such circumstances are often overlooked when only classification performance scores are used to assess stress models [1].

There is therefore a need for stress inference method that would be tested under the same information constraints as the ones it is going to operate under later. The impact of the temporal HRV features construction, labeling, model architecture, cold start, confidence, and forecasting should be assessed simultaneously and not separately.

The proposed research will address the issue by conducting research on multi-level HRV stress inference under real-time deployment constraints. It will study the cause of the observed positive impact of HRV temporal processing. It will assess the effect of different deployment-related decisions on model performance. The final aim is to come up with a method of stress inference that would be applicable for classification as well as for practical implementation.

1.5 Aim and objectives

1.5.1 Aim

The main aim of this research study is to design and test a multi-level stress inference method based on wearables RR intervals. The proposed model should have good prediction performance and be capable of working in real-time.

Additionally, the study is intended to determine how temporal processing and deployment requirements influence the reported accuracy of HRV stress models.

Specifically, it is worth investigating if the obtained accuracy gains are relevant under the condition that only past and present data are used.

1.5.2 Objective

The specific objectives of this research are:

1. To extract and analyze HRV information contained in RR intervals for multi-level stress inference.
2. To explore the application of Cosinor-based temporal features to improve the HRV-based stress classification accuracy.
3. To determine if the improved performance resulting from Cosinor-based pre-processing is due to the circadian correction or other temporal processing.
4. To design and compare various machine learning and deep learning strategies for multi-level stress inference.
5. To evaluate the influence of causal and non-causal processing on performance of models.
6. To study the influence of window length and labelling location on stress classification offline and in real-time.
7. To test the performance of the designed inference system under practical operating conditions, including cold start, short user history, confidence-aware output and short-term prediction.
8. To select the stress inference method that offers appropriate trade-off between performance and real-time deployment requirements.
9. To incorporate the stress inference strategy into the backend of the entire system written in Python.
10. To generate real-time stress predictions (relaxed, mild, moderate or high) and confidence level to be utilized by the adaptive VR application.

1.6 Research questions or hypotheses

The research does not have hypotheses but has questions formulated for the purpose of investigation. The process of the study has become more exploratory along the way with experiments being conducted. Specifically, the study has been evolving

from testing whether Cosinor features are helpful for the classification process to finding out why and whether such an improvement will still be there in real time.

The research questions that guide this research are:

RQ1: How well are wearable RR intervals and HRV features able to classify stress in four levels including relaxed, mild, moderate, and high stress?

RQ2: Does using Cosinor features for temporal features improve the inference of multi-level HRV stress, and how does it happen?

RQ3: Which models including machine learning and deep learning are best at the multi-level HRV stress inference?

RQ4: In what way does the choice of deployment parameters, including window size and labeling position, influence the performance and latency measures of the stress inference system?

RQ5: In what way do real-world conditions of deployment, including cold start behavior, limited historical information about users, confidence-based predictions, personalization, and short-term forecasts, impact the utility of the inference system?

RQ6: Which particular configuration of the stress inference process provides the right tradeoff between the effectiveness, safety, and real-time deployment of the adaptive VR system?

These research questions represent the final direction of the work as well as the progression that the work made from the initial hypothesis. Consequently, the research not only analyzes the performance but also the validity of the inference process under real-time conditions.

1.7 Scope limitations and assumptions

The current research is concerned with the problem of multi-level stress inference based on physiological data from wearables. The primary objective of the research is to classify the four stress levels: relaxed, mild, moderate, and high. Moreover, the current research aims at investigating the feasibility of the inference process under real-time conditions.

The WESAD dataset is chosen as the main benchmark dataset for experiments. It consists of physiological recordings of fifteen subjects. The experiment procedure involves such conditions as baseline, stress, amusement, and meditation [2]. The dataset includes cardiac data and wrist skin temperature measurements.

The primary cardiac input is the RR interval. The RR intervals are used for calculating the time-domain and frequency-domain HRV characteristics. Another physiological input taken into consideration is the skin temperature. HRV serves as the principal physiological feature for the stress detection, while skin temperature is investigated in the process of the temporal modeling and physiological analysis. During the temporal modeling, Cosinor models were applied to both HRV and skin temperature data.

The research focuses on the analysis of the Cosinor-based temporal modeling. However, the analysis revealed that the average recording length in the WESAD database is around 1.5 hours which constitutes just about 6.2% of the 24-hour period. Thus, this research does not make any assumptions about the possibility of detecting the full circadian rhythm within such recordings. Instead, it investigates the information obtained through the Cosinor modeling.

Deployment experiments were performed using the WESAD database. It brings about a significant limitation in that the database consists of fifteen participants. It means that small performance differences need to be interpreted cautiously since some of the experiments have low statistical power. The results may differ when other populations, sensors, and recording environments are used.

Another limitation lies in the high overlap between the windows used in the model evaluation. It means that neighbouring windows are not independent of each other. Subject-level evaluation and leave-one-subject-out validation are used to adress this limitation.

There is also a limitation regarding the four-level stress labels which were not clinically validated in the WESAD. It means that in this research the experimental conditions are adjusted for four-level inference of the stress. The stress condition is

split using local HRV variability. It means that these four levels of stress are considered as research labels.

Assumption is made that the physiological signals provided as inputs to the inference framework are pre-processed according to the needed criteria. Timestamps are also assumed to be available and necessary for the generation of temporal features and real-time processing.

It is also assumed that, for the deployment of the system, a prediction can use only the information available at or prior to the prediction point of time. This assumption is the basis of the causal inference framework used in this research. Physiological samples in the future are considered as unavailable. A prediction cannot make any assumptions on the future or on a complete user session.

Thus, the range of this work can be defined as the implementation of a multilevel system of stress estimation which is based mostly on HRV features extracted from RR intervals and also includes temperature as another physiological signal. The work does not seek to offer any clinical stress diagnosis. It also does not try to calculate a 24 hour cycle of a circadian rhythm from short laboratory records.

1.8 Individual contribution

The individual contribution made by this research is the creation and analysis of the stress inference layer in the overall adaptive VR environment. The contribution includes physiological features processing, temporal modeling, machine learning, methodology analysis, implementation evaluation, and real-time implementation.

The first individual contribution made by this research is the creation of a four-level stress inference pipeline. The pipeline processes the RR intervals to extract HRV features. Temperature is used as an additional physiological measure for temporal modeling. The state inferred by the system is one out of four: relaxed, mild, moderate or high.

The second individual contribution is the investigation of Cosinor-enhanced temporal modeling. Originally the research direction was focused on application of Cosinor rhythmometry to modeling the time-related variation in physiological parameters.

Cosinor models have been created for HRV and skin temperature signals. However, later experiments have showed that the effect detected in short session data cannot be attributed to 24-hours circadian rhythm.

The research then proceeded to decompose the effect of the temporal features. It was found that most of the effect was attributed to the residuals rather than the estimated circadian baseline. This means that the effect of the method was mostly on within-session baseline normalisations than on the circadian correction effect. This became an important methodological contribution of the research.

Thirdly, the development and comparison of various machine learning and deep learning methods were developed and compared. These are XGBoost, BiLSTM, and MS-CGCA based models. Two-way and three-way architectures were considered in the comparison. The comparisons were made based on predictive performance, variations between training seeds, and suitability for real-time application.

Fourthly, there was an exploration of the limitations of deploying a model trained offline in real-time setting. Various factors such as window size and label location were considered in the study. Personalization, cold start initialization, confidence-aware predictions, and short-term predictions were considered.

The fifth contribution is the causality-aware real-time inference algorithm. In the final implementation, only historical data until the point of prediction is utilized. No future physiological samples are needed. In addition, the system utilizes endpoint-based window labeling and causality-aware baseline which can work when the new user appears in the system.

The sixth contribution is the trade-off between the quality of predictions and the safety of deployment. Model with a slight improvement in offline prediction cannot be selected automatically for deployment. The final decision is based on whether the model can still remain causality-aware even when there are changes in the rest of the pipeline. This choice results in the configuration which provides a trade-off between performance and deployment safety.

The seventh contribution is the integration of stress inference pipeline into the full adaptive VR system. The back-end processes incoming physiological data and

generates the stress inference result along with confidence measure. It passes the result to the adaptive VR application which can adapt the meditation environment based on the inferred state. This stress inference process is considered as the connection between physiological sensing and adaptive interventions in the original system design.

In summary, the personal contribution from this work includes both the research results and one aspect of a working system. The research details the impact of temporal assumptions, physiological baseline measurements, modeling techniques, and evaluation methods on stress estimation. This is then applied to the real-time inference process of the adaptive virtual reality system.

1.9 Thesis structure

The thesis is composed of eight chapters. Every chapter covers one of the phases of the research and development process.

Chapter 1 is an introduction to the general project and the research element in particular. In this chapter, there is presented the problem statement, research gap, objective, goals, research questions, scope, limitations, and individual contribution.

Chapter 2 is a review of the literature on topics of wearable stress detection, heart rate variability analysis, physiological monitoring based on body temperature, multilevel stress classification, Cosinor rhythmometry, deep learning, and stress recognition in real time.

Chapter 3 provides details of the research methodology. It includes information about datasets, physiological signals, feature extraction, temporal modeling, model creation, training procedure, performance metrics, statistical analyses, and reproducibility.

Chapter 4 outlines the design of the stress prediction solution. It includes information about the requirements, internal structure, design choices, causal processing paradigm, and integration into the overall adaptive virtual reality system.

Chapter 5 outlines the implementation, integration, and testing procedure. It includes information about the development environment, models implementation, backend pipeline development, integration process, software testing, and deployment process.

Chapter 6 shows the results of the research and technical investigation. It includes comparison of models, analysis of the Cosinor mechanism, causal processing evaluation, labelling tests, cold-start analysis, confidence-based inference, forecasting results, and deployment performance.

Chapter 7 includes the discussion of the findings. It includes a comparison of the findings to the state-of-the-art, research contributions, limitations, potential threats to validity, and implications.

Chapter 8 concludes the thesis. It reviews the achievement of the research objectives, summarizes the individual research and product contributions, and presents possible directions for future work.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The literature review was done in order to find relevant literature regarding the design and assessment of the proposed component. The literature was chosen according to specific areas that include wearable stress detection, physiological signal analysis, temporal modeling, multi-level stress classification, machine learning, deep learning, and real-time deployment.

The main source of literature was the set of research papers reviewed during the course of the project. Attention was paid to research papers discussing RR intervals, heart rate variability (HRV), skin temperature, stress classification, Cosinor rhythmometry, and wearable stress detection. Other references found earlier during the course of the research process were included in the review if they provided useful material in the area of temporal modeling, causality processing, and deployment evaluation.

The literature review started with research on wearable stress detection. Wearables can collect physiological data in the longitudinal way, which allows continuous stress estimation. The WESAD database is commonly used in wearable stress detection due to the presence of various physiological signals collected under different affective states [2]. According to systematic reviews, wearable stress detection developed quickly, but it lacks generalizability to different people and environments [1].

Another important sphere is focused on RR-intervals and HRV (heart rate variability). Indeed, RR-intervals form the basis for HRV feature extraction and HRV is widely used for the purpose of stress recognition owing to the fact that changes in the cardiac cycle reflect the variations of autonomic nervous system activity. Previous literature has also shown that information obtained from HRV can be helpful for the classification of individuals to more than two stress categories [3].

This review also takes into account skin temperature, which is available as the additional physiological signal in WESAD [2]. In this paper, temperature is mainly

considered as the supplementary signal in the course of temporal analysis and not the main source for the purpose of stress classification.

The next very important issue is multilevel stress classification. Indeed, Mortensen et al. have shown that HRV can be used for the purpose of multiclass stress recognition using deep neural network models [3]. At the same time, Hemakom et al. studied multilevel stress classification using physiological data and machine learning algorithms [7]. These studies support the use of several stress levels rather than only stress and non-stress states.

The review further discussed the topic of temporal physiological modeling. The Cosinor rhythmometry was used in the review since it allows for the mathematical representation of periodic biological signals [4]. Special attention was paid to Shen et al.'s works showing improvement of HRV-based stress assessment after eliminating circadian variations [5]. The study contributed to the early temporal modeling step of the current research.

Deep learning techniques were considered since physiological stress data possess temporal relationships which cannot be fully accounted for in feature vectors. CNNs, LSTMs, and hybrid techniques are widely used for physiological stress recognition [6].

With further progression of the inquiry, the scope of the review expanded to the problem of generalization and real-time implementation. Benchekroun et al. showed that the performance of HRV-based stress models is likely to vary significantly while transferring between different datasets [8]. It was stressed that besides the model accuracy, its robustness should be taken into consideration.

The literature review includes three distinct but interconnected aspects. The first aspect concerns the inference of physiological stress using HRV and body temperature readings. The second is related to the temporal modeling and multilevel classification of stress. The third is concerned with the assessment of validity of the proposed methods in a practical real-world setting.

2.2 Domain and theoretical foundations

2.2.1 Wearable Physiological Stress Inference

Stress inference with wearable sensors aims at inferring the stress levels of the individual through the analysis of physiological metrics recorded using the wearable sensors. Wearable sensors provide a means for collecting continuous data from individuals throughout the session, hence providing an alternate source of data collection other than the stress questionnaire forms.

WESAD is an example of the use of various physiological metrics gathered using wearables to determine stress and affect [2]. The dataset consists of signals collected from the chest (chest ECG) and wrist (Blood Volume Pulse, EDA, Temperature, and Acceleration).

The main signals used in this study include heart rate variability from the RR intervals and temperature data. HRV is the main cardiac measurement used for stress inference, whereas temperature is an additional physiological signal used, especially in temporal analysis.

2.2.2 RR Intervals and Heart Rate Variability

RR Interval indicates the time gap between two consecutive beats. With the help of RR intervals, heart rate variability (HRV) can be calculated. HRV describes the variation in the timing between heartbeats and has wide usage in the study of autonomic functions.

The various types of features which may be used to characterize HRV include time domain features such as mean RR interval, standard deviation of normal-to-normal intervals (SDNN) and root mean square of successive differences (RMSSD). HRV is also analyzed in the frequency domain to analyze the distribution of the variance across different frequencies.

Standardized protocols for measuring, interpreting, and presenting HRV have been developed by the Task Force of the European Society of Cardiology and the North American Society of Pacing and Electrophysiology [9].

HRV features have found wide application in stress recognition. Mortensen et al. have shown that HRV data can be used not only for the detection of stress but also for multiclass stress recognition [3].

2.2.3 Skin Temperature

Skin Temperature represents another physiological signal that was considered in this study. The WESAD data contains skin temperature of the wrist in addition to other wearable signals such as heart rate variability [2].

In this study, the temperature signal plays a role mainly in the temporal analysis. Temporal analysis is carried out for HRV and temperature using cosinor modeling, thus creating another physiological signal.

2.2.4 Multi-Level Stress Representation

Detection of stress often comes down to a binary classification of being stressed or not stressed. This may, however, limit the amount of information that an adaptive system would need to use when reacting to different levels of stress.

Previous literature has investigated stress detection in multi-class fashion using physiological measurements and machine learning. In particular, Mortensen et al. have investigated stress detection based on HRV and deep neural networks [3], while Hemakom et al. have considered multilevel physiological stress classification using several machine-learning methods [7]. Altogether, these papers show that physiological data is capable of representing stress in terms more complex than just two classes.

In the current research, four classes will be used: relaxed, mild, moderate, and high. The purpose of these classes is to provide a more detailed measure of stress for the adaptive VR system.

It is important to note that these classes are research classifications and not clinical diagnoses. Their main goal is to allow the inference of different levels of stress by the adaptive system.

2.2.5 Cosinor Rhythmometry

There are certain elements that might influence the measurements independent of stress. One of these elements is the circadian rhythm, which is a daily biological rhythm. In the absence of consideration of these changes, the model might incorrectly identify a biological rhythm as being associated with stress.

The cosinor rhythmometry is a mathematical method used in analyzing periodic biological signals [4]. A basic single component Cosinor model is given by:

$$\boldsymbol{Y(t) = M + A\cos\left(\frac{2\pi t}{\tau} + \phi\right) + \epsilon(t)} \tag{2.1}$$

where M is the MESOR, A is the amplitude, φ is the acrophase, and τ is the supposed period [4].

MESOR is a measure of the central level of the fitted rhythm; amplitude is the size of the oscillation; acrophase is the time of the maximum value. The residual is the difference between the measured physiological and fitted values.

Improved HRV-based stress estimation was obtained by Shen et al. upon elimination of the circadian component of the HRV signal [5]; thus, the Cosinor-based temporal processing served as the initial inspiration for this work. Yet, the fit of the mathematical rhythm does not necessarily mean identification of the physiological rhythm; the validity of the fitted rhythm relies on the amount of temporal information available, especially since the benchmark data used in this work consists of short recordings.

2.2.6 Machine Learning and Deep Learning

Machine Learning techniques can be used to detect the relationships between physiological features and states of stress. Gradient-boosted decision tree, for example, is a good choice for learning from structured physiological features. Deep-learning algorithms expand this scope to learn the relationships between physiological sequences. Local dependencies can be learned by CNNs, and temporal dependencies by recurrent neural networks like LSTMs. The review of deep-learning

based stress detection shows that convolutional, recurrent, and hybrid architectures are widely used in this area [6].

The representation of features is important as well. Xiang et al. investigated deep-learning methods that use both time and frequency domains of physiological data [10] which means that different representations of physiological signals may carry useful additional information about stress for classifiers. Quoy et al. considered stress detection using CNNs with HRV-based data [11].

2.2.7 Causality in Real-Time Stress Inference

Another theoretical requirement of this study is causality. In an offline experiment, one can analyse a recording after completion of a whole session, whereas in real time, the system does not have information about future measurements. Given that prediction has been made at time *t*, causal inference requires only information from before time *t*; the prediction should be based only on available information, and future samples cannot affect the prediction.

This limitation affects various parts of the inference chain, such as feature generation, normalisation, baseline calculation, windows creation, windows labelling, and model design. In this particular case, this difference becomes important since the stress estimate will affect an adaptive VR system in real time. Therefore, good offline classification results alone are not enough; the entire inference chain must be valid according to the information available in real time.

The theoretical approach used in this research thus incorporates HRV analysis, temperature data, multi-level representation of stress, temporal rhythms modelling, machine learning, deep learning, and causal real-time inference.

2.3 Related research

Wearable devices have been used for stress detection by leveraging different physiological parameters, feature extraction methods, and machine learning models. There is ample literature available on automatic stress inference using wearable

devices; however, some concerns such as those regarding generalization, temporal interpretation, and practicality have been identified by the research community.

WESAD, an open-access database for wearable emotion and stress analysis, was created by Schmidt et al. [2]. It consists of data collected from fifteen subjects under baseline, stress, amusement, and meditation states. It contains ECG data recorded on the chest, along with data collected from different sensors on the wrist. WESAD serves as a benchmark for automatic stress inference using wearable devices.

However, superior performance on a single benchmark dataset does not necessarily imply the same effectiveness for different users and environments. According to Vos et al., generalization is an important issue for wearable stress detection systems [1].

Benchekroun et al. tackle this problem in their cross-dataset study of HRV-based stress-detection models [8]. The results show that performance may change depending on the conditions in which the evaluation of a model is performed. This problem is especially important for stress detectors developed for practical applications, because users and the environment where the sensors operate might differ from the training set.

There have been several advances made beyond the binary classification of stress. Mortensen et al. researched multi-class stress classification based on HRV and deep neural networks [3], showing that the cardiac data could be used for classification into several classes of stress. Hemakom et al. investigated multilevel stress classification using physiological measurements [7], developing methods using machine learning and applying them to ECG and EEG signals. All these papers suggest that stress states should be characterized with more detail than just two classes.

Deep learning has become increasingly popular in the study of stress with physiological measurements. Kyrou et al. reviewed deep learning algorithms applied to stress detection and described different types of these algorithms: based on CNNs, recurrent networks, and hybrids of the mentioned ones [6]. These methods are relevant when physiological information is represented as a sequence.

Quoy et al. have researched the detection of stress using HRV in a CNN architecture [11]. The authors concluded that the use of convolutional networks allows for the extraction of information about stress from HRV features.

Xiang et al. have studied a multi-modal deep learning system for the fusion of time-domain and frequency-domain physiological features [10]. It has been shown that complementary physiological features can improve the amount of information available for the stress classifier.

The issue of temporal modeling is one more important aspect of this field. Cornelissen formulated the main concepts of Cosinor rhythmometry as applied to the analysis of rhythmic physiological data [4], which provides a set of parameters such as MESOR, amplitude, and acrophase to describe rhythmic physiological processes.

Next, Shen et al. have developed circadian processing techniques for HRV-based stress assessment [5]. These authors showed improvements in stress classification after the elimination of circadian component from HRV data. This result directly motivated the early temporal modelling stage of the present research.

Temporal coverage of the data is an essential aspect for the interpretation of the fitted rhythm. Shen et al. studied physiological signals in different moments of the day [5]. In turn, WESAD is based on relatively short sessions in a lab [2]. Therefore, a technique that would be able to fit the daily rhythm into long data may give a different interpretation of physiology in case of a short session.

Previous work on the subject focused on classification performance as well. Measures like accuracy or F1-score allow for the comparison of models. However, those measures do not necessarily define the ability of the same model to function properly in real-time mode.

A system for stress inference has certain requirements as well. First, it cannot use any future physiological readings. It might lack information about a user at the beginning of the inference procedure. Moreover, it should make predictions within an acceptable time range. Thus, an offline stress classifier and a real-time inference system have an important difference.

In general, previous research shows that it is possible to use physiological signals in detecting stress and that HRV can be used in multi-level classification. There is also prior research that presents approaches in temporal modeling, feature extraction, and deep learning. However, it is still needed to determine whether the gains made during offline experiments hold up when considered together with temporal information. It is the main connection between past research and this thesis.

2.4 Existing systems products and techniques

Existing technologies in the field of stress detection using wearables can be classified into five main categories: consumer-based wearable stress detection, traditional HRV feature-based classification, deep learning techniques, time-based modeling, and continuous/real-life deployable systems.

2.4.1 Wearable and Consumer Monitoring

Consumer wearable devices offer continuous monitoring of physiological parameters. One of the available parameters that can be collected using consumer wearables is heart rate variability (HRV). HRV can be monitored longitudinally to study changes in cardiac performance. Researchers have studied resting heart rate variability measured using consumer wearables in several longitudinal studies showing the possibility of using consumer wearables for measuring heart rate variability repeatedly not only in laboratory conditions [12].

Still, collecting physiological parameters and inferring stress level are two different things. One can measure heart rate variability using wearable devices without determining the stress level of the user. This means that building a stress inference system involves extra stages, feature extraction and classification.

In this work, wearable devices are primarily employed for physiological parameter collection. Stress level inference is performed in the back end, not on the wearable or VR headset.

2.4.2 HRV Feature-Based Stress Detection

One of the popular ways for detecting stress is using heart rate variability (HRV) feature extraction from cardiac signals and providing them to the machine learning

classifier. The most widely used features are RMSSD, SDNN, average RR, and different frequency domain features [9].

It should be noted that such an approach allows extracting structured features which can be used by traditional classifiers such as gradient-boosted decision trees. Moreover, it provides a possibility of analyzing each feature's contribution better than in the case of full end-to-end neural network architecture.

However, the quality of features plays a big role in the final performance of the model. The works of Chen et al. focused on HRV features augmentation with transformations of the heart rate signal [13]. Their work shows that the representation and augmentation of HRV information can influence stress-detection performance.

2.4.3 Deep-Learning Techniques

Deep learning provides another approach to physiological stress detection. Convolutional neural networks are proficient at detecting local patterns in the physiological representation, whereas recurrent networks detect temporal relations in the sequence [6].

Quoy et al. studied the detection of stress based on CNN using HRV data [11]. Xiang et al. combined time and frequency domain information about physiological signals in their deep learning model [10]. All these works demonstrate the ability of neural networks to detect stress patterns based on different types of physiological signals.

Though deep models provide excellent predictive accuracy, they also bring some extra complexity. The behavior of the model is highly dependent on the way the sequence is built, labeling, and availability of the future part of the sequence during the prediction process. These issues become particularly important when moving from offline experiments to real-time deployment.

2.4.4 Temporal Baseline Modelling

Another possible solution would be to build the model of the expected baseline prior to classification. Cosinor rhythmometry provides one possible methodology for describing the periodic variations [4].

Shen et al. removed circadian trends from HRV features using the Smoothness Priors Approach, applied to recordings taken in the morning, at noon and in the evening, and found that classification of stress states improved after the estimated daily variation was removed [5]. Their recordings covered several times of day, so a daily trend was observable in their data. It follows that there may be valuable information encoded in temporal context which a non-temporal classifier ignores.

However, the temporal model should fit the dataset. If the objective is to model a 24-hour rhythm, there needs to be enough temporal data to justify that assumption. Laboratory recordings with limited temporal data pose a major constraint in this regard.

Therefore, this study considers temporal detrending and circadian correction to not necessarily mean the same thing. Instead, temporal characteristics are evaluated to find out the real information content.

2.4.5 Multi-Level Stress Inference

Some of the stress detection systems provide output in binary form. However, many researchers have investigated multiclass stress classification problem. Mortensen et al. have shown multiclass stress detection using HRV features together with deep neural network [3]. Hemakom et al. have also studied multilevel stress classification using physiological measurements [7].

Multi-level output becomes especially valuable in adaptive systems, as they may react in a different manner to different states predicted by the system. Thus, in our system, we generate 4 stress levels instead of binary classification.

2.4.6 Real-Time and Deployable Techniques

Another category of methods involves working in an environment other than controlled offline experiments. Generalization was one of the issues pointed out by Vos et al. in relation to wearable stress detection [1]. Another paper by Benchekroun et al. shown the potential of changes in the performance of HRV-based models depending on the dataset used [8].

Also, a deployable system should be able to operate based on the current data only. The system cannot use future information as well as the session-level statistics. Thus,

there appear additional demands in terms of normalization, baseline estimation, window generation, model architecture, and latency.

The existing methods thus provide some useful pieces in the solution of the task at hand, however, not even one of them solves the entire problem discussed in this thesis. The present research combines physiological feature processing, temporal analysis, multi-level classification, causal processing, and real-time backend deployment.

2.5 Gap synthesis and conceptual framework

In this regard, the literature indicates that physiological signals worn by a person can be used for stress detection. The use of HRV is popular in this respect; some papers provide evidence about both binary and multi-class classifications of stress state[2], [3], [7]. The use of deep learning methods has also contributed to improving the capacity of learning complex dependencies [6], [10], [11].

However, there are some related gaps identified in the literature.

Firstly, there is a gap regarding temporal interpretation. The authors of [5] indicated that HRV-based stress detection became more effective after excluding circadian variability. The method of cosinor rhythmometry allows estimating periodic behavior mathematically [4]. However, such an interpretation is conditioned by sufficient temporal coverage. In short-session benchmarking, one may receive parameters fitted but not enough information to detect the existence of 24-hour physiological rhythm. Hence, the underlying reason for performance improvement needs to be tested.

The second gap pertains to generalization. The stress models may work well in the case of the data they were developed from; however, they may demonstrate worse performance in case the user group, sensor or environment differs [1], [8]. Thus, a model should not be judged only by its best score on a single benchmark.

The third gap pertains to the distinction between the offline testing and real-time inference. Offline analysis has access to the complete recording. The real-time prediction may make use of no more information than what is known at the current moment. Thus, feature normalization, sequence construction, labelling and model

architecture may lead to the overly optimistic results due to the inclusion of the future information in prediction.

The fourth gap pertains to the actual deployment conditions. The real-time prediction system may need to start working with an unknown user. The predictions should also be produced timely, and uncertainty of the predictions should be communicated. These issues are not fully represented by standard classification metrics alone.

The last gap is related to the relationship between multi-level physiological inference and adaptive application. The existing literature has shown that there are different levels of stress that could be classified [3], [7]; however, for an adaptive application, a reliable interface, confidence information, and causal predictions need to be established during the process.

The conceptual framework for this research can be identified with five stages:

Wearable physiological input → physiological and temporal feature processing → multi-level stress modelling → deployment-aware causal evaluation → real-time stress output for adaptive VR.

In the first stage, we have the input of RR intervals and temperature data obtained by the wearable sensors. The former are used to extract HRV characteristics whereas the latter add additional physiological information.

In the second stage, we use these signals with standard HRV characteristics as well as temporal data. At this stage, we evaluate cosinor-based characteristics and study their influence to see whether the benefit obtained here is due to circadian correction or something else.

The third stage deals with the application of machine-learning and deep learning approaches to estimating the user's stress level. Here, different architectures are compared instead of supposing that one class of models is better than the other one.

At the fourth stage, real-time limitations are taken into account. These include such problems as causal feature building, endpoint-based labelling, window length choice, cold start, confidence-based inference, and other deployment-related issues. The

purpose is to determine whether performance observed offline remains valid during live operation.

At the final stage, one of four categories is produced: relaxed, mild, moderate, or high along with confidence data. This output goes into the adaptive VR system in order for the environment to react to the state of the user.

Therefore, the theoretical framework helps connect the research problem with the end product. This work does not seek merely to achieve good results when measuring stress through classification scores. This work seeks to create a stress inference system that will still have a valid performance score when put in a live system.

3 RESEARCH METHODOLOGY

3.1 Research design

This research involves a design-and-evaluate strategy that includes the design of a stress inference system as well as its evaluation using experiments. Unlike in some other researches where the goal is simply to develop a classifier, this research attempts to check the validity of the assumptions made when designing the model once the latter is implemented.

Several consecutive steps were taken within the scope of the research. First of all, the multi-level stress inference pipeline that utilized RR interval-based HRV features as well as temperature readings was developed. The primary dataset used in this part of the study was WESAD that allows for recording of the physiology under controlled affective conditions [2].

Secondly, temporal modeling of physiology was explored. Cosinor features were introduced as a means of describing the variations in physiological signals over time. The reason for choosing this direction was the previous reports of improved stress classification after the circadian correction of HRV [5].

Nevertheless, this research design assumed that the improvement of the performance does not necessarily mean that the circadian correction is the cause. In order to test this hypothesis, experiments were conducted to separate the contribution of the fitted temporal baseline, residual features and other information. Different assumed periods were also compared to determine whether the improvement depended specifically on a 24-hour cycle.

Thirdly, this research focused was about comparing the models. Classic machine learning methods and sequence-based deep learning approaches were studied. This included gradient-boosted trees, bidirectional sequence models and causal neural networks. The idea was to see if there were any gains in terms of improvement due to increased model complexity.

Fourthly, deployment conditions were taken into account. Offline processing provides the full recording to work with. A deployed system doesn't have access to this data. It means that in the later experiments the predictions had to rely only on past and current physiological data.

Several deployment-related aspects were considered. These include window size, window labeling, causal baseline prediction, cold-start operation, personalization, confidence-based prediction and short-term prediction. The goal was to measure the impact of these design choices on the performance and feasibility of the system.

Evaluation was done subject-independently since wearable stress models might show performance variation when applied to new subjects and other datasets [1], [8]. Thus, leave-one-subject-out evaluation was used for comparison purposes.

Moreover, a secondary dataset was used to determine whether the temporal discovery is specific to WESAD. The purpose of this step was to perform a cross-dataset analysis based on physiological measurements made by another wearable sensor.

Finally, choosing a model architecture that should be incorporated into the real-time inference pipeline was done taking into account factors beyond just the highest possible classification rate. Causal safety, latency, cold start behavior, implementation difficulties, and compatibility with the full-scale adaptive VR system were taken into account.

Thus, the general structure of research followed the scheme:

physiological data preparation → temporal feature investigation → model development and comparison → mechanism analysis → deployment-aware evaluation → real-time system integration.

It allowed this research to study not only whether a certain technique improves results, but also why it works. Furthermore, it allowed evaluation of the selected model in an environment similar to that in real time.

3.2 Data participants or study materials

3.2.1 WESAD Dataset

The primary dataset is the Wearable Stress and Affect Detection (WESAD) dataset [2]. The WESAD dataset consists of the physiological data of fifteen subjects measured during an experimental protocol.

There are multiple conditions within the protocol, namely, baseline, stress, amusement, and meditation [2]. These conditions serve as labels for the physiological data, allowing the effects of stress on physiological state to be examined. The stress condition was induced using a modified Trier Social Stress Test [14].

Chest ECG is recorded with the frequency of 700 Hz. Measurements from wrist sensor also include signals such as blood volume pulse, electrodermal activity, acceleration, and skin temperature [2]. The current component uses heart data obtained from ECG, as well as skin temperature.

The intervals between R-peaks in the cardiac signal are used for computing RR intervals. HRV is calculated based on RR intervals. Standard measures and features for HRV calculation follow the measurement guidelines [14].

Time- and frequency-based measures of HRV are used in the research. Mean RR, RMSSD, SDNN, and frequency measures are examples. The skin temperature is an additional input from the physiological perspective especially during temporal HRV experiments.

3.2.2 Stress-Level Construction

The initial labeling of the WESAD dataset did not include the four labels for the classification performed by the final model. Hence, the labels were modified to facilitate multi-level stress classification.

The stress state was represented by three levels of stress intensity according to the short-term physiological variability of HRV. Together with the relaxed state, these yielded four ordinal experimental labels:

Relaxed → Mild → Moderate → High

This set of labels is used for further model development and comparison in the research and represents experimental stress states rather than clinical diagnoses.

Since the construction of the intensity involves the same physiological variability which is used for constructing the models, such labeling is considered to be a methodological limitation of the research. Consequently, the same label definition is preserved in the major comparisons.

3.2.3 Secondary Wearable Dataset

The second physiological data set used for the replication of temporal analysis between different datasets was provided by Hongn et al. [15] and was recorded using the Empatica E4 wearable sensor.

This dataset includes blood volume pulse, skin temperature, electrodermal activity, and movement data collected under standardized stress and physical exertion protocols [15]. Information about inter-beat intervals could be extracted from the blood volume pulse signal.

The stress portion of the dataset contains recordings from 36 volunteers, stored in 37 folders because one session (f14) was split into two files after a Bluetooth disconnection. Following the dataset documentation, S02 was excluded because its files contain duplicated signals, and f07 because the protective cover was never removed from the wristband, which invalidates its PPG and temperature readings. A third recording, f13, was also excluded during initial screening; the dataset documentation reports no fault for it and the reason for its exclusion was not recorded. The remaining 34 folders were used for the replication experiment. Because the two parts of f14 were treated as separate units, they correspond to 33 distinct participants, and because they were also placed in separate cross-validation groups, the replication contains a small subject-level leak, discussed in Section 7.5. In this case, the goal of using this dataset was not to replace WESAD as the major training dataset. Instead, the dataset was used to find out whether the behavior of temporal features on WESAD is observable for physiological recordings taken using a different type of wearable devices and protocols.

As the dataset does not include native multi-level stress labels, four intensity categories were derived for each subject from the quartiles of a local RMSSD statistic computed over that subject's recording. This difference means that absolute classification scores should not be directly compared between the two datasets.

Study Materials

The main study materials used in the experimental work therefore consisted of:

1. WESAD physiological recordings and annotations as the primary benchmark dataset.
2. RR interval sequences derived from cardiac recordings.
3. HRV time-domain and frequency-domain features.
4. Skin temperature measurements.
5. Time-of-day and temporal information.
6. A secondary Empatica wearable dataset for cross-dataset replication.
7. Derived multi-level stress labels used for the four-class inference task.

The core research results are therefore based mainly on public physiological datasets. The physical wearable system is used as part of the final system integration, while the main model comparisons are evaluated using controlled benchmark data.

3.3 Experimental or study procedure

3.3.1 Physiological Data Preparation

The RR interval was calculated using the R-peaks in the chest ECG recording in WESAD. RR intervals not found within the expected physiological range and with sudden changes between beats were considered as possible artifacts. Only the RR intervals within approximately 300-2000 ms were used, together with a consecutive difference cut-off value of 20%. Artifacts were handled using linear interpolation before calculating features.

Afterward, segmentation was done by using overlapping windows. During preliminary tests, the 120-beat windows with a step size of five beats were used to ensure that there would be enough information for calculating the HRV while having many overlapping segments. In the deployment experiments, this scheme was

compared against 60-beat windows where again there were advances of five beats. The short window was implemented to minimize the delay between collecting the physiological data and the estimation of stress.

3.3.2 HRV and Temporal Feature Construction

Time and frequency domain HRV features were calculated from each RR interval window. The feature set included statistics like mean RR, RMSSD, SDNN, among others frequency related HRV features.

At first, the temporal model consisted of 24 hours Cosinor model fitted by nonlinear least squares. From the fit, the MESOR, amplitude, and residuals were estimated. The Cosinor models were calculated for HRV and skin temperature.

For the main feature-based experiments, each window was represented by a 25-dimensional feature vector that included 13 time and frequency domain HRV features, five residual based features, MESOR and amplitude features, and five cyclic time of day features. In the deployment-oriented experiments, the residual features were computed against the causal baseline described later in this section, and the MESOR and amplitude positions were filled with causal baseline levels, so the vector kept the same 25 dimensions.

The 25-dimensional vector was assessed with the help of XGBoost algorithm that is the gradient-boosted tree model [16]. In case of sequence-based experiments, normalized physiological sequences were used as input to neural networks. Initially, the neural network pipeline combined convolutional layers, bidirectional recurrent processing and attention, following the hybrid architecture of Tanwar et al. [17].

For training of neural models, focal loss function was used because of the imbalance in classes [18]. The non-target (relaxed) class contained the majority of the windows, which made class sensitive training necessary.

3.3.3 Temporal Mechanism Experiments

Another independent series of experiments was designed to examine which temporal representation aspect is responsible for the observed improvement. Six sets of features were considered:

1. Baseline: HRV features combined with time-of-day encoding.
2. Baseline with MESOR and Amplitude: Baseline features supplemented with MESOR and amplitude components.
3. Baseline with Residual Features: Baseline features supplemented with temporal residual features.
4. Full Temporal Feature Set: Baseline features combined with all temporal features.
5. Baseline with Session Level: Baseline features supplemented with the mean and standard deviation of the subject's RR intervals over the session, in place of MESOR and amplitude.
6. Session Level with Residual Features: Configuration 5 supplemented with the residual features.

Configurations 5 and 6 were added because a 24-hour Cosinor fitted to a recording of about 1.5 hours cannot estimate its MESOR reliably (Section 6.1). The session mean and standard deviation describe the same subject-level baseline in a way that is always well defined. Comparing configurations 2 and 5 therefore separates the value of baseline information itself from the quality of the Cosinor estimate. The decomposition was also repeated with window overlaps of 83%, 67% and 50%, to check that the ordering of contributions did not depend on the 96% overlap used in the main analysis.

Also, various periods were assumed for the temporal model with evaluations being done for 24, 12, 6, 3, and 1 hour, as well as 30 minutes and 15 minutes. The cubic polynomial detrending without any period assumption and an adaptive period technique were tested.

These experiments allowed us to explore whether the improvement was dependent only on the 24 hours periodicity or whether some similar behavior would appear with other approaches of local baseline subtraction. The main temporal result was additionally explored using the secondary dataset with the wearable sensors as

described in Section 3.2. This provided a cross-dataset check using a different population and wearable sensing setup.

3.3.4 Window Labelling and Causal Evaluation

Two approaches were used during the deployment process to label a physiological window.

For the first approach known as the midpoint approach, the label is defined by the midpoint of the window and hence, in this case, the second half of the physiological data comes after the labeled time point.

For the second approach referred to as the endpoint approach, the label is assigned at the endpoint of the window in which all data points are either equal to or before the prediction point.

Because a live system has no access to beats recorded after the prediction point, all deployment-oriented experiments used endpoint labels. For the same reason, the time-of-day encoding was computed from the timestamp of the window's final beat rather than its centre.

3.3.5 Causal Baseline Estimation

In the temporal analysis described earlier, each subject's baseline was obtained by fitting a curve to the complete recording. This means that the expected value assigned to the first minutes of a session was shaped, in part, by beats that were recorded an hour or more later. Such a procedure is acceptable when the aim is to analyse a session that has already ended, but it cannot be carried out by a system that must produce a stress estimate while the session is still in progress, since the later beats on which the fit depends do not yet exist at the moment of prediction. A different way of estimating the baseline was therefore required for the deployed pipeline, one in which the baseline at any beat is determined entirely by the beats that came before it.

The approach adopted was the exponentially weighted moving average (EWMA). An EWMA maintains a single running value which is revised each time a new RR interval arrives, combining the new observation with the previous estimate according to

$$\boldsymbol{b_t = \alpha x_t + (1-\alpha) b_{t-1}, \quad 0 < \alpha < 1} \tag{3.1}$$

where x_t is the RR interval of beat t, b_t is the baseline estimate after that beat has been received, and α is a smoothing factor. A small value of α produces a baseline that moves slowly and remembers the distant past; a large value produces one that follows recent beats closely. Because Equation (3.1) uses only x_t and b_{t-1}, the estimate at beat t can never depend on a beat that has not yet occurred, and the computation requires one multiplication and one addition per beat regardless of how long the session has been running.

Rather than choosing α directly, each tracker was specified by a half-life h, expressed in beats, which states how quickly the influence of a past observation should fade. Requiring that the weight of any beat be halved after a further h beats gives $(1-\alpha)^h = 1/2$, and solving for the smoothing factor yields

$$\boldsymbol{\alpha = 1 - 2^{-1/h}}. \tag{3.2}$$

Expanding the recursion in Equation (3.1) makes the role of the half-life explicit:

$$\boldsymbol{b_t = (1-\alpha)^t b_0 \; + \; \alpha \sum_{k=1}^{t} (1-\alpha)^{t-k} \, x_k}. \tag{3.3}$$

so that every past beat contributes to the current baseline, but with a weight that decays geometrically with its age. The same expansion shows that the initial value b_0 is never discarded outright; its weight falls to $2^{-t/h}$ after t beats, which means that a tracker with a long half-life continues to reflect its starting value for a substantial part of the session.

No single half-life is appropriate for every purpose. A tracker short enough to register a sudden autonomic reaction is too responsive to serve as a stable reference for the session as a whole, while a tracker long enough to describe the session level will barely register a brief change. For this reason three trackers were run in parallel on the same RR stream, each with a different half-life, as summarised in Table 3.1 The approximate durations assume a heart rate of 80 beats per minute and are given only as a guide, since the trackers themselves operate in beats rather than in seconds.

Table 3.1: Causal baseline trackers

Tracker	**Half-life h (beats)**	**Approx. duration at 80 bpm**	**Intended role**
Fast	60	45 s	Short-term autonomic reactions
Medium	300	4 min	Sustained state within the session
Slow	1800	22 min	Gradual drift across the session

For each tracker, the deviation of the current beat from its baseline is

$$\boldsymbol{r_t^{(h)} = x_t - b_t^{(h)}}. \tag{3.4}$$

and the residual features of a window are computed from these deviations exactly as they were computed from the Cosinor residuals in the offline analysis. In this way, the causal trackers take the place of the fitted rhythm without changing anything else in the feature construction.

Equation (3.3) also shows why the starting value matters. When a new user begins a session, there is no previous history from which b_0 could be taken, and yet some value must be supplied before the first beat can be processed. In the deployed pipeline, every tracker was initialised with the population mean RR interval computed from the training subjects:

$$\boldsymbol{b_0^{(h)} = \bar{x}_{\text{pop}} \quad \text{for } h \in \{60, 300, 1800\}}. \tag{3.5}$$

Several alternatives to this choice, including seeding from the most similar training subject and seeding from the user's own session mean, were evaluated in the deployment experiments described below. The last of these was included only as a reference point, because a session mean cannot be computed until the session has finished.

The same restriction was applied to every other quantity in the feature pipeline. Feature normalisation and variance estimation were performed using only beats that had already been received, and no statistic was computed over a window that extended beyond the current beat. Taken together, these choices ensure that the deployed system can produce a complete feature vector at any point during a session, using nothing more than the beats observed up to that point and a population mean that is fixed before the session begins.

3.3.6 Model Comparison

Various model setups were tested under similar endpoint labeled windows. Four configurations were compared. The first used gradient-boosted trees alone. The second and third were two-way ensembles that combined the trees with a neural network: in one case the bidirectional convolutional–recurrent network, and in the other the causal MS-CGCA network. The fourth was a three-way ensemble that added a personalised copy of the MS-CGCA network, fine-tuned on a calibration segment from the held-out subject.

The MS-CGCA network was designed so that no hidden state could depend on a future sample. It uses dilated causal convolutions, a unidirectional recurrent layer, and a cross-attention stage in which the time-of-day encoding forms the query.

The main model architectures were compared using five random seeds to cater for variations associated with model training to avoid model selection based on a single experiment.

3.3.7 Deployment Experiments

Several additional experiments have been run to evaluate the deployment performance of the system. In cold start experiments, different techniques for initialization of the physiological baseline of a new user have been compared. Four conditions were compared: no causal baseline at all, seeding from the population mean, seeding from the most similar training subject (donor matching), and seeding from the user's own session mean. The last of these is not deployable, because the

session mean is only known once the session ends, and it was included as a reference point.

Personalisation was studied separately, to test whether calibrating the model to an individual user produces a measurable improvement in accuracy. Confidence-aware prediction has been run by calculating the difference between probabilities of two most likely classes. When the prediction confidence is low, the system outputs a band spanning the two most probable stress levels instead of a single class. Threshold values have been estimated on the training set only and then applied to the held out subject.

Short-term forecasting has been tried as well, where predictions were evaluated at 30, 60, and 120 seconds in the future with comparison to persistence baseline.

The experimental process was therefore moved from offline to an experimental framework with deployment in mind. Not only prediction accuracy but causality, latency, cold start problem, confidence management, and suitability for inclusion in the adaptive VR system were taken into consideration in choosing the final model.

3.4 Measures and evaluation metrics

Several measures were used because a single classification score cannot fully describe the performance of a four-level real-time stress inference system.

3.4.1 Macro-F1 Score

The main classification metric was the macro-averaged F1-score. This metric was selected because the number of samples was not evenly distributed across the four stress categories.

For each class *c*, precision and recall are defined as

$$\boldsymbol{Precision_c = \frac{TP_c}{TP_c + FP_c}} \tag{3.6}$$

and

$$\boldsymbol{Recall_c = \frac{TP_c}{TP_c + FN_c}} \tag{3.7}$$

The F1-score for each class is

$$\boldsymbol{F1_c = 2 \times \frac{Precision_c \times Recall_c}{Precision_c + Recall_c}} \quad \textbf{(3.8)}$$

For *C* classes, macro-F1 is calculated as

$$\text{Macro-F1} = \frac{\mathbf{1}}{\boldsymbol{C}} \sum_{\boldsymbol{c=1}}^{\boldsymbol{C}} \boldsymbol{F\,1_c} \quad \textbf{(3.9)}$$

Each stress level therefore contributes equally to the final macro-F1 value regardless of the number of samples in that class.

3.4.2 Quadratic Weighted Kappa

Quadratic weighted kappa (κ) was used as an additional primary measure because the stress labels have an ordinal structure.

The four classes follow the order:

Relaxed → Mild → Moderate → High

A prediction that differs by one neighbouring level is less serious than a prediction that differs by several levels. Quadratic weighting gives greater penalties to predictions that are further from the correct ordinal class.

This makes κ useful alongside macro-F1. Macro-F1 evaluates classification performance across all classes, while quadratic kappa also considers the distance between the predicted and actual stress levels.

3.4.3 Precision, Recall and Confusion Matrix

Per-class precision and recall were examined to identify which stress levels were easier or more difficult to detect.

Confusion matrices were also used to show how predictions were distributed between the four classes. This allowed errors between neighbouring levels to be separated from more serious errors across distant stress categories.

3.4.4 Ordinal Error Measures

Some experiments also examined the distance between the true and predicted stress level.

A severe or distant error was treated as a prediction that differed from the correct class by two or more ordinal levels. For example, confusing relaxed with high stress is more serious than confusing mild with moderate stress.

Ordinal absolute error measures were also used during parts of the temporal analysis. These measures provided additional information that was not captured by macro-F1 alone.

3.4.5 Prediction Latency

Latency was an important deployment measure. In this research, latency mainly refers to the amount of physiological information that must be collected before a window can generate a prediction.

The 120-beat and 60-beat configurations were therefore compared. A shorter window can reduce the waiting time before the system provides an updated stress estimate.

The final deployment decision considered both classification performance and this prediction delay rather than optimizing classification performance alone.

3.4.6 Model Stability

For the main model comparisons, training was repeated using five random seeds. Results were reported using the mean and standard deviation. With only five paired seeds, the smallest two-sided Wilcoxon p-value that can be obtained is 0.0625. Seed-level comparisons can therefore never reach significance at the 0.05 level, so they were interpreted through effect sizes and seed spread rather than p-values.

This was important because small differences between neural architectures can be smaller than the variation caused by model initialization and training. A model was

therefore not considered better only because it produced a slightly higher result in one run.

3.4.7 Subject-Independent Evaluation

The main experiments used the leave-one-subject-out cross-validation. During each fold, one subject was left out for testing while all other subjects provided data for training the model.

This approach helps to estimate the generalization ability of the model to unseen subject and, therefore, is more suitable for the application of the proposed method than approaches based on evaluating random windows from the same subjects both in training and testing sets. Ensemble weights and confidence thresholds were always selected from the training subjects of each fold. The one exception was the personalisation experiment, which fine-tuned the model on a calibration segment taken from the held-out subject. In that comparison, every configuration was scored only on the held-out windows that remained after the calibration segment, so all results in the comparison came from identical data.

3.4.8 Confidence and Coverage

Studies using confidence aware designs provided measurements of prediction accuracy and coverage. Coverage refers to the proportion of windows where the system generates a prediction. If a confidence threshold is used, then ambiguous windows can be either filtered out or represented using a combined stress band. Macro-F1 and severity of error were calculated for windows generating predictions and these metrics were analyzed in light of coverage because classification performance on answered windows may increase with more difficult predictions being filtered out.

3.4.9 Forecasting Performance

Short-term forecasting was evaluated based on macro-F1 values at roughly 30-, 60-, and 120-second time intervals. The forecasting algorithm was compared to a persistence method, which assumes that the stress state will not change in the short term future, and thus provides a simple benchmark against which the forecasting algorithm can be compared to see if it adds any predictive value beyond it. Overall,

these measures allowed to evaluate classification accuracy, ordinal consistency, error severity, robustness, delay, confidence, subject generalizability, and short-term forecasting. Statistical comparisons of these metrics are described in detail in Section 3.5.

3.5 Data analysis

All comparisons in this research were conducted using leave-one-subject-out cross-validation. Each experiment produced fifteen results, with one result for each held-out WESAD subject. A pooled score was also calculated using all evaluation windows. These two results were used for different purposes. The pooled macro-F1 and quadratic weighted kappa scores were reported as the main performance measures because they show how well each model performed across all evaluation windows. In contrast, the fifteen subject-level scores were used for statistical testing, since individual subjects can reasonably be treated as independent observations, whereas windows from the same subject cannot.

This distinction is important because consecutive windows contain much of the same data. For example, when a 120-beat window moves forward by only five beats, neighbouring windows share approximately 96% of their content. Even with the shorter 60-beat window used during deployment, the overlap remains around 92%. As a result, treating thousands of overlapping windows as independent observations would produce unrealistically narrow confidence intervals. It could also make small performance differences appear statistically significant when there is not enough evidence to support that conclusion. To avoid this problem, all statistical tests were carried out at the subject level. Window-level results were used only to describe overall model performance.

3.5.1 Paired comparisons

When two model configurations were evaluated using the same cross-validation folds, the difference in macro-F1 was calculated separately for each subject. This produced fifteen paired differences, which were then compared using the two-sided Wilcoxon signed-rank test. The Wilcoxon test was preferred over the paired t-test because it does not require the differences to follow a normal distribution. With only

fifteen observations, it would be difficult to reliably determine whether that assumption holds. The same test was also used to compare neural network configurations across five training seeds, while considering the limitation on possible p-values discussed in Section 3.4. All Wilcoxon tests were performed using the SciPy library, with the significance level set at 0.05.

3.5.2 Effect sizes

Statistical significance alone does not show how large or meaningful a performance difference is. A p-value helps determine whether there is sufficient evidence of a difference, but it does not describe the size of that difference. This is particularly important when only fifteen subjects are available, as a difference that matters in practice may still fail to reach statistical significance. For this reason, an effect size was reported alongside every paired comparison. For paired observations, Cohen's d was calculated using the differences between the subject-level scores, as follows:

$$\boldsymbol{d_z = \frac{\bar{\Delta}}{s_\Delta}} \tag{3.10}$$

where $\bar{\Delta}$represents the mean of the paired differences and s_Δ is their sample standard deviation. Using this convention, Cohen's d values of approximately 0.2, 0.5, and 0.8 are generally interpreted as small, medium, and large effects, respectively.

3.5.3 Multiple comparisons

When several models were evaluated in the same analysis, the Friedman test was first used to determine whether there were any overall differences in performance. This test was applied to the subject-level macro-F1 scores of all models before examining individual model pairs. If differences were found, pairwise comparisons were performed using the Wilcoxon signed-rank test. The resulting p-values were then adjusted using the Holm procedure to account for multiple comparisons and reduce the risk of identifying differences as statistically significant by chance. For m comparisons, the unadjusted p-values were arranged in ascending order, from the smallest to the largest, as $p_{(1)} \leq \cdots \leq p_{(m)}$. The adjusted p-value for the i-th comparison was then calculated as follows:

$$\tilde{p}_{(i)} = \min\left(1,\ \max_{1 \le j \le i}\left[(m - j + 1)p_{(j)}\right]\right) \quad (3.11)$$

The Holm correction was applied separately within each analysis, with all pairwise comparisons in that analysis treated as one family of tests. The adjusted p-values were kept in non-decreasing order and limited to a maximum value of 1.0. Each adjusted p-value was then compared with the significance threshold of 0.05. A performance difference was considered statistically significant only when its Holm-adjusted p-value was below this threshold.

The Holm procedure controls the risk of false positives across the entire set of statistical tests, similar to the Bonferroni correction. However, it is more likely to detect real differences when they exist. This made it a better choice for the present study, where the small number of subjects limited the statistical power of each test.

3.5.4 Confidence intervals

Confidence intervals for the main model configurations were estimated using bootstrap resampling at the subject level. A total of 10,000 resamples were generated. In each resample, fifteen subjects were randomly selected with replacement from the original fifteen, meaning that the same subject could be selected more than once. The performance score was then recalculated using the held-out results of the selected subjects. The 95% confidence interval was obtained from the 2.5th and 97.5th percentiles of the resulting distribution. By resampling subjects instead of individual windows, all windows belonging to the same person were kept together. This allowed the intervals to reflect actual performance differences between individuals rather than the much smaller differences between overlapping windows.

3.5.5 Correlation analyses

Pearson's correlation coefficient was used to examine whether differences in model performance across subjects were related to how well a circadian curve fitted their data. For each of the fifteen folds, the held-out macro-F1 score was compared with potential explanatory variables. The main idea was that if temporal features improved performance because they captured circadian patterns, subjects whose data

followed a clearer fitted rhythm should benefit more from those features. This analysis was therefore used to test whether such a relationship existed.

3.5.6 Reporting conventions

For experiments involving repeated training, results were reported as the mean and standard deviation across five training seeds. Some experiments were conducted using only one seed. In those cases, performance differences were smaller than the variation observed between seeds in the main comparisons were not considered meaningful. A difference that was not statistically significant was also not taken as proof that two model configurations performed equally well. Demonstrating equivalence requires a separate statistical procedure, such as two one-sided tests using a predefined equivalence margin. Since no such procedure was performed, this thesis does not make claims of equivalence.

Finally, some reported performance scores depended on choices that could have been influenced by the evaluation data, such as selecting the blend weight for an ensemble model. In these cases, the nested estimate was reported to avoid overly optimistic results. This means that the choice was made using only the remaining folds, without using the fold being evaluated. The more optimistic pooled score was therefore not reported as the final result.

3.6 Ethics privacy security and responsible AI

3.6.1 Data and Consent

The models developed in this research were trained and evaluated using two publicly available datasets: WESAD [2] and the wearable stress and exercise dataset by Hongn et al. [15]. Both datasets were collected by their original researchers through structured laboratory experiments involving human participants. The data was released in de-identified form. The Hongn et al. dataset was collected with approval from the Ciencias de la Vida department of the Instituto Tecnológico de Buenos Aires, and all participants provided written informed consent before participating. No new physiological data was collected from human participants during the development or evaluation of this component. A small-scale usability study with university students is planned before the final presentation (Section 6.3). It will be

carried out with each participant giving written informed consent and being free to withdraw at any time. Physiological signals will be processed in real time and not stored, and feedback will be recorded anonymously under participant codes. Dataset Licensing

The two datasets have different licensing conditions. WESAD is available for scientific and non-commercial research purposes, provided that its original authors are acknowledged. In contrast, the dataset by Hongn et al. is distributed under the Open Data Commons Attribution License v1.0, which allows both commercial and non-commercial use as long as proper attribution is provided.

Since the deployed models were trained using WESAD, commercial use would require permission from the dataset owners. Therefore, developing a commercial version of this component would require either obtaining that permission or retraining the models using data licensed for commercial use.

3.6.2 Privacy

Heart-rate and skin temperature data are health-related personal information and must be handled carefully, especially in a system that continuously receives physiological signals. Both types of data can reveal information about an individual's physiological state. For this reason, the component was designed to minimise the amount of data retained.

The backend receives raw PPG signals and skin temperature measurements. It then derives RR intervals from the PPG signal and processes the physiological features, including temperature, to estimate the user's stress level. The output contains a predicted stress level, a confidence value and, when confidence is low, a band covering two adjacent stress levels.

The incoming physiological data, including PPG signals, RR intervals and skin temperature measurements, is not retained after a prediction is completed. Only the latest prediction is kept in server memory, and it is overwritten whenever a new prediction becomes available. No history buffer is maintained because storing physiological recordings from an entire session is unnecessary for the component's

operation. Neither heart-rate-related data nor temperature measurements are saved to disk or included in application logs.

In Sri Lanka, the Personal Data Protection Act No. 9 of 2022 provides the legal framework for handling personal data. Any deployment beyond the current research setting would therefore need to comply with applicable requirements concerning lawful processing, purpose limitation and the rights of individuals whose data is processed.

3.6.3 Security

This discussion focuses only on the stress inference backend developed as part of this research component. Security measures for the wearable device, mobile relay and VR application are handled by the components responsible for those systems.

Since physiological data is not stored, the backend does not maintain a persistent collection of sensitive recordings. However, its protection of data during transmission remains limited. Both WebSocket endpoints, one for receiving sensor data and the other for streaming predictions, currently use unencrypted ws:// connections. The server does not use TLS, authentication is not required, and cross-origin requests are accepted from any origin.

These settings were used for the current research environment, where devices communicate over a single local network during development, testing and demonstrations. However, they are not suitable for deployment over an open or untrusted network.

Before wider deployment, the WebSocket connections should be secured using wss:// with TLS enabled. Client authentication should also be introduced, and cross-origin access should be restricted to trusted clients. These improvements are identified as future work in Section 8.4.

3.6.4 Responsible Use of the Predictions

The four stress levels used in this research are experimental categories derived from laboratory conditions and a physiological variability measure. They are not clinical classifications. Therefore, the component should not be considered a medical or diagnostic system.

Its main purpose is to support the adaptation of a VR meditation environment based on the user's estimated stress level. This limits the possible consequences of an incorrect prediction. If the system predicts a higher stress level than the user is experiencing, the environment may become calmer than necessary. If it predicts a lower level, the user may receive less environmental adaptation than would have been helpful. Neither outcome directly influences medical treatment or clinical decisions.

Nevertheless, the system was designed to avoid presenting uncertain predictions as definite results. When the two most likely stress levels have similar probabilities, the component reports a band covering both levels instead of selecting only one. This allows the application to respond while also recognising uncertainty in the model's prediction.

3.6.5 Fairness and Generalisation

The primary dataset contains fifteen participants, while the replication dataset includes thirty-three. These relatively small samples cannot fully represent the diversity of people who may eventually use the system. Differences in age, physical fitness, health conditions and normal physiological responses may influence model performance.

As a result, the models may perform differently for individuals or groups who were not adequately represented in the training data. Another concern is the use of population-level values to initialise the causal baseline. A population average may be closer to the resting heart rhythm of some users than others, which could affect the accuracy of their predictions. These limitations are discussed further in Section 7.5.

3.6.6 Integrity of Reporting

The research reports findings that did not support the original expectations, alongside those that did. This includes the observation that the performance improvement associated with Cosinor features was not caused by circadian correction. Short-term forecasting also failed to outperform a simple persistence baseline, while personalisation did not produce a measurable improvement in performance.

Another unexpected result was that initialising the baseline using the population mean performed better than using a subject's own session mean. These findings are included to provide a balanced account of the experiments and their outcomes, rather than reporting only favourable results.

The use of generative AI tools during the preparation of this thesis is disclosed in Appendix B.

3.7 Reproducibility and research artefacts

3.7.1 Experimental Environment

All experiments in this research were implemented using Python. Each experimental notebook was designed to run from start to finish using the raw datasets, without requiring manual intervention between steps. The analysis notebooks were executed on Kaggle using Python 3.12. The main libraries included NumPy, pandas, SciPy, scikit-learn, XGBoost and NeuroKit2. NeuroKit2 was specifically used for ECG signal cleaning and R-peak detection.

The neural network models were developed and trained using TensorFlow 2.20.0 and saved in the native Keras 3 format (.keras). TensorFlow 2.16 and later versions use Keras 3 by default, while earlier versions generally use Keras 2, which may not support models saved using newer Keras 3 features. To maintain compatibility with the saved models, the project's dependency file specifies TensorFlow version 2.16 or later, but below version 3.0. The original training environment used TensorFlow 2.20.0, which remains the reference version for reproducing the experiments.

3.7.2 Controlling Randomness

Random seeds were fixed in each experimental notebook to improve reproducibility. This ensured that running the same notebook again with the same data would produce consistent data windows, cross-validation folds and tree-based models.

Neural network training, however, can still be affected by differences in model initialisation, even when random seeds are controlled. To account for this variation, each neural network configuration was trained using five different seeds. The results were then reported using the mean and standard deviation across these runs. This

approach reduced the possibility of drawing conclusions from a single run that happened to produce unusually good results.

3.7.3 Built-in Verification Checks

Two important types of errors were addressed through verification tests included in the experimental notebooks. These errors could affect the validity of the results without necessarily causing an obvious failure during execution.

The first check focused on causality. To verify that each baseline estimator behaved as intended, the second half of a subject's RR interval series was replaced with an unrealistic value. The estimator was then applied to both the original and modified series. For a causal estimator, the outputs for the first half should remain unchanged because calculations at any point must depend only on the current and previous observations, not on future data.

The offline Cosinor model was also included as a control. Unlike the causal estimators, it fits a curve using the complete recording and was therefore expected to change when future values were modified. All four causal estimators passed this test, while the Cosinor fit changed as expected.

The second verification check focused on the forecasting experiments. A separate test confirmed that each forecasting target belonged to the same subject as its corresponding input window. This prevented targets from being taken across the boundary between two different subjects' recordings, which could otherwise produce invalid training or evaluation results.

3.7.4 Traceability of Reported Figures

All numerical results presented in this thesis were linked to specific outputs in the experimental notebooks. This made it possible to trace each reported value back to the experiment that produced it.

Intermediate results were also saved as JSON files during execution. These files allowed tables and figures to be recreated without having to repeat the complete model training process. Any figure that could not be traced to a reliable notebook output was excluded from the thesis rather than reported without supporting evidence.

Similarly, when a result could not be reproduced after retraining a model, it was discarded instead of being modified to match the earlier value. This approach helped maintain the reliability, transparency and reproducibility of the reported findings.

3.7.5 Research artefacts

The main artefacts produced in this research are listed in Table 3.2. All notebooks are located under ‘component-b/notebooks/’.

Table 3.2: Research Artefacts

Artefact	**Purpose**
01_pipeline/notebook-improvements.ipynb	Offline ensemble experiments used in the mechanism analysis
02_mechanism/notebook-rigorous-analysis.ipynb	Temporal-basis audit and statistical validation
02_mechanism/notebook-baseline-control.ipynb	Comparison of the fitted MESOR with the session-level baseline
02_mechanism/notebook-atbd-novel-method.ipynb	Detrending-period sweep, including polynomial and adaptive baselines
02_mechanism/notebook-close-gaps.ipynb	Subject-level bootstrap confidence intervals
02_mechanism/notebook-empatica-mechanism.ipynb	Cross-dataset replication on the Empatica wrist-PPG dataset

02_mechanism/notebook-ordinal-corn.ipynb	Ordinal classification head
04_scope_expansion/notebook-scopeexpansion-full.ipynb	Causal baseline trackers, causality verification, confidence-aware output and forecasting checks
04_scope_expansion/notebook-coldstart-baseline.ipynb	Cold-start seeding comparison
04_scope_expansion/changed4_with_personalization.ipynb	Personalisation experiment
04_scope_expansion/notebook-nestedensembleweights.ipynb	Nested selection of ensemble weights
05_deployment/notebook-deployment-decision.ipynb	Midpoint versus endpoint labelling and window-length comparison
05_deployment/notebook-mscgca-ablation.ipynb	Ablation of the MS-CGCA network
05_deployment/notebook-train-export-2way.ipynb	Training and export of the deployed two-way ensemble
component-b/src/componentb/	Source code of the real-time inference backend

3.7.6 Code and data availability

The source code and notebooks are available in the R26-DS-006 project repository. Both datasets are publicly available from their original authors, allowing the experiments reported in this thesis to be reproduced by obtaining the datasets and executing the corresponding notebooks in the repository.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements for this component were initially defined in the project proposal, before the experiments began. However, some of these requirements were revised as the research progressed. This was necessary because the experimental findings revealed that certain assumptions made during the proposal stage were not supported by the data.

One important change involved the circadian baseline. Following earlier work showing that removing circadian trends from HRV can improve stress assessment [5], the original proposal required the component to model each user's 24-hour HRV rhythm using Cosinor rhythmometry [4] and to estimate stress based on deviations from that rhythm. However, the temporal-basis audit (Section 6.1) showed that a reliable 24-hour rhythm could not be identified from the WESAD recordings [2], which last approximately 1.5 hours. Further analysis revealed that the performance improvement previously attributed to circadian modelling actually came from normalising each user's physiological measurements against their own baseline within the recording session, a technique already established in wearable stress detection [19], [20]. Based on these findings, the requirement was revised to focus on a causal baseline that continuously adapts to the user's physiological state during the session.

Some requirements were also introduced as a direct result of the experiments rather than being part of the original proposal. For example, the requirement that predictions must not depend on future samples came from findings that midpoint labelling and baselines calculated using complete recordings could produce overly optimistic offline accuracy. Such results would not reflect the performance of a real-time system, where future measurements are unavailable, and deployment considerations of this kind have been identified as one of the least evaluated aspects of wearable stress detection research [1].

The requirements presented below therefore have two main purposes. They define the functions that the stress inference component must perform within the complete system and establish the conditions needed to maintain the validity of the reported model performance during deployment. Section 4.1.4 explains how each original requirement was carried forward or revised, along with the reasons for those changes.

These requirements apply only to the stress inference component developed in this research. The wearable sensor was developed jointly by the group, and the mobile application and adaptive VR application by other group members. They are included in this discussion only to explain how physiological data is received by the component and how its predictions are delivered to the rest of the system.

4.1.1 Stakeholders

The stakeholders of the component were identified in the project proposal. They are summarised in Table 4.1, together with their interest in the component as it was finally built.

Table 4.1: Stakeholders of the stress inference component

Stakeholder	Type	Interest in this component
Users of the VR meditation system	Primary	Receive an environment that adapts to their stress level without needing to report it themselves
Developers of the VR adaptation component	Primary (internal)	Consume the stress predictions, and depend on their timeliness, format and stated confidence
Researchers in affective computing and digital health	Primary	Use the findings on baseline normalisation and deployment cost, and the reproducible research artefacts
Wellness platforms and corporate wellness programmes	Secondary	Potential adopters of stress inference as a service

Wearable device manufacturers	Secondary	Provide the physiological signals and are potential integration partners
Mental health practitioners	Secondary	May use stress trends as supporting information; the component provides no clinical diagnosis

4.1.2 Functional requirements

Table 4.2: Functional requirements

ID	Requirement	Rationale	Verified in
FR1	The component shall receive a continuous stream of PPG and skin-temperature samples over a network connection from the mobile relay.	The backend is the only part of the system that performs inference, so all sensor data must reach it.	§5.3
FR2	The component shall detect heartbeats in the PPG signal, derive RR intervals, and reject intervals outside the physiological range of 300–2000 ms or differing from the previous interval by more than 20%.	RR intervals are the primary input of every model, and the rejection rules match those applied to the training data (§3.3).	§5.2, §5.4
FR3	The component shall maintain three causal baseline trackers with half-lives of 60, 300 and 1800 beats, each updated once per beat.	The residual features used by the models are defined relative to these baselines (§3.3).	§5.2, §5.4
FR4	The component shall initialise the baseline trackers of a new user from the population mean, without requiring a calibration session.	A new user has no history, and population seeding outperformed seeding from the user's own session mean (§3.3,	§6.1

		Chapter 6).	
FR5	The component shall form a 60-beat analysis window ending at the most recent beat, advance it by five beats, and compute from each window the 25-dimensional feature vector and the sequence input of the neural network.	This is the deployed configuration selected in the deployment experiments.	§5.2
FR6	The component shall classify each window into one of four ordinal stress levels (relaxed, mild, moderate, high) using the two-way ensemble of gradient-boosted trees and the causal MS-CGCA network.	This is the configuration selected for deployment (§4.3).	§6.2
FR7	The component shall compute a confidence value for each prediction as the margin between the two most probable classes and, when the margin falls below the deployed threshold, report a band spanning those two levels instead of a single level.	Prevents the application from acting on false precision when a window is ambiguous (§3.3).	§6.1, §6.2
FR8	The component shall publish each prediction to every connected client, including the VR application and the mobile application.	The VR environment adapts to the latest estimate, and the mobile application displays it to the user.	§5.3, §6.4
FR9	The component shall expose a status endpoint reporting whether the service is running, and shall notify the sending client when its models are unavailable.	Allows the other components and the operator to confirm readiness before a session	§5.4

		begins.	

4.1.3 Non-functional requirements

Table 4.3: Non-functional Requirements

ID	Category	Requirement	Rationale	Verified in
NFR1	Causality	No feature, baseline, normalisation step or model output may depend on any sample received after the beat at which the prediction is made.	The central constraint identified by the research; violating it would invalidate the offline accuracy figures in deployment.	§3.7, §5.4
NFR2	Latency	The first prediction shall be available after 60 beats, about 45 seconds at 80 beats per minute, and a prediction shall then be computed every five beats, about every 4 seconds of heart activity. Because the mobile application sends PPG in 15-second frames, predictions are delivered in groups as each frame is processed, which adds up to about 15 seconds	A prediction must describe the user's current state closely enough for the VR environment to respond to it; the 60-beat window halved the delay of the 120-beat window without reducing accuracy.	§6.2

		of delay.		
NFR3	Processing time	Computing one prediction shall take substantially less time than the interval between updates, so that processing never falls behind the incoming beats.	Otherwise predictions would lag progressively further behind the user's state.	§6.2
NFR4	Compute placement	Inference shall run on the backend computer rather than on the VR headset.	The headset's processing capacity is needed for rendering the environment.	§4.2
NFR5	Privacy	Physiological data shall not be written to disk or to logs, and only the most recent prediction shall be held in memory.	Health-related data should be retained only where there is a need for it (§3.6).	§3.6, §5.4
NFR6	Interoperability	Predictions shall be sent as JSON messages over WebSocket, so that clients written in different languages can consume them.	The VR application is written in C# (Unity) and the mobile application in TypeScript (React Native).	§5.3
NFR7	Reproducibility and maintainability	The trained models shall be stored as versioned artefacts, loaded separately from	Allows models to be replaced without changing the server, and guards against	§3.7, §5.1

		the server code, with the software versions required to load them pinned.	version incompatibility (§3.7).	
NFR8	Deployment scope	In its current form, the component shall operate only on a closed local network.	The backend has no transport encryption or authentication (§3.6).	§3.6

4.1.4 Changes from the proposed requirements

Table 4.4 shows how each requirement from the original project proposal was carried forward or revised during the research. Most requirements were either kept unchanged or made more specific based on the findings. However, three requirements were removed or significantly modified. These changes were not caused by time limitations but were based on experimental results that showed why the original requirements needed to be reconsidered.

Table 4.4: Traceability from proposed to final requirements

Proposed requirement	Status	Final form and reason
FR1: Receive processed physiological signals (HRV derived from ECG or PPG) from the acquisition component	Modified	The component receives the raw PPG signal and skin temperature and derives RR intervals itself (FR1, FR2), so that artefact rejection follows exactly the rules used in training
FR2: Extract HRV features including RMSSD, SDNN and LF/HF	Retained and extended	Thirteen time- and frequency-domain features are computed for each window (FR5)
FR3: Model circadian rhythm using Cosinor or	Retained for research;	Cosinor modelling was carried out and analysed (Chapter 3), but a

similar rhythmometry	replaced in deployment	Cosinor fit requires the complete recording and cannot run causally. The deployed component uses causal EWMA baselines instead (FR3)
FR4: Maintain an ongoing baseline that follows each user's biological rhythm	Modified	The baseline is updated with every beat within a session, at three timescales (FR3). It does not attempt to follow a 24-hour rhythm, which the data cannot support
FR5: Compute residuals of HRV against the circadian baseline	Modified	Residuals are computed against the causal baselines (Equation 3.4). The research showed that these residuals, not the circadian component, carry the useful information
FR6: Pass HRV and residual features to a machine learning classifier	Retained	FR5, FR6
FR7: Identify four stress levels	Retained	FR6
FR8: Send the stress level to the VR system	Retained and extended	FR8, with a confidence value and an uncertainty band added (FR7)
FR9: Continuously update a personalised baseline for each user	Partly retained	Within-session updating is retained (FR3). Per-user personalisation was tested and gave no measurable gain (+0.0066 macro-F1, $p = 0.625$) while requiring calibration data and an additional model for every user, so it was not deployed
FR10: Store all	Dropped	Storage was needed only to build

physiological data and predictions in a secure cloud database		long-term circadian profiles. Once circadian modelling was shown not to apply to the available data, there was no reason to retain physiological data, and the component deliberately keeps none (NFR5)
NFR Security: encryption in transit and at rest; authentication; pseudonymisation	Partly met	Nothing is stored, so no data at rest needs protection. Encryption in transit and authentication are not implemented, so deployment is restricted to a local network (NFR8) and these measures are future work
NFR Performance: low-latency continuous inference	Retained and made measurable	NFR2, NFR3
NFR Scalability: multiple simultaneous users on cloud infrastructure	Not addressed	The component is a single-user prototype on a local network; multi-user operation is future work
NFR Reliability: consistent results over time	Retained and made specific	Expressed as causality (NFR1) and reproducibility (NFR7)
NFR Usability: outputs that are easy to understand	Retained	Named stress levels, with a band when the estimate is uncertain (FR7)
NFR Maintainability: models updatable independently	Retained	NFR7

4.1.5 Constraints and Exclusions

Three main constraints define the scope of this component.

First, the component provides research-based stress estimates that are used only to adapt a VR meditation environment. As stated in the original project proposal, it is not intended to diagnose medical conditions or support medical decisions. The four stress levels reported by the system are experimental categories derived from the laboratory protocol used in the WESAD dataset [2]. Therefore, these levels should not be interpreted as clinical measurements of a person's mental health.

Second, short-term stress forecasting was explored as a possible extension of the research. The idea was that predicting an increase in stress could allow the VR environment to respond before the user's stress level actually rises. However, the forecasting model performed worse than a simple persistence baseline across all tested prediction horizons. This baseline assumes that the current stress level will remain unchanged. Since forecasting did not provide additional predictive value, it was excluded from the deployed component to avoid introducing unnecessary complexity.

Third, the original project proposal included plans to collect new physiological data from participants for model validation and personalisation. This activity was not carried out for model development; instead, a small usability study with university students is planned to gather user feedback on the deployed system (Section 6.3). Instead, the component was developed and evaluated entirely using publicly available datasets, as described in Section 3.2.

The dataset by Hongn et al. [15] was also selected in place of the SWELL-KW [21] and TILES-2018 [22] datasets identified in the original proposal. Access to TILES-2018 was not granted. SWELL-KW was unsuitable because it provides pre-computed HRV features rather than raw inter-beat intervals and does not contain skin temperature measurements. As a result, the baseline and residual features required by this component could not be generated from that dataset.

In contrast, the Hongn et al. dataset is openly available and provides raw inter-beat intervals derived from wrist-based blood volume pulse (BVP) signals, along with skin temperature measurements. The dataset originally included 36 volunteers, of whom 33 were used in this research after applying the exclusion criteria described in

Section 3.2. This made it suitable for independently evaluating the findings obtained from WESAD using a different wearable sensor and participant population.

4.2 Component architecture within the complete solution

4.2.1 Position in the Complete System

The complete system operates as a closed loop, where the user's physiological signals are continuously collected, analysed and used to adapt the VR meditation environment. A wearable sensor measures the photoplethysmography (PPG) signal and skin temperature, then transmits the data through Bluetooth Low Energy (BLE) to a mobile application. The mobile application forwards these measurements to a backend computer, where they are processed to estimate the user's stress level. The resulting prediction is sent to the VR meditation application running on a Meta Quest 2 headset, allowing the virtual environment to adapt based on the user's estimated stress level. At the same time, the prediction is sent back to the mobile application, which displays the physiological measurements and stress-related information to the user in real time. Figure 4.1 illustrates how these components are connected.

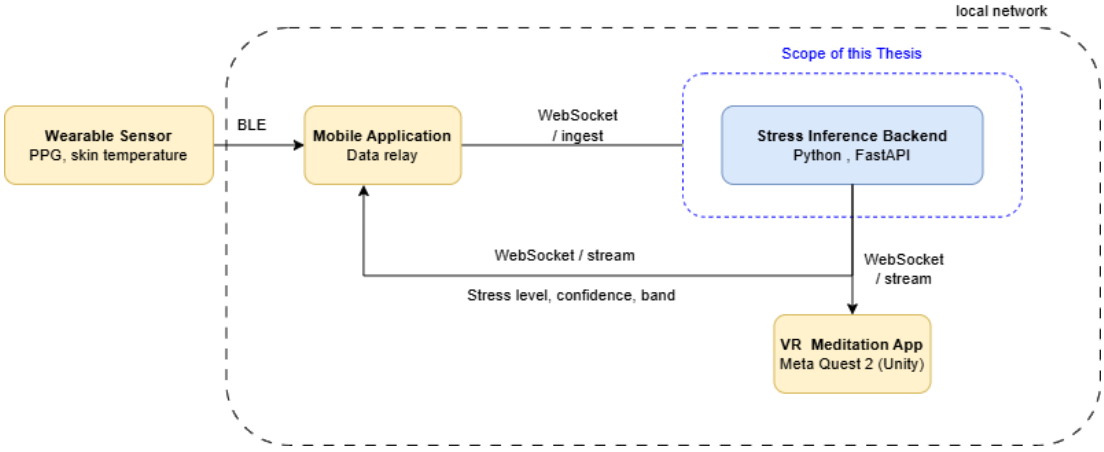


Figure 4.1: Position of the stress inference component within the complete system

The stress inference component plays a central role in this closed-loop system. It is the only component responsible for converting raw physiological measurements into estimates of the user's stress level. Other parts of the system depend on these predictions, either directly or indirectly, to perform their functions. The component communicates through two main network interfaces. The first receives physiological

sensor data from the mobile application, while the second sends stress predictions to the VR application and the mobile application. The wearable sensor was developed jointly by the group, and the mobile application and VR environment by other group members; these are therefore outside the scope of this thesis.

The mobile application has two responsibilities when communicating with the backend. First, it receives physiological measurements from the wearable and forwards them to the stress inference component. Second, it receives the predicted stress levels from the backend and displays them to the user. These operations use separate connections, allowing sensor data to be transmitted without being delayed by outgoing predictions. The mobile application also receives the same predictions as the VR application, so the backend does not need separate prediction processes for each client.

The decision to perform stress inference on a separate backend computer was made to avoid the limitations of running the models on the headset, mobile device or wearable sensor. The VR headset needs sufficient processing resources to render the meditation environment smoothly. Running neural network inference on the headset could compete with rendering and affect the user's experience. The wearable sensor also has limited memory and processing power, making it unsuitable for running the models. Although inference could be performed on a mobile device, this would introduce platform-specific requirements and hardware limitations. Using a separate backend computer connected to the same local network avoids these issues, although it requires the devices to maintain a network connection.

The final system architecture differs from the original project proposal in three main ways. First, the proposal suggested using MQTT and REST interfaces for communication between components. However, the implemented system uses WebSocket connections [23]. Since physiological measurements and stress predictions are continuously transmitted, WebSockets allow each data stream to use a persistent connection. This avoids the need for a separate message broker or repeated requests for individual samples.

Second, the original proposal included a time-series database to store each user's physiological history. This was intended to support the development of long-term circadian baselines. However, the research findings showed that reliable circadian baselines could not be estimated from the available recordings. Further analysis also revealed that circadian modelling was not responsible for the observed improvement in prediction accuracy. As a result, the database was removed from the final architecture because maintaining long-term physiological records was no longer necessary for the component's operation.

Third, the inference engine was also changed from what was originally proposed. The project proposal planned to evaluate XGBoost, LSTM and CNN–LSTM models as separate classification approaches, with only the predicted stress level being sent to the VR application. However, the final implementation uses a two-model ensemble that combines gradient-boosted trees with a causal neural network. In addition to the predicted stress level, the system provides a confidence value for each prediction. When the prediction is uncertain, it also reports a band covering two adjacent stress levels.

These changes were based on the experimental findings. The causal neural network was selected instead of a bidirectional recurrent model because real-time predictions must rely only on data that has already been received. A bidirectional model could use future samples during offline evaluation, which would not be available in a live system. The confidence output was also introduced after the experiments showed that neighbouring stress levels were often difficult to distinguish. Reporting only a single stress level in such cases could give a misleading impression of certainty. By including confidence values and a two-level band when needed, the final system provides a more realistic indication of how certain each prediction is. The tools and platforms used are listed in Section 5.1.

4.2.2 External interfaces

The component communicates with other parts of the system through the endpoints presented in Table 4.5. Both endpoints responsible for transferring data use WebSocket connections instead of repeated HTTP requests. This approach was chosen because physiological measurements and stress predictions are transmitted

continuously. WebSockets maintain a persistent connection, allowing data to be exchanged without establishing a new connection for every sensor sample or prediction. This reduces communication overhead and supports continuous data transfer between the components.

Table 4.5: External interfaces of the stress inference backend

Endpoint	Type	Direction	Content
/ingest	WebSocket	Mobile application → backend	One JSON frame per 15 seconds: a POSIX timestamp, the sampling rate (fixed at 64 Hz), 960 raw PPG samples, and an optional skin-temperature reading. The backend acknowledges each frame and reports problems (invalid frame, failed beat detection, models unavailable).
/stream	WebSocket	Backend → VR application and mobile application	One JSON message per prediction: timestamp, window start and end, heart rate, RMSSD and SDNN of the window, signal quality, and a stress block containing the mode (point or band), the level or the two band levels, a label, the confidence margin, the class probabilities and a continuous score.
/stress/latest	HTTP GET	Client → backend	The most recent prediction, for clients that poll rather than subscribe. Returns status 503 until the first window is complete.

Keeping the incoming and outgoing data streams separate allows the data source and the clients receiving predictions to operate independently. Physiological samples are sent to the /ingest endpoint without the backend needing to know which clients are

connected to receive predictions. Similarly, any client can connect to the /stream endpoint at any time to receive stress predictions without requiring changes to other parts of the system.

This separation also made testing easier. The backend could be tested from start to finish by replaying previously recorded physiological data through the /ingest endpoint. As a result, the complete inference process could be evaluated without requiring the physical wearable sensor or mobile application to be connected.

4.2.3 Internal Architecture

The internal architecture of the component follows a pipeline structure, where incoming physiological samples pass through a series of processing stages in a fixed order, as illustrated in Figure 4.2. Each stage maintains only the information needed to process the next incoming beat. It does not use any physiological measurements that have not yet been received.

This design ensures that the entire pipeline follows the causality requirement (NFR1). Since each processing stage operates using only current and previously received data, the complete pipeline also remains causal. This prevents future physiological measurements from influencing earlier predictions and allows the same processing approach to be used during both offline evaluation and real-time deployment.

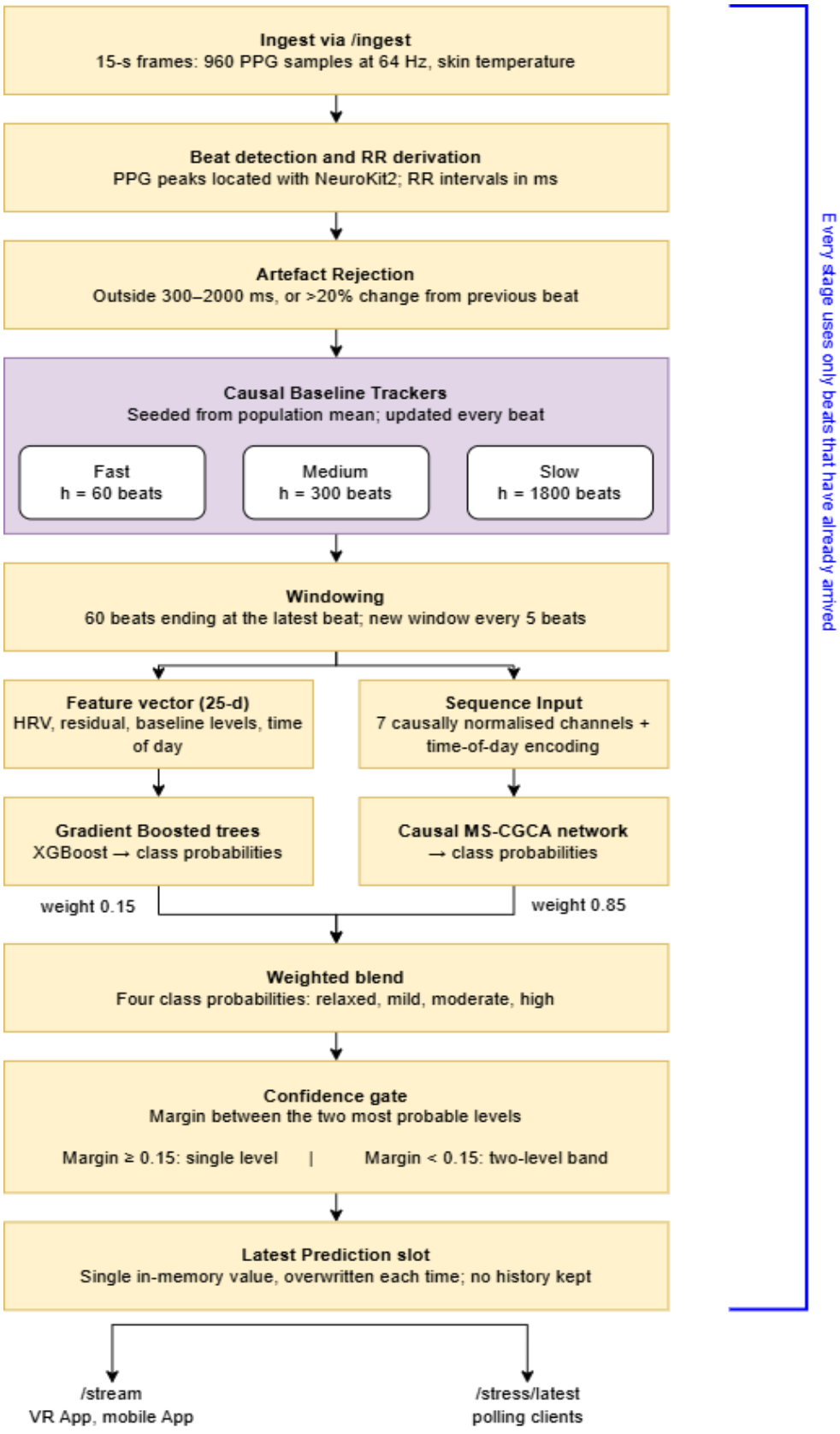


Figure 4.2: Internal pipeline of Stress Inference Component

4.2.4 Ingestion and Signal Processing

Physiological data received through the /ingest endpoint is first processed into individual heartbeat records. Heartbeats are detected from the incoming PPG signal, and RR intervals are calculated by measuring the time between consecutive beats. The intervals are then checked to identify invalid measurements. Any interval that falls outside the expected physiological range or changes too sharply compared with the previous beat is rejected. These checks follow the same rules used during model training and are consistent with established HRV measurement practices [9].

Skin temperature measurements are also aligned with the detected heartbeats. This ensures that each heartbeat record contains the physiological information needed for the remaining processing stages. As a result, the later stages can work with a consistent beat-by-beat representation of the incoming data.

4.2.5 Causal State

Each new RR interval is used to update the three baseline trackers described in Section 3.3. At the beginning of a new session, these trackers are initialised using the population mean instead of requiring the user to complete a separate calibration period. This allows the component to begin generating features once enough heartbeats have been received to form an analysis window.

For every incoming beat, the difference between its RR interval and each baseline is calculated immediately. These differences are referred to as residuals. Once a residual has been calculated, its value remains unchanged, even as more physiological data arrives. This ensures that previously calculated features are not affected by future measurements.

4.2.6 Windowing and Features

Once 60 heartbeats have been collected, the component creates its first analysis window, ending at the most recently received beat. A new window is then generated after every five additional heartbeats. This allows the system to update its stress estimates regularly while using the most recent physiological information.

Two different representations are generated from each window. The first is a 25-dimensional feature vector used by the gradient-boosted tree model. It combines

time-domain and frequency-domain HRV features, residual features calculated from the causal baselines, the current baseline values and a cyclic representation of the time of day.

The second representation is a multi-channel sequence of heartbeat-related measurements, which is used as input to the neural network. These two representations allow the ensemble to analyse the same physiological window using different modelling approaches.

4.2.7 Inference

The two representations are passed to the corresponding models in the ensemble. The gradient-boosted tree model, implemented using XGBoost [16], processes the feature vector and produces a probability for each stress class. At the same time, the causal MS-CGCA neural network processes the multi-channel sequence and generates its own class probabilities.

The predictions from both models are then combined using a weighted average. The XGBoost model contributes approximately 15% of the final prediction, while the MS-CGCA network contributes the remaining 85%. These weights were selected through nested cross-validation, as explained in Section 3.5.

The MS-CGCA network was selected over the bidirectional neural network used in the model comparisons, which followed the hybrid convolutional–recurrent architecture described by Tanwar et al. [17]. This decision was based mainly on the requirements of real-time inference rather than a difference in prediction accuracy.

Unlike a bidirectional recurrent model, MS-CGCA uses causal convolutions and unidirectional recurrence. These operations prevent the network from accessing future samples when generating a prediction. Its ability to maintain causality is therefore built into the model architecture and does not depend entirely on how the input windows are prepared. This makes it more suitable for continuous stress inference, where only previously received physiological data should influence the current prediction. The reasons for this model selection are discussed further in Section 4.3.

4.2.8 Output

After the two models' predictions are combined, the resulting class probabilities are passed through a confidence gate. This stage determines whether the system should report a single stress level or a range of two possible levels.

When the probability difference between the two most likely stress classes exceeds the deployed threshold, the component reports the most likely stress level along with its confidence value. However, when the difference is below the threshold, the prediction is considered uncertain. In that case, the component reports a band covering the two most likely stress levels instead of forcing a single classification.

The final prediction is stored temporarily in memory, replacing the previous prediction. It is then transmitted to all clients connected through the /stream endpoint. During normal operation, these clients include the VR meditation application and the mobile application, allowing both to receive the same stress estimates.

The backend keeps only the latest prediction and does not maintain a history of previous results. Once a prediction is replaced, the earlier result is no longer retained by the component. This approach avoids unnecessary storage of stress-related information and ensures that the backend does not maintain a record of the user's session after it has ended.

4.3 Design alternatives and decisions

Most of the design decisions in this component were based on experimental findings rather than personal preferences. At each stage, the available alternatives were implemented and evaluated using the same leave-one-subject-out cross-validation approach and identical data windows. This ensured that the comparisons were fair and that any differences in performance could be linked to the specific design choice being tested.

The selection criteria were established before carrying out the comparisons. When one approach showed a measurable improvement in prediction accuracy, it was preferred over the others. However, when the differences were not statistically significant, other factors were considered. These included whether the approach

could operate without using future data, how simple it was to implement and maintain, and the computational resources it required.

This approach was particularly important because several experiments produced results with no statistically significant differences between the alternatives. In such cases, choosing a model based only on a slightly higher performance score could lead to a decision based on random variation rather than a meaningful improvement. Therefore, the final design favoured approaches that were more suitable for real-time deployment when there was insufficient statistical evidence to justify a more complex alternative.

Table 4.6 summarises the main design decisions and their outcomes. The following subsections discuss the decisions that had the greatest influence on the final architecture and explain the experimental findings behind them.

Table 4.6: Design alternatives and decisions

Decision	**Alternatives evaluated**	**Evidence**	**Choice**
Labelling convention	Midpoint; endpoint	Midpoint labelling inflated macro-F1 by +0.071 to +0.084 across four configurations	Endpoint
Baseline estimator	Offline Cosinor fit; causal EWMA, sliding-mean and Kalman trackers	The Cosinor fit needs the complete recording; a causal baseline added +0.068 macro-F1 over no baseline	Three causal EWMA trackers
Window length	120 beats; 60 beats	60 beats halved the delay (about 90 s to 45 s) and raised macro-F1 by +0.034 (MS-CGCA) and +0.075 (bidirectional)	60 beats

Neural architecture	Bidirectional CNN–LSTM with attention; causal MS-CGCA; four experimental architectures	MS-CGCA against bidirectional: −0.0068, p = 0.625; the experimental architectures did not outperform the engineered features	MS-CGCA
Ensemble composition	Trees alone; two-way; three-way with personalised member	Trees alone trailed by 0.022 to 0.029; the remaining configurations could not be separated ($p \geq 0.625$)	Two-way
Personalisation	None; per-user fine-tuning	+0.0066 macro-F1, p = 0.625	Not deployed
Cold-start seeding	None; population mean; donor match; own session mean	Population 0.565, donor 0.563, own session 0.552, none 0.496	Population mean
Output form	Forced single level; confidence-gated band	66.9% of errors among the least confident 20% of windows fell between adjacent levels	Confidence-gated band
Forecasting	Model forecast; persistence	Persistence was better at every horizon tested	Not deployed
Inference placement	VR headset; mobile device; backend	Headset capacity is needed for rendering; the wearable cannot run the models	Backend

4.3.1 Labelling Convention

A 60-beat window can be assigned a stress label in two ways. Midpoint labelling uses the stress level at the centre of the window, while endpoint labelling uses the

level recorded at the final beat. With midpoint labelling, half of the beats in the window occur after the point being labelled. This introduces future information that would not be available during real-time operation.

To examine the effect of this difference, both labelling methods were evaluated under identical conditions. Midpoint labelling increased macro-F1 by between 0.071 and 0.084 across four different model configurations. Similar improvements were observed in gradient-boosted trees and two different neural networks. This showed that the increase was caused by the labelling method rather than a particular model architecture.

Endpoint labelling was therefore selected for all deployment-related experiments. Every performance result reported for the deployed component in this thesis follows this convention, ensuring that the evaluation reflects the information available during real-time operation.

4.3.2 Baseline Estimator

The Cosinor model used in the mechanism analysis [4] requires the complete recording to calculate its baseline. This makes it unsuitable for real-time use because future physiological measurements are not yet available while a session is running.

To address this limitation, three causal baseline estimation methods were implemented and evaluated: an exponentially weighted moving average (EWMA), a sliding-window mean and a scalar Kalman filter. Each method was tested for causality by replacing the second half of a recording with invalid values and checking whether the outputs for the first half remained unchanged.

None of the causal methods achieved the same accuracy as the offline Cosinor fit. This was expected because the offline method has access to the entire recording. However, the more relevant comparison for deployment was whether using a causal baseline improved performance compared with having no baseline at all. The results showed an improvement of 0.068 in macro-F1 when a causal baseline was included.

Three EWMA trackers with different half-lives were selected for the final implementation. Each tracker requires only one multiplication and one addition for every incoming beat, making it computationally efficient. Using different half-lives

also allows the model to capture both short-term physiological changes and slower variations in the user's baseline.

4.3.3 Window Length

Window length affects how quickly the system can respond to changes in a user's physiological state. Shorter windows generally allow predictions to be updated sooner, although they are often expected to reduce accuracy because less information is available.

The experiments showed a different outcome. Reducing the window length from 120 beats to 60 beats decreased the delay from approximately 90 seconds to 45 seconds. At the same time, macro-F1 improved for both neural network architectures evaluated.

This meant that the shorter window provided better responsiveness without sacrificing prediction accuracy. In fact, it improved both measures. Based on these findings, the 60-beat window was selected for the deployed component. Windows containing fewer than 60 beats were not evaluated, so it remains uncertain whether further reductions would produce similar improvements.

4.3.4 Neural Architecture

The original offline pipeline used a bidirectional convolutional–recurrent neural network with attention, based on the hybrid architecture described by Tanwar et al. [17]. A bidirectional recurrent layer processes the input sequence in both forward and backward directions. This can be acceptable when the complete input window ends at the prediction point. However, if the windowing method changes, there is a risk that the model could use physiological measurements recorded after the intended prediction time without producing an obvious error.

The MS-CGCA network was developed to avoid this risk. It uses dilated causal convolutions, a unidirectional recurrent layer and a cross-attention mechanism in which the time-of-day encoding acts as the query. Its causal convolutions and forward-only recurrence prevent the model from using future samples within its input sequence. This makes the network less dependent on assumptions about window preparation and more suitable for real-time inference.

The experimental results showed no statistically significant difference in accuracy between MS-CGCA and the bidirectional network. The difference in macro-F1 was −0.0068, with a p-value of 0.625. Since neither model demonstrated a clear accuracy advantage, MS-CGCA was selected because its architecture provides stronger protection against the use of future information.

Four additional neural network architectures were also developed to investigate whether the models could learn deviations from a physiological baseline directly, without relying on engineered baseline features. However, none of these approaches performed better than the models using engineered features. They were therefore not selected for further development.

4.3.5 Ensemble Composition and Personalisation

The ensemble experiments examined whether combining different model types could improve stress classification. Gradient-boosted trees used alone performed worse than every evaluated ensemble, with macro-F1 scores lower by between 0.022 and 0.029. This suggests that the beat-by-beat sequence contains useful information that is not fully represented by the summary features used by the tree model.

Three main ensemble configurations were compared: a two-way combination of XGBoost and MS-CGCA, a two-way ensemble using the bidirectional neural network, and a three-way ensemble that included an additional personalised network. The results showed no statistically significant differences among these configurations, with p-values of at least 0.625.

Adding the personalised model improved macro-F1 by only 0.0066, and the difference was not statistically significant ($p = 0.625$). Personalisation would also introduce several practical requirements. Each user would need to complete a calibration recording, the system would have to perform fine-tuning during operation, and an additional model would need to be stored and loaded.

Furthermore, a two-way ensemble would still be necessary for users who had not completed calibration. Since the three-way approach did not provide a measurable improvement, the simpler two-way ensemble combining XGBoost and MS-CGCA was selected for deployment.

4.3.6 Cold-Start Seeding

When a new user begins a session, the system has no previous physiological measurements from that person to initialise the baseline trackers. Several methods were evaluated to determine how the initial baseline values should be selected.

Using the population mean produced a macro-F1 score of 0.565, compared with 0.496 when no causal baseline was included. Initialising the trackers using values from the most similar training subject produced a score of 0.563, showing no detectable improvement over the population mean.

Another experiment used the mean calculated from the user's own complete session. However, this value would not be available at the beginning of a real-time session. It also produced a lower macro-F1 score of 0.552. The difference was statistically significant ($p = 0.0054$), with an effect size of $d = -0.71$. Performance was lower for 11 of the 15 subjects.

Based on these results, population-mean seeding was selected for the final system. It provided better overall performance than the alternatives tested and allowed the baseline trackers to be initialised immediately. As a result, users do not need to complete a separate enrolment or calibration session before using the component.

4.3.7 Output Form

The four stress categories used in this research are not always clearly separated in terms of their physiological characteristics. Neighbouring stress levels can have similar patterns, making it difficult for the model to distinguish between them. When a classifier is required to return only one level, it may present an uncertain prediction as though the result were definite.

The experimental results supported this concern. Among the 20% of evaluation windows where the model had the lowest confidence, 66.9% of the classification errors occurred between adjacent stress levels.

To address this problem, a confidence-based output mechanism was introduced. When the difference between the two highest class probabilities is sufficiently large, the component reports a single stress level. If the difference is small, it instead

returns a band covering the two most likely levels. This allows the system to communicate uncertainty without forcing a single classification.

The confidence mechanism was evaluated at different coverage levels. Coverage refers to the proportion of windows for which the model provides a single-level prediction. At full coverage, the severe-error rate was 0.0674. Reducing coverage to approximately 80% lowered this rate to 0.0337, while macro-F1 on the windows receiving single-level predictions improved by 0.0251.

However, reducing coverage below approximately 70% caused performance to decline sharply. At these lower coverage levels, too few windows from the less common stress classes received single-level predictions, making their performance scores less reliable. These findings showed that the confidence mechanism could reduce serious classification errors, but that its threshold needed to balance prediction reliability with the proportion of windows receiving single-level outputs.

4.3.8 Forecasting

Short-term stress forecasting was explored as a possible extension of the component. The idea was that if the system could predict an increase in stress before it occurred, the VR meditation environment could respond earlier and provide a more timely adaptation.

Forecasting was evaluated at prediction horizons of approximately 30, 60 and 120 seconds. However, the forecasting model performed worse than a simple persistence baseline at every tested horizon. This baseline assumes that the current stress level will remain unchanged over the prediction period.

Since forecasting did not provide an improvement over this simpler method, it was excluded from the final implementation. The deployed component therefore focuses on estimating the user's current stress level rather than predicting future stress.

4.4 Detailed Design

This section explains the individual stages of the processing pipeline illustrated in Figure 2. The design is presented in sufficient detail to support reimplementation of

the component and to show how physiological measurements are converted into stress predictions.

The deployed component follows the same processing methods used during model training. Feature extraction functions are shared between the experimental notebooks and the backend server to maintain consistency. When the trained models are loaded, the system also checks whether the order of the input features matches the configuration saved during training.

This verification is important because an incorrect feature order may not cause an execution error. Instead, the models could continue producing predictions using mismatched inputs, leading to unreliable results. For this reason, checking feature alignment was included as a required part of the system design rather than being treated as an additional verification step.

4.4.1 Signal Processing

The mobile application sends PPG data to the backend in frames containing 960 samples recorded at a sampling rate of 64 Hz. Each frame represents 15 seconds of physiological measurements. The incoming signal is first filtered, and pulse peaks are detected using NeuroKit2 [24]. Frames containing fewer than ten detected peaks are discarded because they do not provide enough heartbeat information for reliable processing.

RR intervals are then calculated from the time differences between consecutive pulse peaks. These intervals are cleaned in two stages to identify and correct potentially invalid measurements.

First, intervals shorter than 300 milliseconds or longer than 2000 milliseconds are marked as invalid. These limits correspond to heart rates above 200 or below 30 beats per minute. Second, an interval is also considered invalid if it differs from the previous interval by more than 20% [9].

Invalid intervals are corrected using linear interpolation between the nearest valid intervals within the same frame. This allows the processing pipeline to continue with repaired RR interval values instead of passing invalid measurements directly to the models.

Although the models use the corrected intervals, the system also tracks the quality of the original measurements. For every analysis window, it calculates the proportion of beats that were valid before correction. This value is included in the prediction output as a signal-quality measure. It allows the receiving application to identify predictions that rely partly on reconstructed physiological data.

4.4.2 Causal State

Each incoming heartbeat updates a small set of running statistics used by the inference pipeline. These calculations rely only on the current and previously received beats. Future measurements are not used, ensuring that the processing remains causal during real-time operation.

The component maintains three exponentially weighted moving average (EWMA) baseline trackers, following Equations 3.1 and 3.2. These trackers use half-lives of 60, 300 and 1800 beats, allowing them to capture physiological changes over different timescales. At the beginning of each session, all three trackers are initialised using a population mean RR interval of 780 milliseconds. This allows baseline tracking to begin without requiring a separate user calibration period.

Some of the inputs to the neural network also require standardisation. Instead of using the mean and variance calculated from an entire recording, which would include future information, the component maintains running estimates that are updated as new measurements arrive.

For an incoming value x_t, the running mean and variance are updated using the following equations:

$$\boldsymbol{m_t = m_{t-1} + \alpha\,(x_t - m_{t-1})} \quad \textbf{(4.1)}$$

$$v_t = (1-\alpha)(v_{t-1} + \alpha\,(x_t - m_{t-1})^2) \quad (4.2)$$

$$z_t = \frac{x_t - m_t}{\sqrt{v_t}} \quad (4.3)$$

Here α is obtained from a half-life of 300 beats using Equation 3.2, the mean is seeded with the first value received, and the variance starts at one. Short-term

variability is tracked separately by computing RMSSD and SDNN over the most recent 20 standardised beats, which gives a beat-by-beat measure of local variability.

4.4.3 Feature Vector for the Gradient-Boosted Trees

Each window produces the 25 features listed in Table 4.7, in the order that the trained model expects.

Table 4.7: Feature vector for the gradient-boosted trees

Group	Features	Count
Time-domain HRV	Mean RR, SDNN, RMSSD, pNN50, coefficient of variation	5
Frequency-domain HRV	VLF, LF and HF power, LF/HF ratio, normalised LF	5
Non-linear HRV	Poincaré SD1, SD2 and SD1/SD2	3
Residual	Mean, standard deviation, maximum absolute value, slope and mean square of the deviation from the medium baseline	5
Baseline level	Current level of the fast and the slow tracker	2
Time of day	Sine and cosine of the 24-hour and 90-minute cycles; a fixed curve approximating the typical daily cortisol profile	5

Frequency-domain HRV features are calculated from the RR interval series after resampling it at 4 Hz. Welch's method is then used to estimate the frequency components of the signal. The analysis covers three standard frequency bands: very low frequency (VLF), from 0.003 to 0.04 Hz; low frequency (LF), from 0.04 to 0.15 Hz; and high frequency (HF), from 0.15 to 0.4 Hz [9]. Before these features are passed to the gradient-boosted tree model, they are standardised using the scaling

parameters obtained from the training data. This ensures that the same feature scaling is applied during both training and inference.

4.4.4 Sequence Input for the Neural Network

The neural network uses two separate inputs. The first is a sequence containing 60 consecutive heartbeats, with seven feature channels for each beat, as presented in Table 4.8. This input allows the network to analyse how physiological measurements change across the window.

The second input is a seven-dimensional representation of the time of day, calculated at the final heartbeat of the window. This provides temporal information alongside the physiological sequence, allowing the network to consider time-of-day patterns when estimating the user's stress level.

Table 4.8: Channels of the sequence input

Channel	Definition
rn	Causally standardised RR interval
rm	RMSSD of rn over the last 20 beats
sd	SDNN of rn over the last 20 beats
hr	Instantaneous heart rate, 60000 / RR
rrn	Causally standardised deviation of RR from the medium baseline
tn	Causally standardised skin temperature
trn	Causally standardised deviation of skin temperature from its own medium-timescale average

4.4.5 MS-CGCA Network

The Multi-Scale Circadian-Guided Cross-Attention (MS-CGCA) network is named after two key features of its architecture. The first is its use of convolutions at different temporal scales, which allows the model to capture physiological patterns occurring over different time periods. The second is a cross-attention mechanism guided by time-of-day information. This allows the network to consider temporal

context when analysing physiological signals and predicting stress levels. The individual layers of the MS-CGCA network are presented in Table 4.9.

Table 4.9: Layers of MS-CGCA Network

Stage	Layers	Output
Multi-scale causal convolution	Three parallel one-dimensional convolutions with 32 filters each, kernel size 3, causal padding, dilation rates 1, 2 and 4 and ReLU activation; outputs concatenated	60 × 96
Normalisation and pooling	Batch normalisation; max pooling by 2	30 × 96
Recurrent encoding	Unidirectional LSTM with 128 units returning the full sequence; dropout 0.4	30 × 128
Query from time of day	Dense layer of 128 units applied to the time-of-day encoding and repeated across the 30 time steps	30 × 128
Cross-attention	Dot-product attention with the time-of-day projection as query and the LSTM states as keys and values; global average pooling	128
Time-of-day branch	Dense layer of 32 units applied to the time-of-day encoding	32
Classifier	Concatenation; dense layer of 64 units with dropout 0.4; four-way softmax	4

The convolutional stage uses three dilation rates, giving receptive fields of 3, 5 and 9 beats. This allows the network to capture both short-term and slightly longer patterns in beat-to-beat physiological changes at the same time. Causal padding ensures that the output for each beat depends only on the current beat and those received before it. The LSTM layer also processes the sequence in the forward direction only.

Together, these design choices prevent the network from using future beats when generating its predictions.

The network was trained using focal loss with a gamma value of 2 ($\gamma = 2$), along with balanced class weights. Focal loss was used to reduce the influence of the majority relaxed class and place greater emphasis on samples that were more difficult to classify [18].

Training was performed using the Adam optimiser with a learning rate of 10^{-4} and a batch size of 32. The maximum number of training epochs was set to 120. Of the available training windows, 15% were reserved for validation to monitor model performance during training.

Early stopping was applied when the validation loss failed to improve for 15 consecutive epochs. The learning rate was also reduced by half if no improvement was observed for 7 consecutive epochs. These settings helped control training and reduce unnecessary updates once the model stopped improving.

4.4.6 Gradient-Boosted Trees

The gradient-boosted tree model was implemented using the XGBoost classifier [16]. It was configured with 300 trees, each having a maximum depth of 6, and a learning rate of 0.05. Both row subsampling and column subsampling were set to 0.8, meaning that 80% of the available training samples and features were considered during the respective sampling steps.

To address differences in the number of samples belonging to each stress class, training samples were assigned weights inversely proportional to their class frequency. This reduced the influence of the majority classes and gave greater importance to less frequently represented stress levels during training.

4.4.7 Ensemble and Confidence Gate

The class probabilities of the two models are combined as follows.

$$\mathbf{p} = \boldsymbol{w_{xgb}}\,\mathbf{p}_{\boldsymbol{xgb}} + \boldsymbol{w_{net}}\,\mathbf{p}_{\boldsymbol{net}} \tag{4.4}$$

The deployed weights are $w_{xgb} = 0.15$ and $w_{net} = 0.85$. The weight was selected from a grid between 0.10 and 0.90 in steps of 0.05, and the performance reported for it is

the nested estimate, in which the weight is chosen without the fold being scored. The confidence of a prediction is the margin between the two largest probabilities.

$$c = p_{(1)} - p_{(2)} \tag{4.5}$$

The deployed component reports a single level when c ≥ 0.15 and a band spanning the two most probable levels otherwise. In the experiments of Chapter 6 the threshold was set separately in each fold from the training subjects, whereas the deployed component uses this single fixed value. Each prediction also carries a continuous stress score, defined as the expected level under the blended distribution.

$$s = \sum_{i=0}^{3} i \; p_i \tag{4.6}$$

The continuous score is supplementary. A client that derived a level from it, or from the raw probabilities, would bypass the confidence gate, so the level or band reported by the component is the authoritative output.

4.4.8 Output Message

Each prediction is published as a JSON message with the fields listed in Table 4.10.

Table 4.10: Fields of the prediction message

Field	Content
timestamp	Time of the final beat in the window (POSIX seconds)
windowStart, windowEnd	Times of the first and last beats in the window
heartRate	Mean heart rate of the window in beats per minute
rmssd, sdnn	RMSSD and SDNN of the window in milliseconds
signalQuality	Fraction of the window's beats that arrived valid
stress.mode	"point" or "band"
stress.level, or stress.level_low and stress.level_high	The reported level, or the two levels of the band

stress.label	Name of the level or band, for example "mild-to-moderate"
stress.confidence	Margin between the two most probable levels
stress.adjacent	Whether the two levels of a band are neighbours
stress.probabilities	Blended probability of each level
stress.continuous_score	Expected level under the blended distribution

4.5 Standards and professional practice

The component was developed by following a combination of established external standards and professional best practices. These helped guide the design, implementation and evaluation of the system. Table 4.11. presents the relevant standards and practices, along with the specific areas of the component where each was applied.

Table 4.11: Standards and professional practices applied

Standard or practice	How it was applied	Where
HRV measurement standards of the European Society of Cardiology and the North American Society of Pacing and Electrophysiology [9]	Definitions of time- and frequency-domain features, frequency bands and artefact handling	Feature extraction (§4.4)
WebSocket protocol, RFC 6455 [23]	Persistent bidirectional connections for both data streams	/ingest and /stream (§4.2)
JSON with a declared schema	Every message is validated against a typed schema, and	Server interfaces (§5.2)

	malformed messages are rejected without closing the connection	
Sri Lanka Personal Data Protection Act No. 9 of 2022	Data minimisation: no physiological data is stored or logged	Privacy design (§3.6)
Dataset licence terms	WESAD used for non-commercial research only; attribution given for both datasets	Data use (§3.6)
Version control and code review	All code and notebooks kept in a Git repository	Whole component
Automated testing	Unit, parity, causality and interface tests run with pytest	Verification (§5.4)
Dependency pinning and artefact fingerprints	Library versions constrained to those that can load the trained models; artefacts identified by SHA-256 fingerprints	Deployment (§5.5)
Research integrity	Pre-stated decision rules, subject-level statistics, nested estimates and reporting of negative results	Chapters 3 and 6

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The research experiments and backend development were carried out in separate environments. The experimental notebooks were executed on Kaggle, which provided the computational resources needed to train and evaluate the neural network models across fifteen cross-validation folds and five different random seeds.

The backend was developed on a Windows workstation using Visual Studio Code. It was designed to run on any compatible computer capable of supporting the required Python environment and dependencies. This allows the backend to operate without relying on the Kaggle environment used during the experiments.

Table 5.1 summarises the main technologies used in the research and development process, along with the reasons for selecting each one.

Table 5.1: Technology stack of the component

Technology	**Version**	**Purpose and reason for choice**
Python	3.12 (notebooks)	Single language for research and deployment, so that feature code can be shared between the two
NumPy, SciPy, pandas	NumPy ≥ 1.24, SciPy ≥ 1.11, pandas ≥ 2.0	Numerical processing, spectral analysis and data handling
NeuroKit2	≥ 0.2.7	ECG and PPG cleaning and peak detection [24]
scikit-learn	1.6.1 (pinned)	Feature scaling and evaluation utilities; pinned because the saved scaler must be loaded by the same version that created it
XGBoost	3.2.0 for training; ≥ 3.0 and < 4.0 for	Gradient-boosted tree model [16]

	deployment	
TensorFlow and Keras	2.20.0 for training; ≥ 2.16 and < 3.0 for deployment	MS-CGCA network; Keras 3 model format
FastAPI, Uvicorn, Pydantic	FastAPI ≥ 0.110, Uvicorn ≥ 0.27, Pydantic ≥ 2.6	Asynchronous web server with WebSocket support and typed message validation
pytest	≥ 8.0	Automated testing
Git and GitHub	–	Version control and sharing with the group
Kaggle notebooks	–	Training and evaluation of all experimental configurations

Using Python for both model training and deployment was one of the most important technology decisions in this research. A common problem in machine learning systems is training–serving skew. This occurs when the data processing methods used during deployment differ slightly from those used during training. Such differences can affect prediction accuracy and may be difficult to detect because the system can continue running without producing any errors.

To reduce this risk, the experimental notebooks and the backend server use the same feature extraction functions. This ensures that the processing methods used to generate the reported experimental results are also applied during real-time inference. However, differences in library versions could still affect the results. For this reason, compatibility between the training and deployment environments is checked through the verification tests described in Section 5.4.

5.2 Component Implementation

The component is structured as a Python package containing the validated physiological signal processing and stress inference pipeline. A separate, lightweight server layer provides the network interfaces needed to communicate with the mobile application and VR environment.

This separation allows the core processing and prediction functions to remain independent of the network communication logic. It also makes the component easier to test, maintain and extend without changing the underlying inference pipeline.

Table 5.2. presents the main modules of the component and their responsibilities.

Table 5.2: Main modules of the component

Module	Responsibility
componentb/config.py	Constants that are fixed by the trained models: window length, step, half-lives, population seed, feature order and confidence threshold
componentb/signal/ppg.py	Peak detection in PPG frames, RR derivation and artefact rejection
componentb/baseline/ewma.py	Causal multi-timescale baseline engine
componentb/features/hrv.py	Time-, frequency-domain and non-linear HRV features and residual features
componentb/features/causal.py	Causal standardisation and rolling variability, in batch form for training
componentb/features/channels.py	The same transforms in streaming form, updated once per beat
componentb/features/circadian.py	Time-of-day encodings for both models
componentb/models/loader.py	Loading of models, scaler and ensemble weights, with a check of the exported configuration
componentb/inference/stream.	Streaming inference engine: buffering, windowing,

py	ensemble and confidence gate
server/main.py	HTTP and WebSocket endpoints
server/schemas/messages.py	Typed schemas of the incoming frames and outgoing predictions
server/state.py	Single in-memory slot holding the latest prediction

5.2.1 Single Source of Truth

The Python package serves as the single source of truth for the processing and inference pipeline. Both the training notebooks and the live backend server import their functions from this package. This avoids implementing the same processing steps separately in different files or programming languages, which could introduce inconsistencies between training and deployment.

This design is particularly important because some of the transformations used by the models depend on information collected throughout a session. For example, running standardisation and exponentially weighted moving averages maintain values that are continuously updated as new heartbeats arrive. If these calculations were repeated using only the current 60-beat window, the results could differ from those generated during training. Such differences might not produce an error, but they could affect the accuracy of the predictions.

To prevent this problem, the streaming module maintains the state of each transformation and updates it as new beats are received. Parity tests are also used to verify that the streaming implementation produces the same outputs as the batch processing method used during training. This helps maintain consistency between experimental evaluation and real-time inference.

5.2.2 Streaming Inference Engine

The streaming inference engine separates the processing of incoming heartbeats from the generation of stress predictions. Each new beat is passed to an observation step, where it is added to fixed-length buffers and used to update the running statistics. The model does not generate a prediction for every individual beat.

Instead, inference is triggered only when a complete analysis window is available and the required step interval has been reached. At that point, the engine indicates that a new prediction is needed, and the ensemble models are executed.

This approach keeps the processing required for each incoming beat relatively small and ensures that predictions are generated at clearly defined intervals. It also avoids running the models unnecessarily between prediction points.

A separate inference engine is created for each connection to the ingestion endpoint. Each engine maintains its own buffers, baseline trackers and running statistics. This prevents physiological measurements and causal states from different users from being mixed during processing.

5.2.3 Server Layer

The backend server was developed using FastAPI and runs with Uvicorn. Incoming PPG frames are checked against a predefined data schema before processing. Each frame must contain exactly 960 valid and finite PPG samples, a positive timestamp and a sampling rate of 64 Hz.

If a frame does not meet these requirements, the server returns a message explaining the validation error. However, the connection remains open. This allows the session to continue receiving new data instead of being interrupted by a single invalid frame.

Loading the TensorFlow and XGBoost models can take several seconds. To prevent this operation from blocking other server activities, model loading is performed outside the main event loop. This allows other clients to continue connecting and receiving predictions while a new session is being initialised.

The server also handles situations where the required model files are unavailable. In such cases, it reports that the models cannot be loaded but continues to accept and validate incoming data. This allows the communication and data-processing paths to be tested independently of the trained models.

5.2.4 Model Artefacts

The trained models and their supporting files are exported from the experimental notebooks for use during deployment. Four main artefacts are maintained: the neural

network saved in Keras format, the XGBoost model saved as a JSON file, the fitted feature scaler and a configuration file containing the ensemble weights, window settings and expected feature order.

Each artefact is associated with a SHA-256 fingerprint recorded in the project documentation. These fingerprints help verify file integrity and ensure that the correct model files are used during deployment.

When the backend starts, the model loader checks whether the exported configuration matches the settings defined in the implementation. If there is a mismatch, the loader refuses to run the models. This prevents predictions from being generated using incompatible settings or an incorrect feature arrangement.

The model artefacts are stored in the project repository rather than being regenerated whenever the server is started. This is important because retraining a neural network may produce slightly different model weights, even when the same training procedure is followed. Using files from different training runs could also create a model combination that was never evaluated experimentally.

By preserving the validated artefacts together, the deployed component uses the same model configuration that was tested during the research.

5.3 Integration with the Other Components

The stress inference component communicates with three other parts of the complete system through the network interfaces described in Section 4.2. These connections allow physiological measurements to reach the backend and enable the resulting stress predictions to be delivered to the applications that use them.

The mobile application sends physiological measurements collected from the wearable sensor to the backend. It also receives stress-related predictions and displays them to the user. The VR meditation application receives the same predictions and uses them to adapt the virtual environment according to the estimated stress level. These interactions allow the stress inference component to operate as part of the complete closed-loop system.

Table 5.3. summarises the connections between the stress inference component and the other system components, including the purpose of each interface.

Table 5.3: Integration points with the other components

Consumer or source	Interface	Integration behaviour
Mobile application (React Native)	/ingest; /stream	Assembles the PPG samples received from the wearable into frames of exactly 960 samples and sends one frame every 15 seconds; displays the stress estimates to the user
VR meditation application (Unity, Meta Quest 2)	/stream	Maintains a WebSocket connection, parses each prediction and handles both the point and the band forms of the stress output
Voice-based stress component	/stress/latest	Requests the latest prediction at defined moments of its own session and combines it with its own estimate; treats a band as an uncertain reading

The communication interface was defined before the other system components were developed to use it. This ensured that all components followed the same data formats and communication requirements. The incoming physiological data format and outgoing prediction messages were defined using typed schemas in the backend. These schemas were also made available through the server's automatically generated API documentation, which other group members used as a reference when developing their applications.

Each application receiving stress predictions was required to support both output formats. The first provides a single stress level, while the second provides a band covering two possible stress levels when the prediction is uncertain. Supporting both formats was important because an application designed to handle only single-level predictions could fail when the backend returned a two-level band.

Integration testing was performed in stages. Initially, the backend was tested independently by sending previously recorded physiological data and artificially generated test signals to the ingestion endpoint. Small driver and viewer programs were developed to simulate the mobile application and VR headset. This allowed the complete inference process, from receiving physiological data to generating and delivering predictions, to be tested before the other system components were available.

Once the backend had been tested independently, the mobile and VR applications were developed to communicate through the same endpoints. Each application also included its own tests to verify that incoming messages could be correctly interpreted. This approach helped identify communication and data-format issues separately from problems in the inference pipeline. The results of the backend-level integration tests are presented in Section 6.4.

A full-system integration test was conducted before progress presentation 2 to verify communication between the wearable sensing unit, mobile application, backend stress inference service and VR application. The test involved the wearable device developed for the project, an Android smartphone, an MSI Cyborg 15 A13UCX laptop running the Python backend and a Meta Quest 2 headset.

During testing, the wearable device collected physiological measurements and transmitted them to the mobile application using Bluetooth Low Energy (BLE). The mobile application then forwarded the received data to the Python backend through a network connection. The backend processed the incoming measurements, updated the causal physiological state and formed analysis windows from the collected heartbeats. Once enough data was available, the inference models generated a stress estimate using four possible stress levels, along with confidence values and class probabilities.

The resulting predictions were sent to the connected applications. The mobile application successfully received the latest stress prediction and displayed the information to the user. At the same time, the Meta Quest 2 application received the same prediction data for use in adapting the VR meditation environment. This

confirmed that physiological measurements collected from the wearable could be processed and delivered through the complete system without requiring manual transfer of data between components.

Some integration issues were identified during the initial tests. One problem involved differences in timestamp formats and field names used by the mobile application and backend. These inconsistencies were resolved by standardising the message structure across the components.

Another issue occurred when the network connection was temporarily interrupted. This caused the backend and the connected applications to lose synchronisation. To address this, reconnection handling in the mobile and VR clients was tested to ensure that physiological data transmission and stress prediction updates could resume after the connection was restored.

The final integration test verified the following communication flow:

Wearable → Mobile Application → Python Backend → Stress Inference → Mobile Application and Meta Quest 2

Overall, the test confirmed that the stress inference component could receive live physiological data, generate predictions and deliver the results to both the mobile and VR applications. This demonstrated that the component could function within the complete adaptive VR meditation system rather than operating only as a separate offline machine learning model.

5.4 Verification and software testing

Verification was carried out at two levels to ensure that both the research methods and the deployed software behaved as intended. At the research level, the checks described in Section 3.7 confirmed that the baseline estimators operated causally and that forecasting targets were taken from the correct subjects. This prevented future physiological measurements and data from other subjects from affecting the experimental results.

At the software level, an automated test suite was developed to verify that the deployed component followed the same processing methods used in the training

notebooks. The tests also checked that the system did not rely on future data and that its network interfaces behaved according to the defined requirements. Table 5.4. summarises the automated tests and the areas they cover.

Table 5.4: Automated Test Suite

Test file	Tests	What is verified
test_parity.py	15	The saved models reproduce the exported predictions; the streaming engine produces the same features, channels and probabilities as the batch pipeline on a reference fixture of 200 windows; windows end at their final beat; the blend uses the shipped weights
test_feature_parity.py	34	Every HRV and residual feature, and the full 25-dimensional vector, match the notebook implementation; feature names and order match the exported configuration
test_causality.py	6	The baseline engine and every sequence transform give identical past output when future samples are corrupted; a new user starts from the population seed
test_api.py	16	Frame validation, rejection of malformed frames without closing the connection, separation of state between connections, model loading off the event loop, behaviour of /stress/latest before and after the first window, and the content of the prediction message
test_temperature.py	3	Handling of the skin-temperature value supplied with each frame
test_engine.py	1	Missing model files are reported without stopping the server

The parity tests use strict numerical tolerances to check consistency between training and deployment. A tolerance of 10^{-9} is used for individual features, while 10^{-5} is used for model probabilities. The slightly larger tolerance for probabilities accounts for small floating-point differences that may occur during neural network inference.

The causality tests follow the same approach used in the research experiments. Future samples are deliberately modified, and the earlier outputs are compared with those generated from the original data. If the earlier results remain unchanged, it confirms that the processing does not depend on information from future measurements. Applying this test to the deployed implementation helps ensure that the causality established during the experiments is preserved in the software.

The complete test suite was executed using a fresh copy of the repository in a clean Linux environment with Python 3.13 and TensorFlow 2.21. A total of 75 tests were included, of which 74 passed and 1 was skipped.

The skipped test compares the streaming inference engine against a segment of raw WESAD data. This dataset is not included in the repository because of its licensing restrictions. However, the test is designed to run automatically when the required dataset is available locally.

The verification environment used a newer TensorFlow version than the TensorFlow 2.20.0 environment used during model training. Since the parity tests passed, the results also confirmed that the saved models could be loaded and reproduce their exported predictions within the specified numerical tolerances under TensorFlow 2.21.

Overall, these tests provide evidence that the deployed component maintains consistency with the training pipeline, follows the required causal processing rules and handles its network interfaces as intended. They also demonstrate that the validated model artefacts can operate in the tested deployment environment without requiring retraining.

5.5 Deployment and operation

The component is deployed as a single server process running on a computer connected to the same local network as the mobile application and VR headset. An internet connection is not required during operation because all communication takes place within the local network. This allows the system to operate without depending on external internet connectivity.

All trained models and their supporting files are stored in the project repository. As a result, the backend can be started from a fresh copy of the repository without downloading additional model files.

Setting up the component involves four main steps. First, the required Python dependencies are installed using the pinned requirements file. Next, the backend server is started on a selected network port. The server computer's network address is then configured in both the mobile application and the VR headset. Finally, a session begins when the mobile application establishes a connection to the backend's ingestion endpoint and starts sending physiological measurements.

The system generates its first stress prediction after receiving 60 heartbeats, which takes approximately 45 seconds at a resting heart rate. Until enough data has been collected, the `/stress/latest` endpoint returns HTTP status code 503 to indicate that a prediction is not yet available. The health endpoint can also be used by other components to check whether the backend server is running before starting a session.

During operation, the backend reports problems through status messages instead of immediately closing the connection. For example, an invalid incoming frame, unsuccessful heartbeat detection and missing model files are reported as separate conditions. This helps the mobile application identify the cause of a problem and allows communication to continue when recovery is possible.

Two important rules must be followed when operating the component. First, applications receiving stress predictions must support both output formats: a single stress level and a band covering two possible levels. They should use the output provided by the confidence gate rather than calculating a stress level directly from the class probabilities or continuous stress score. Doing so would bypass the

uncertainty-handling mechanism and could produce a result that differs from the backend's intended prediction.

Second, the backend should only be operated on a trusted local network. As discussed in Section 3.6, the current implementation does not use encrypted or authenticated connections. Running it on an open or untrusted network could expose physiological data and prediction messages during transmission.

The component also avoids retaining session information after use. Physiological processing states and the most recent prediction are maintained only in memory while the session is active. When the connection is closed, the session-specific state is cleared, and no physiological recordings or session history are stored on the server. This reduces unnecessary data retention and supports the privacy-focused design of the component.

6 RESULTS AND VALIDATION

6.1 Research results

This section presents the results of the experiments described in Chapter 3. The first part investigates why the Cosinor features improved model performance and whether this improvement was actually related to circadian rhythms. The second part examines how model performance changed when moving from offline experiments to a system suitable for real-time deployment.

Unless stated otherwise, all results were obtained using leave-one-subject-out cross-validation on the fifteen WESAD participants. Quadratic weighted kappa is represented by κ, and all statistical tests were performed using subject-level scores, following the methods described in Section 3.5.

6.1.1 Temporal-Basis Audit

The first step was to determine whether the available recordings were long enough to identify a 24-hour circadian rhythm. The WESAD recordings had an average duration of 1.50 hours, while the longest recording lasted 1.74 hours. On average, each recording covered only 22.4° of a complete 24-hour cycle, which represents approximately 6.2% of a day. Therefore, none of the recordings contained enough information to observe a full circadian rhythm.

When a 24-hour Cosinor model is fitted to such a short recording, it can still produce numerical results without reporting an error. However, these results may not be physiologically meaningful. In this experiment, the estimated MESOR values ranged from 1,325 ms to 25,811 ms across subjects. In comparison, the actual mean RR intervals for the same recording sessions ranged from 653 ms to 983 ms.

A MESOR value exceeding 25 seconds per heartbeat is not physiologically realistic. This large difference occurred because a small section of a 24-hour cosine curve can closely resemble a straight line. With only about 1.5 hours of data, the model cannot reliably distinguish between different combinations of baseline level and rhythm amplitude. As a result, it may fit the observed data while producing unrealistic parameter estimates.

These findings showed that a reliable 24-hour circadian rhythm could not be identified from the available WESAD recordings. They also raised an important question about the earlier performance improvements associated with Cosinor features. Since the fitted rhythm itself was unreliable, the improvement was likely caused by some other information captured by those features rather than by accurate circadian modelling.

6.1.2 Mechanism Decomposition

To investigate the source of this improvement, the temporal features were separated into different groups and evaluated using XGBoost. All configurations were tested using the same leave-one-subject-out cross-validation folds to ensure a consistent comparison.

This experiment examined the contribution of different temporal features to model performance, rather than treating them as a single combined set. Table 6.1. presents the results of this additive decomposition and shows how each feature group affected the model's performance.

Table 6.1: Additive decomposition of the temporal features (XGBoost, WESAD)

Configuration	Macro-F1	SD	κ
HRV features only	0.464	–	0.482
HRV + time-of-day encoding (baseline)	0.551	0.148	0.668
Baseline + fitted MESOR and amplitude	0.558	–	0.696
Baseline + fitted MESOR + residual	0.642	–	0.834
Baseline + session mean and SD	0.602	0.162	0.777
Baseline + session mean and SD + residual	0.643	0.135	0.838
Baseline + residual	0.646	0.140	0.834

SD represents the standard deviation of macro-F1 scores across the fifteen cross-validation folds, where individual fold results were available. Compared with the baseline model, adding residual features improved macro-F1 by 0.095, while

including the fitted MESOR increased it by only 0.007. This suggests that residual features contributed much more to the improvement in model performance.

However, the fitted MESOR values were shown to be unreliable in the temporal-basis audit. Comparing residual features only against these values could therefore make their contribution appear larger than it actually was. To address this, the analysis was repeated using a more reliable measure of the subject's physiological baseline. This measure included the mean and standard deviation of the RR intervals recorded during the session.

The new baseline-level features improved macro-F1 by 0.051, which was approximately seven times the improvement obtained from the fitted MESOR. This showed that information about a subject's baseline was useful on its own. However, a different result appeared when baseline-level features were added to a model that already included residual features.

With residual features alone, the model achieved a macro-F1 of 0.646. Adding the fitted MESOR reduced the score slightly to 0.642, while adding the session mean reduced it to 0.643. In both cases, including additional baseline information did not improve performance. This suggests that once the model knows how much the current RR intervals differ from the subject's baseline, the baseline level itself provides little additional predictive value.

When a reliable baseline-level measure was used, residual features still contributed almost twice as much to the improvement in macro-F1, with gains of 0.095 and 0.051, respectively. However, the standard deviations across subjects ranged from 0.135 to 0.162, which were relatively large compared with the small performance differences between the best configurations. For this reason, the overall pattern of results was considered more informative than the exact differences in their scores.

The analysis was also repeated using lower levels of overlap between consecutive windows. At overlap levels of 83%, 67% and 50%, residual features continued to provide greater improvements than the baseline-level features. The corresponding contribution ratios were 2.3, 1.6 and 1.8. Although the absolute improvement from

residual features decreased from 0.096 to 0.061 as window overlap was reduced, their advantage over baseline-level features remained.

6.1.3 Timescale Invariance

If the improvement in model performance was genuinely caused by circadian rhythms, it would be expected to depend on using the correct 24-hour period. To investigate this, the residual features were recalculated using different assumed rhythm periods. Two additional methods that did not require a fixed period were also evaluated.

This experiment examined whether the improvement depended specifically on a 24-hour circadian pattern or whether similar results could be obtained using other approaches to baseline estimation. Table 6.2. presents the results of these comparisons.

Table 6.2: Performance against the assumed detrending period (XGBoost)

Assumed period	Macro-F1	κ
24 h	0.645	0.838
12 h	0.639	0.834
6 h	0.647	0.845
3 h	0.640	0.840
1 h	0.576	0.773
30 min	0.583	0.809
15 min	0.587	0.827
Cubic polynomial (no period)	0.624	0.826
Adaptive period per subject	0.625	0.812

The results can be divided into two groups based on the assumed rhythm period. Longer periods, ranging from 3 to 24 hours, produced very similar results, with macro-F1 scores differing by no more than 0.008. The difference between the 12-

hour and 24-hour periods was also not statistically significant at the subject level ($p = 0.107$, $d = 0.35$). In contrast, shorter periods of 15 to 60 minutes consistently performed worse. Compared with the 24-hour period, they produced an average reduction of 0.059 in macro-F1, and this difference was statistically significant ($p = 0.0067$).

These findings can be explained by the mathematical behaviour of the fitted curves rather than by physiological rhythms. For recordings lasting approximately 1.5 hours, a cosine curve with a period of several hours or longer closely resembles a smooth trend. As a result, the longer periods remove similar slow changes from the RR interval data and produce similar residual features.

Shorter periods behave differently. They complete one or more cycles within the recording and follow faster changes in the physiological signal. This causes them to remove not only the slow trend but also some of the variations that may contain useful information about stress. Consequently, the model loses part of the information needed for classification.

A polynomial trend that did not depend on any fixed period also performed reasonably well, with a macro-F1 score within 0.025 of the periodic approaches. Selecting a separate period for each subject did not provide any additional improvement. Overall, the results showed that model performance depended mainly on how much smoothing was applied to the physiological signal, rather than on identifying a specific circadian rhythm.

6.1.4 Statistical Validation and Per-Subject Analysis

The statistical analysis provided further evidence that the temporal residual features improved model performance. A Friedman test comparing the main model configurations found a statistically significant overall difference ($\chi^2 = 21.64$, $p = 0.0001$).

Adding residual features to the HRV baseline produced a large and statistically significant improvement, with a Holm-corrected p-value of 0.0005 and an effect size of $d = 1.28$. The improvement was also consistent across most participants. Macro-

F1 increased for 14 of the 15 subjects, with an average improvement of 0.108. Only one subject experienced a decrease, which was 0.025.

The bootstrap confidence intervals calculated at the subject level showed considerable variation in performance between individuals. The HRV baseline achieved a macro-F1 of 0.551, with a reported 95% confidence interval of [0.423, 0.568]. After adding residual features, macro-F1 increased to 0.646, with a confidence interval of [0.534, 0.670].

These intervals were relatively wide because model performance differed across subjects. Resampling individual windows instead would have produced narrower intervals, but this would incorrectly treat highly overlapping windows as independent observations. For this reason, only subject-level bootstrap confidence intervals were reported.

The analysis also examined whether combining models provided an additional improvement. The two-way ensemble did not perform significantly better than the neural network alone ($p = 0.52$, $d = 0.34$). This suggests that the main performance improvement was associated with the temporal features rather than simply increasing the number of models or the overall model capacity.

The per-subject analysis was then used to investigate whether the improvement could be explained by circadian rhythms. If the residual features worked by correcting a circadian rhythm, subjects whose physiological data fitted the estimated rhythm more closely would be expected to show greater benefits.

However, the results did not support this explanation. The quality of the 24-hour Cosinor fit to each subject's RR intervals was not significantly correlated with their macro-F1 score ($r = 0.374$, $p = 0.170$). A similar result was found for skin temperature ($r = 0.227$, $p = 0.416$), even though the fitted temperature rhythms were considerably better.

A different relationship was observed when the proportion of windows belonging to minority stress classes was examined. This variable showed a strong positive correlation with subject-level macro-F1 ($r = 0.775$, $p = 0.001$). Subjects who

contributed a higher proportion of minority-class stress data tended to achieve better classification performance.

These findings suggest that differences in model performance across subjects were more closely related to the distribution of stress classes in their recordings than to how well their physiological data followed a fitted circadian rhythm. This provides further evidence that the benefits of temporal residual features were not primarily caused by circadian correction.

6.1.5 Cross-Dataset Replication and Label Sensitivity

To examine whether the findings were consistent across different datasets, the temporal feature decomposition was repeated using the wrist-PPG dataset by Hongn et al. [15].

This experiment investigated whether the contributions of baseline-level and residual features observed in WESAD could also be found in physiological data collected from a different participant population and wearable sensor. Table 6.3. compares the contributions of these features across the two datasets.

Table 6.3: Contribution of the fitted MESOR and the residual on the two datasets

Contribution over baseline	WESAD (chest ECG)	Hongn et al. (wrist PPG)
Fitted MESOR	+0.007	+0.013
Residual	+0.095	+0.050

The results from the wrist-PPG dataset showed lower overall classification performance than those obtained from WESAD. Macro-F1 increased from 0.304 for the baseline model to 0.355 when the complete feature set was included. These lower scores may be related to the greater noise present in wrist-based physiological signals and the use of proxy stress labels derived from RMSSD quartiles.

Despite the lower performance, the overall pattern remained similar to that observed in WESAD. The residual features contributed considerably more to the improvement than the fitted baseline features. This suggests that the benefit of residual features is

not limited to the WESAD dataset or to physiological measurements collected using chest-based ECG sensors. Similar behaviour was also observed using wrist-based PPG data.

Another concern was how the stress labels were generated. The four intensity levels within the stress condition were created using a short-term heart-rate variability measure. Since RMSSD was also included among the model's input features, there was a possibility that the observed improvement was influenced by a circular relationship between the labels and the features used for prediction.

To investigate this possibility, the feature decomposition was repeated using binary stress labels taken directly from the experimental protocol. These labels separated stress from non-stress conditions without relying on physiological measurements to define the categories. This allowed the contribution of the temporal features to be examined independently of the RMSSD-based labelling method. The results of this comparison are presented in Table 6.4.

Table 6.4: Decomposition with protocol-defined binary labels (XGBoost, WESAD)

Configuration	Macro-F1	κ
HRV + time-of-day encoding (baseline)	0.866	0.731
Baseline + fitted MESOR	0.875	0.750
Baseline + residual	0.955	0.911
Baseline + fitted MESOR + residual	0.956	0.912

The binary classification task produced higher overall scores because distinguishing between stress and non-stress is simpler than predicting four different stress levels. However, the contribution of the temporal features followed the same pattern as in the earlier experiments. Adding residual features improved macro-F1 by 0.090, while the fitted MESOR contributed only 0.009.

These results show that the benefit of residual features was not dependent on the method used to create the four stress intensity levels. Even when the labels were taken directly from the experimental protocol, residual features remained more useful

than the fitted MESOR. This provides further support for the finding that the improvement was mainly associated with baseline-relative physiological changes rather than circadian modelling.

6.1.6 Ordinal Error Structure

Adding residual features improved more than just the overall classification accuracy. It also changed the types of mistakes made by the model. In particular, errors involving stress levels that were two or more categories apart became less common.

The proportion of these larger errors decreased from 42.3% to 27.5%. The macro-averaged mean absolute error also fell from 0.675 to 0.467, indicating that incorrect predictions were generally closer to the actual stress level. In addition, the rate of incorrectly detecting stress during relaxed periods decreased from 6.7% to 1.7%.

These changes help explain why quadratic weighted kappa (κ) showed a greater improvement than macro-F1. While macro-F1 measures classification performance across the stress classes, quadratic weighted kappa also considers how far an incorrect prediction is from the correct level. Therefore, reducing large classification errors has a stronger effect on κ.

A further experiment examined whether explicitly modelling the order of stress levels could improve performance. For this purpose, the neural network's standard softmax output was replaced with a rank-consistent ordinal classification head [25]. This approach allows the model to consider the natural ordering of the stress categories rather than treating every class as completely independent.

The ordinal approach reduced the proportion of distant classification errors from 5.4% to 4.2%. Quadratic weighted kappa also increased from 0.794 to 0.822. However, the improvement in macro-F1, from 0.619 to 0.630, was not statistically significant ($p = 0.60$).

These findings suggest that explicitly modelling the order of stress levels helped reduce the severity of classification errors, even though it did not produce a measurable improvement in the overall frequency of correct classifications.

6.1.7 Cost of the Labelling Convention

The deployment-related experiments began by examining how the choice of labelling method affected model performance. Two approaches were compared: midpoint labelling, which assigns a window the stress level at its centre, and endpoint labelling, which uses the stress level at the final beat of the window.

Both methods were evaluated using the same analysis windows, extracted features and leave-one-subject-out cross-validation folds. Each model configuration was also trained using five different random seeds to account for variation between training runs.

This comparison was designed to measure how much the choice of labelling convention influenced the reported performance, while keeping the other experimental conditions unchanged. Table 6.5. presents the results for midpoint and endpoint labelling across the evaluated model configurations.

Table 6.5: Inflation from midpoint labelling relative to endpoint labelling

Configuration	Δ Macro-F1	Δ κ
Gradient-boosted trees alone	+0.084	+0.075
MS-CGCA two-way ensemble	+0.071	+0.034
Bidirectional two-way ensemble	+0.076	+0.034
MS-CGCA three-way ensemble	+0.079	+0.042

The increase in macro-F1 caused by midpoint labelling was observed across all model configurations, with similar differences in each case. The improvement was approximately 0.08 in macro-F1, which is larger than many of the performance differences reported between model architectures in stress detection research. This shows how strongly the choice of labelling method can affect evaluation results. Using endpoint labelling avoids this source of inflated performance without requiring additional computational resources.

A related effect was also observed in how the stress intensity labels were generated. When the intensity statistic was calculated using only past observations instead of a

centred window, macro-F1 increased by approximately 0.056. This happened because the past-only statistic was more closely aligned with the physiological features used as model inputs. However, the centred definition was retained throughout the experiments to maintain consistency and allow fair comparisons between configurations.

6.1.8 Window Length and Model Configuration

Reducing the analysis window from 120 beats to 60 beats decreased the prediction delay from approximately 90 seconds to 45 seconds. The shorter window also improved classification performance. Macro-F1 increased by 0.034 for the MS-CGCA ensemble and by 0.075 for the bidirectional ensemble.

These results showed that a shorter window could improve both prediction responsiveness and accuracy. This was an important finding for real-time deployment, where the system needs to respond to changes in the user's physiological state without unnecessary delay.

Following this experiment, four configurations suitable for deployment were compared using endpoint labelling. Each configuration was evaluated with five different training seeds, using identical evaluation windows to ensure a consistent comparison. Table 6.6. presents the results of these configurations.

Table 6.6: Deployable configurations (endpoint labelling, five seeds)

Configuration	Macro-F1	κ
Bidirectional two-way ensemble	0.5993 ± 0.0120	0.7897 ± 0.0232
MS-CGCA three-way ensemble	0.5991 ± 0.0109	0.7773 ± 0.0225
MS-CGCA two-way ensemble (deployed)	0.5925 ± 0.0129	0.7755 ± 0.0174
Gradient-boosted trees alone	0.5703 ± 0.0040	0.7229 ± 0.0036

The results showed no statistically significant differences between the three ensemble configurations, with all pairwise p-values being 0.625 or higher. The performance differences were also smaller than the standard deviations observed

across the five training seeds. This suggests that none of the ensembles had a clear accuracy advantage over the others.

When used alone, the gradient-boosted tree model performed worse than all three ensembles, with macro-F1 scores lower by between 0.022 and 0.029. Each comparison produced a p-value of 0.0625, which was the smallest attainable value for the two-sided Wilcoxon signed-rank test using five paired seeds. Although the ensembles achieved higher scores, these differences did not reach the significance threshold of 0.05.

The effect of personalisation was also examined under both labelling conventions. With endpoint labelling, adding a third personalised model improved macro-F1 by only 0.0066, and the difference was not statistically significant ($p = 0.625$). However, the same comparison under midpoint labelling had suggested a larger improvement of 0.031. This indicates that much of the apparent benefit of personalisation was related to the choice of labelling convention rather than a consistent improvement from the personalised model itself.

The MS-CGCA network also did not outperform the bidirectional neural network it was compared against. Its macro-F1 was lower by 0.0068, but this difference was not statistically significant ($p = 0.625$). Therefore, MS-CGCA was not selected because of higher prediction accuracy. Instead, it was chosen for its causal architecture, which provides stronger protection against using future physiological measurements during inference. The reasons for this decision are discussed in Section 4.3.

6.1.9 Cold-Start Seeding

When a new user begins a session, the system has no previous physiological measurements from that person to initialise the causal baseline trackers. Different initialisation methods were therefore evaluated to determine which approach would provide the most reliable starting point.

Table 6.7. compares four methods of initialising the causal baseline for a new user. These experiments were conducted using only the gradient-boosted tree component, allowing the effect of each seeding method to be examined without the influence of the neural network or ensemble combination.

Table 6.7: Cold-start seeding of the causal baseline (endpoint labelling)

Seeding strategy	Macro-F1	κ
No causal baseline	0.496	0.615
Population mean (deployed)	0.565	0.737
Most similar training subject	0.563	0.731
Subject's own session mean (not deployable)	0.552	0.730

Using a causal baseline improved macro-F1 by 0.068 compared with using HRV features without a baseline. This shows that including information about the user's changing physiological baseline can improve stress classification performance.

The experiment also examined whether selecting the most similar training subject could provide a better starting point for a new user. However, this approach did not produce a statistically significant improvement over using the population mean. The difference in macro-F1 was only +0.0077, with a p-value of 0.934 and an effect size of $d = 0.20$. Interestingly, the same comparison under midpoint labelling had shown a statistically significant result against donor matching ($p = 0.0026$). This difference further demonstrates how the choice of labelling method can influence experimental conclusions.

An unexpected result was observed when the baseline trackers were initialised using the subject's own session mean. This approach performed worse than using the population mean, with a reduction of 0.0186 in macro-F1. The difference was statistically significant ($p = 0.0054$), with an effect size of $d = -0.71$. Performance was lower for 11 of the 15 subjects.

One possible explanation is related to how quickly the baseline trackers adapt to new physiological measurements. The fast and medium trackers gradually lose the influence of their initial values within a few hundred beats. However, the slow tracker has a half-life of 1800 beats, meaning that its starting value continues to influence the baseline throughout much of the recording. The value of this tracker is also included in the model's input feature vector.

When the slow tracker is initialised using the population mean, it retains information about how the user's physiological baseline differs from the overall population. In contrast, initialising it with the subject's own session mean may remove this difference, potentially reducing the information available to the model.

This explanation remains a hypothesis because the effect of the slow tracker's initialisation was not tested separately. Further experiments would be needed to confirm whether this is the reason population-mean seeding performed better.

6.1.10 Confidence-Gated Output and Forecasting

The confidence gate was evaluated to determine how prediction reliability changes when the system is allowed to report a single stress level only for sufficiently confident predictions.

Table 6.8. presents the results obtained using different confidence thresholds. As the threshold increases, fewer windows receive a single-level prediction. This proportion is referred to as coverage. Windows that do not meet the confidence requirement are instead represented by a band covering the two most likely stress levels.

Both macro-F1 and the severe-error rate were calculated using only the windows that received single-level predictions. This allowed the analysis to examine how the reliability of these predictions changed as the confidence threshold was adjusted.

Table 6.8: Confidence gating at different coverage levels

Coverage	Macro-F1	Δ Macro-F1	Severe-error rate	Δ Severe
1.000	0.5647	–	0.0674	–
0.950	0.5748	+0.0100	0.0606	−0.0068
0.899	0.5804	+0.0157	0.0522	−0.0151
0.849	0.5868	+0.0221	0.0435	−0.0239
0.799	0.5899	+0.0251	0.0337	−0.0336
0.750	0.5832	+0.0184	0.0236	−0.0437

0.699	0.5520	−0.0128	0.0121	−0.0552

Macro-F1 for the windows receiving single-level predictions reached its highest value at approximately 80% coverage. At this point, the severe-error rate was reduced to half of the rate observed at full coverage. Although the improvement in macro-F1 was relatively small compared with the variation across training seeds reported in Table 6.6, the reduction in severe errors was still important. For an adaptive VR meditation environment, reducing errors that place a user's stress level two or more categories away from the correct level can help the system respond more appropriately.

However, performance declined sharply when coverage fell below 70%. Macro-F1 dropped to 0.359 at 60% coverage and further decreased to 0.250 at 50% coverage. This happened because fewer windows from the less common stress classes received single-level predictions, leaving insufficient samples for reliable classification performance. These results show that increasing the confidence threshold too much can reduce the effectiveness of the system.

Further analysis was carried out on the 20% of windows where the model had the lowest confidence. Among the classification errors in this group, 66.9% occurred between adjacent stress levels (n = 1,469). These are the types of errors that can be handled by reporting a band covering the two most likely levels instead of forcing a single prediction.

Overall, the findings support the use of a confidence gate to reduce severe classification errors while maintaining a reasonable proportion of single-level predictions.

Table 6.9. compares the performance of the short-term stress forecasting model with a simple persistence baseline, which assumes that the current stress level will remain unchanged over the prediction period.

Table 6.9: Forecasting against persistence (endpoint labelling)

Horizon	Model macro-F1	Persistence macro-F1	Δ

About 30 s	0.466	0.543	−0.077
About 60 s	0.453	0.499	−0.046
About 120 s	0.420	0.453	−0.033

The forecasting model was worse than persistence at every horizon. On this dataset and at these horizons, there was nothing for the model to forecast beyond the fact that stress levels change slowly.

6.2 Product and technical results

6.2.1 Deployed Operating Point

The final deployed configuration uses a two-way ensemble combining gradient-boosted trees with the causal MS-CGCA neural network. It processes physiological data using 60-beat windows and follows endpoint labelling to ensure that predictions are based only on information available during real-time operation.

Under leave-one-subject-out cross-validation, the exported models achieved a macro-F1 score of 0.5923 and a quadratic weighted kappa (κ) of 0.7525. These results were obtained using nested evaluation, where the ensemble blending weight was selected without using data from the subject being evaluated. This helped prevent the evaluation results from influencing the model selection process.

The pooled estimate would have been 0.0100 higher, but it was not reported as the final performance result because it could provide an overly optimistic estimate. The exported model's macro-F1 score was also consistent with the five-seed results presented in Table 6.6, which reported a mean of 0.5925 and a standard deviation of 0.0129.

Across different model exports, the selected weight for the gradient-boosted tree component varied between 0.15 and 0.20. However, these changes did not produce meaningful differences in performance. This indicates that the ensemble performed similarly across a range of blending weights, rather than depending on one exact value to achieve its best result.

The deployed performance should also be considered alongside the earlier offline research result of 0.646 macro-F1 reported in Table 6.1. However, these two scores were obtained under different experimental conditions and should not be directly compared.

The offline research configuration used 120-beat windows, midpoint labelling and a baseline calculated from the entire recording. In contrast, the deployed configuration uses shorter windows, endpoint labelling and causal baseline tracking. These changes ensure that the deployed system operates using only physiological information available at the time of prediction.

The labelling convention alone was found to affect macro-F1 by approximately 0.08, as shown in Table 6.5. This explains a substantial part of the difference between the offline and deployed results. The lower score of the deployed configuration should therefore not be interpreted simply as a loss of model quality. Instead, it reflects the additional restrictions required for real-time inference.

Overall, the macro-F1 score of 0.5923 provides a more realistic estimate of the component's expected classification performance when applied to previously unseen users during live operation, although actual performance may still vary between individuals and operating conditions.

6.2.2 Processing Performance

The processing time of the deployed component was evaluated by sending 40 consecutive PPG frames through the inference engine using the exported models. Each frame contained 15 seconds of physiological data, giving a total of ten minutes of signal.

The test signal was generated using the project's demonstration tool, which creates a PPG waveform from a controlled sequence of heartbeats. This allowed the processing time to be measured under consistent conditions. The generated signal was used only to evaluate processing performance and was not used to assess the accuracy of stress predictions.

Table 6.10. presents the results of the processing time evaluation.

Table 6.10: Processing performance of the deployed component

Measure	Result
Model loading at start-up	3.5 s
Processing of one 15-second frame, including beat detection, all beat updates and all predictions (median / 95th percentile / maximum)	270 ms / 380 ms / 446 ms
Computation of one prediction (median / 95th percentile)	66 ms / 84 ms
Update of the causal state for one beat (median)	0.04 ms
Predictions produced from 40 frames	148
Peak memory of the server process	About 850 MB

The performance measurements were carried out on a cloud-based Linux machine with two virtual processor cores (Intel Xeon, 2.8 GHz). The testing environment used Python 3.13 and TensorFlow 2.21, with all processing performed on the CPU without GPU acceleration.

The results showed that the component could process physiological data much faster than it was received. Even in the slowest recorded case, a frame containing 15 seconds of PPG data was processed in less than half a second. This provides sufficient processing capacity to keep up with the incoming data stream and demonstrates that the component satisfies the processing performance requirement NFR3 under the tested conditions.

Most of the server's memory usage was associated with the TensorFlow runtime rather than the physiological data processing operations.

6.2.3 Requirement Verification

Table 6.11. records how each requirement in Section 4.1 was verified.

Table 6.11: Verification of the requirements

ID	Evidence	Status
FR1	Frame contract tests; end-to-end run with 16 of 16 frames accepted (Section 6.4)	Met
FR2	Parity and artefact-rejection tests; signal-quality test	Met
FR3	Causality tests on the baseline engine	Met
FR4	Cold-start test; seeding comparison in Table 6.7.	Met
FR5	Window-boundary and feature-parity tests	Met
FR6	Model parity tests; operating point of 0.5923 macro-F1	Met
FR7	Point and band output tests; gating results in Table 6.8.	Met
FR8	Interface tests; end-to-end run with predictions received by a subscriber	Met
FR9	Health and model-availability tests	Met
NFR1	Corruption-based causality tests in the notebooks and in the software	Met
NFR2	First prediction after 60 beats; one prediction every 5 beats	Met
NFR3	Processing measurements in Table 6.10.	Met
NFR4	Architecture (Section 4.2)	Met
NFR5	Review of the code: no physiological data written to disk or logs	Met

NFR6	JSON schema tests; clients of other components parse the messages	Met
NFR7	Pinned dependencies, artefact fingerprints and configuration check	Met
NFR8	Operating rule in Section 5.5	Met

6.3 User or stakeholder validation

At the time of the draft submission, formal user validation had not yet been completed for this component. Most of the model development and evaluation were carried out using publicly available physiological datasets. Practical testing mainly focused on verifying that the stress inference component could communicate and operate correctly with the other parts of the complete system.

A small-scale validation study involving university students is planned before the final presentation. The main purpose of this study is to collect initial feedback on how users understand the predicted stress levels and how well the predictions are presented within the adaptive VR meditation environment. This evaluation will focus on usability and user experience rather than clinical accuracy. It is not intended to establish whether the system can diagnose stress or any medical condition.

During the planned study, participants will interact with the complete system, including the wearable sensing device, mobile application, backend stress inference component and VR environment. The backend will estimate stress using four levels: relaxed, mild, moderate and high. Each prediction will also include confidence information to indicate how certain the model is about its estimate.

Participants will be asked to provide feedback on how clearly the stress levels are presented, how quickly the system responds and whether the changes in the VR environment appear appropriate for the displayed stress information. The study will also examine whether users can easily understand the four stress categories and

whether presenting a two-level confidence band is more helpful than forcing a single prediction when the model is uncertain.

At the draft stage, stakeholder validation has mainly involved the developers responsible for the other system components. The mobile application was developed to send physiological measurements using the agreed data format, while the VR application and other connected clients were designed to receive and interpret the stress predictions. This provides initial evidence that the interfaces are suitable for integration with the complete system. However, successful technical integration does not establish how actual users will perceive or experience the system.

The results of the planned university-student study will be included in this section before the final presentation. These findings will be used to identify possible improvements in the presentation of stress information, system responsiveness and user interaction. They will be treated as preliminary usability feedback rather than evidence of clinical validity or medical effectiveness.

6.4 Integration results

The backend integration was first tested independently by running the server in its deployed configuration and connecting two test clients through WebSocket. The first client simulated the mobile application by sending PPG frames to the ingestion endpoint using the same data format as the actual application. The second client simulated the VR application by connecting to the prediction stream and receiving the generated stress estimates. During the test, 16 frames of artificially generated PPG data were transmitted, with each frame also containing a skin temperature measurement.

All 16 frames were successfully received and accepted by the backend. The first four frames provided the physiological data needed to form the initial 60-beat analysis window. Once this window was available, the backend began generating predictions as additional heartbeats were received. A total of 45 predictions were produced during the test. Among these, 30 predictions reported a single stress level, while the remaining 15 used a confidence band to represent uncertainty between two possible stress levels.

The time between sending a frame and receiving the last prediction generated from that frame was also measured. The median delay was 0.31 seconds, while the maximum recorded delay was 0.46 seconds. All prediction messages received by the subscribing client followed the output format defined in Table 4.10.

These results confirmed that the backend could receive physiological measurements, update its causal state, generate stress predictions and transmit the results to connected clients. The processing and communication time was also considerably shorter than the 15-second interval represented by each incoming PPG frame.

It is important to note that the PPG signal used in this test was artificially generated. Therefore, the predicted stress levels were not considered meaningful measurements of a person's physiological state. The purpose of this test was to verify the communication and processing behaviour of the backend, rather than to evaluate the accuracy of the stress classification models.

Following the independent backend test, a full-system integration test was conducted using the physical wearable device, mobile application, Python backend and Meta Quest 2 headset. During this test, the wearable collected physiological measurements and transmitted them to the mobile application. The mobile application successfully received the data and forwarded it to the backend through the network connection.

The backend then processed the incoming physiological measurements using the deployed inference pipeline and generated stress predictions. These predictions were successfully delivered to both the mobile application and the VR application running on the Meta Quest 2 headset. This confirmed that the complete communication process, from collecting physiological data to generating and delivering stress estimates, worked as intended.

After the initial analysis window was formed, predictions continued to reach both applications as new physiological data became available. No unresolved integration issues prevented the system from operating end to end during the test.

The total delay between collecting a physiological measurement at the wearable and displaying the corresponding prediction in the VR headset was not measured separately. However, the predictions were observed to appear promptly after being

generated by the backend. The earlier timing measurements also showed that backend processing and communication took considerably less time than collecting the physiological data required for an analysis window.

The full-system integration test verified the following communication flow:

Wearable Device → Mobile Application → Backend Stress Inference → Mobile Application and Meta Quest 2

Overall, the two integration tests provided evidence that the stress inference component could operate correctly in both environments. The WebSocket-level test verified the backend's communication interfaces, processing behaviour and prediction message structure under controlled conditions. The full-system test then confirmed that these functions worked when the actual wearable, mobile application and VR headset were connected.

Together, these results demonstrated that the component could function as an independent inference service and as part of the complete adaptive VR meditation system.

6.5 Summary of findings

Table 6.12. summarises the findings against the research questions set out in Section 1.6.

Table 6.12: Summary of findings against the research questions

Question	Finding
RQ1	Four-level stress can be inferred from RR intervals with useful accuracy: 0.646 macro-F1 offline and 0.5923 under real-time constraints, with most errors between adjacent levels. Performance varies widely between individuals.
RQ2	Cosinor features improve performance, but not by correcting a circadian rhythm. The gain comes from the residual, which normalises each subject against their own within-session level; it is insensitive to the

	assumed period and replicates on a second dataset and with protocol-defined labels.
RQ3	Sequence models add about 0.02 to 0.03 macro-F1 over gradient-boosted trees, but the bidirectional and causal networks, with or without personalisation, cannot be separated. The causal MS-CGCA ensemble was selected for its causal safety.
RQ4	Midpoint labelling inflates macro-F1 by about 0.08 across all configurations. A 60-beat window halves the delay to about 45 seconds without loss of accuracy.
RQ5	A causal baseline is worth +0.068 macro-F1; a population seed is sufficient for a new user and outperforms the user's own session mean; personalisation and forecasting bring no measurable benefit; confidence gating halves severe errors at about 80% coverage.
RQ6	The two-way ensemble of gradient-boosted trees and MS-CGCA, with causal baselines seeded from the population mean, 60-beat endpoint-labelled windows and a confidence-gated band, provides the best balance of accuracy, safety and real-time operation.

7 DISCUSSION

7.1 Interpretation of findings

The main finding of this research is that the improvement previously attributed to circadian detrending in short physiological recordings is mainly caused by baseline normalisation. In other words, the models benefit from comparing a person's current physiological measurements with their own baseline, rather than from identifying and correcting a 24-hour circadian rhythm.

Several experimental findings support this conclusion. First, the available recordings were too short to reliably identify a 24-hour rhythm, and the fitted rhythm parameters produced values that were not physiologically meaningful. Second, most of the improvement came from the residual features rather than the estimated baseline itself. Adding baseline-level information to a model that already contained residual features did not provide any further benefit.

The improvement also remained similar when different rhythm periods were used, provided that the periods were long enough to represent a smooth trend. Furthermore, subjects whose physiological recordings fitted a circadian rhythm more closely did not achieve greater improvements in classification performance. The same general pattern was observed in a second dataset collected using a different sensor and participant population. It also remained consistent when stress labels were defined independently of the physiological features used by the models.

Together, these findings suggest that the temporal features mainly help the models understand how much the current heartbeat measurements differ from a person's usual physiological level. This is particularly useful for stress detection because absolute HRV values can vary considerably between individuals. For example, an RMSSD value that is normal for one person may indicate a change in physiological state for another. By expressing the current measurements relative to the individual's baseline, the model can reduce some of these differences between subjects and identify stress-related changes more effectively.

This finding does not necessarily contradict earlier research on circadian detrending. Instead, it provides a clearer explanation of why similar processing methods can improve performance when applied to short recording sessions.

The deployment experiments revealed another important finding. The largest factors affecting the reliability of offline evaluation results were not necessarily related to the model architecture. The choice of labelling convention had a particularly strong influence, changing macro-F1 by approximately 0.08. This difference was larger than the improvements obtained from any of the architectural changes examined in the research.

More importantly, changing the labelling convention affected the conclusions drawn from the experiments. Under midpoint labelling, personalisation appeared to add +0.031 macro-F1, while donor-based baseline initialisation appeared significantly worse ($p = 0.0026$). Under endpoint labelling, neither difference could be detected.

This shows that relying only on midpoint-labelled results could lead to unnecessary design complexity. A more complicated model might appear beneficial during offline evaluation without providing the same advantage during real-time operation.

Several other approaches that initially appeared promising also failed to produce measurable improvements. hese included the specially designed causal neural network and personalised model training, neither of which produced a detectable improvement in classification accuracy, and short-term stress forecasting, which performed worse than a simple persistence baseline at every horizon tested.

An unexpected result was found in the baseline initialisation experiments. Using the population mean to initialise the causal baseline trackers performed better than using the subject's own session mean. This difference was significant under endpoint labelling ($p = 0.0054$) and may be related to how the slow baseline tracker retains information about differences between an individual's physiological level and the population average.

The confidence-gating experiments also provided a practical finding. The confidence gate does not directly improve the underlying model's ability to classify stress. Instead, it changes how uncertain predictions are presented to the receiving

application. When two neighbouring stress levels have similar probabilities, the component reports a band covering both levels rather than forcing a single prediction.

This is useful for an adaptive VR meditation environment because changes to the virtual environment can be made gradually. Providing a range of possible stress levels may therefore be more appropriate than presenting one uncertain estimate as a definite result.

7.2 Comparison with literature baselines and existing solutions

Directly comparing the classification accuracy reported in this research with results from other studies is difficult. Stress detection studies often use different datasets, stress categories, labelling methods, window lengths and evaluation procedures. These differences can strongly influence the reported performance, as also discussed in the existing literature [1]. For this reason, the following comparison focuses mainly on the methods used and the evaluation approaches rather than comparing accuracy scores alone.

The original WESAD study reported classification accuracies of up to 93% for binary stress detection using conventional physiological features and machine learning classifiers [2]. In this research, the model achieved a macro-F1 score of 0.955 when using binary stress labels defined by the experimental protocol, as presented in Table 6.4. This supports the observation that stress and non-stress conditions in WESAD can be separated relatively well.

However, the main task addressed in this thesis is more difficult because it involves classifying stress into four levels. These levels can have overlapping physiological characteristics, especially when neighbouring intensity categories are considered. Therefore, the results from the four-level classification task cannot be directly compared with those from binary stress detection.

Previous studies on multi-class HRV-based stress detection, including the work of Mortensen et al. [3] and Hemakom et al. [7], have shown that identifying more than

two stress levels is possible. However, these studies differ in how their stress categories are defined and how their models are evaluated.

Shen et al. [5] reported that removing circadian trends from HRV measurements improved stress classification across different times of day. Their recordings included measurements taken in the morning, at noon and in the evening, allowing the analysis to cover different parts of the daily cycle. Therefore, the findings of the present research do not contradict the results reported by Shen et al.

Instead, this research identifies an important limitation when applying similar methods to short recordings. In datasets such as WESAD, where each session lasts only around 1.5 hours, it is not possible to reliably identify a complete 24-hour circadian rhythm. The improvement obtained from temporal detrending is therefore better explained by normalising physiological measurements against the individual's baseline within the session.

Using a person's own physiological baseline is already an established approach in wearable stress detection [19], [20]. The contribution of this research is to show that, for short recordings, processing methods described as circadian detrending can produce their benefits mainly through baseline normalisation rather than actual circadian correction.

The bidirectional neural network used as a baseline in this research was based on the attention-based hybrid architecture proposed by Tanwar et al. [17]. Similar convolutional and recurrent neural network approaches are widely used in stress detection research [6], [11].

The experiments showed that the causal MS-CGCA network achieved performance comparable to the bidirectional network but did not provide a statistically significant improvement in accuracy. Its main advantage was therefore related to real-time operation. The causal architecture was selected because it provides stronger protection against using future physiological measurements when generating predictions.

Generalising HRV-based stress detection models across different datasets is also a known challenge [8]. The present research observed this difficulty, as the absolute

classification scores differed between WESAD and the independent wrist-PPG dataset. However, the underlying finding about the contribution of residual features remained consistent across both datasets. This suggests that the baseline-normalisation mechanism is not limited to one sensor type or participant population, although its effect on classification accuracy may vary.

Other researchers have also considered the practical requirements of continuous stress detection. Can et al. [26] developed a wearable-based stress detection approach for daily-life monitoring, while Stržinar et al. [27] considered detection delay alongside classification accuracy.

The present component shares this focus on practical operation but examines an additional issue: how the requirements of real-time inference affect the performance reported during offline evaluation. The experiments showed that the labelling convention had a greater influence on macro-F1 than the architectural changes tested, highlighting the importance of evaluating models under conditions that reflect their intended deployment.

Modern consumer wearable devices can also measure HRV and provide information about physiological activity [12]. However, such devices commonly present summary measurements rather than a four-level stress estimate with explicit uncertainty information that can be used directly by another application. The component developed in this research addresses this need by providing structured stress predictions for an adaptive VR meditation environment.

7.3 Research contribution

This thesis makes four main research contributions.

The first contribution is a clearer explanation of the performance improvement associated with circadian detrending in short HRV recordings. The experiments showed that the improvement was mainly related to normalising physiological measurements against an individual's baseline within the session, rather than correcting an identifiable 24-hour rhythm.

This conclusion was supported by several experiments, including the temporal-basis audit, feature decomposition, comparison of different rhythm periods, subject-level correlation analysis and evaluation on a second dataset. An additional experiment using labels defined independently of the physiological features provided further evidence that the finding was not caused by the original label construction method.

The second contribution is a detailed evaluation of how real-time deployment requirements affect multi-level stress classification performance. In particular, the research measured the effect of changing from midpoint labelling to endpoint labelling. This change reduced macro-F1 by approximately 0.08 across four model configurations, making it the largest individual factor identified in the deployment experiments.

The third contribution is the finding that the labelling convention can change the practical conclusions drawn from model comparisons. Under midpoint labelling, personalisation appeared beneficial and donor-based baseline initialisation appeared significantly worse. These conclusions were not supported under endpoint labelling. This demonstrates why the labelling convention should be clearly reported when evaluating machine learning models using windows of physiological data.

The fourth contribution is the reporting of experimental findings that did not support the original expectations. The purpose-built causal neural network did not achieve a measurable improvement in classification accuracy over the bidirectional baseline, although it offered advantages for real-time operation. Personalisation produced no detectable improvement, and short-term forecasting performed worse than a persistence baseline. In contrast, population-mean baseline initialisation performed better than initialisation using the subject's own session mean.

Reporting these findings provides a more complete understanding of which design choices improved performance and which added complexity without measurable benefits.

7.4 Product contribution

The main product contribution of this research is a working stress inference service developed for the adaptive VR meditation system. The component receives raw PPG signals and skin temperature measurements from the mobile application, processes the physiological data and generates estimates of the user's stress level.

The backend detects heartbeats from the PPG signal, calculates RR intervals and maintains causal baseline trackers that update as new measurements arrive. These trackers are initialised using the population mean, allowing a new user to begin a session without completing a separate calibration procedure.

The component produces stress estimates using four levels: relaxed, mild, moderate and high. Each prediction includes confidence information. When the model cannot clearly distinguish between two neighbouring stress levels, the component reports a two-level band instead of forcing a single classification. This provides the receiving applications with information about uncertainty and allows them to respond more appropriately.

The implementation was verified using an automated test suite containing 75 tests. These tests check consistency between the deployed inference pipeline and the research implementation, enforce causal processing and verify the behaviour of the backend interfaces. The processing performance evaluation also showed that the component could handle each 15-second physiological data frame in less than half a second under the tested conditions, providing sufficient capacity for continuous inference.

The backend uses a documented communication interface that was adopted by the VR application, mobile application and voice-based component developed by other group members. This allows the same stress predictions to be used by different parts of the complete system.

Beyond the working software, the component also demonstrates several useful engineering practices for deploying machine learning systems. These include sharing the same processing functions between training and deployment, storing the exact

model artefacts used during evaluation and verifying their integrity using SHA-256 fingerprints.

Another important design decision was to provide uncertainty information alongside stress predictions rather than always reporting a single level. Together, these practices help maintain consistency between the research results and deployed behaviour while supporting the reliable integration of the stress inference component into the complete adaptive VR meditation system.

7.5 Limitations and threats to validity

7.5.1 Statistical Conclusion Validity

The main dataset used in this research contains only fifteen subjects, which limits the statistical power of the paired comparisons. With such a small sample, some differences between model configurations may not reach statistical significance, even when they are meaningful in practice.

For this reason, a non-significant result was not considered proof that two configurations performed equally well. Demonstrating statistical equivalence would require a separate procedure, such as two one-sided tests using an equivalence margin defined before the analysis. Since this procedure was not performed, the thesis does not claim that any of the compared configurations are equivalent.

Another limitation is the variation in model performance between subjects. The standard deviations ranged from 0.135 to 0.162, which were relatively large compared with many of the performance differences observed between configurations. This makes it difficult to determine whether small improvements represent meaningful changes or variation between individuals.

The cold-start seeding, confidence-gating and forecasting experiments were also conducted using only one random seed. The main model comparisons showed seed-to-seed variation of approximately 0.012, as reported in Table 6.6. Therefore, performance differences smaller than this variation were not considered strong evidence of improvement.

7.5.2 Internal Validity

One important limitation concerns the overlap between consecutive analysis windows. Since neighbouring windows contain much of the same physiological data, they cannot be treated as fully independent observations. Although the experiments produced many windows, the effective number of independent observations was much closer to the number of subjects.

To address this, statistical testing was performed using subject-level results rather than treating every window as a separate observation. The temporal feature decomposition was also repeated with lower levels of window overlap. As overlap decreased, the improvement contributed by residual features fell from 0.096 to 0.061 in macro-F1. However, residual features continued to provide a greater contribution than the baseline-level features.

Another concern is how the four stress intensity levels were defined. The intensity categories within the stress condition were derived from a short-term physiological variability statistic related to the model's input features. This creates a possible relationship between the labels and the information used to predict them.

To investigate this issue, the feature decomposition was repeated using binary stress labels taken directly from the experimental protocol. The results showed that residual features remained more useful than the fitted baseline, suggesting that the main mechanism finding was not dependent on the original intensity-labelling method. However, the absolute accuracy of the four-level classification task cannot be validated against an independent external standard.

The replication dataset also had some limitations related to recording organisation and participant identification. One recording was excluded without a documented reason. In addition, the two files of one split recording (f14) were treated as separate subjects, so that in two cross-validation folds part of that participant's data was present in training while the other part was tested. This affects one of 33 participants and only the cross-dataset replication, not any WESAD result, but it constitutes subject-level leakage, and the replication figures in Table 6.3 should be read with this in mind. However, the replication experiment was used only to examine whether the

main temporal feature findings were consistent across datasets, rather than to establish the performance of the deployed model.

7.5.3 Construct Validity

The four stress levels used in this research are experimental categories based on the laboratory protocol and a physiological variability measure. They have not been clinically validated and should not be interpreted as established medical classifications of stress.

The predicted levels were also not compared with participants' own reports of how stressed they felt. Therefore, a model achieving high accuracy against these experimental labels does not necessarily mean that it can accurately measure a person's actual experience of stress.

This distinction is important because physiological changes do not always correspond directly to perceived psychological stress. The component should therefore be understood as a research-based physiological stress estimation system rather than a tool for assessing clinical or self-reported stress.

7.5.4 External Validity

The findings of this research were obtained from datasets collected from healthy adults under controlled laboratory conditions. The physiological measurements came from chest-based ECG sensors and wrist-worn research devices. These conditions may not fully represent how the system would perform when used by people with different physiological characteristics or in everyday environments.

As a result, model performance on other populations, including individuals with different health conditions, fitness levels and physiological baselines, remains uncertain. The performance of the deployed models on the wearable device developed for the complete system has also not been formally established.

Another limitation is the difference between sensor types. A model trained using HRV measurements derived from chest ECG signals may not perform in the same way when receiving data from wrist-based PPG sensors. Although the Hongn et al. dataset was used to investigate whether the main mechanism finding could be reproduced using wrist-based measurements, it was not used to train the final

deployed models. Therefore, the replication results do not establish the accuracy of the deployed system on wrist PPG.

The wearable used in the complete system also differs from the hardware originally proposed. In particular, skin temperature is measured using a non-contact infrared sensor rather than the contact-based sensor specified during the initial design. These sensors measure temperature differently, which may introduce differences between the temperature measurements used during model training and those received during deployment.

Finally, a formal user study had not been completed at the time of the draft submission. Although the component was tested for software integration and communication with the other system components, its effect on users' actual stress levels has not been measured. Therefore, the research cannot yet establish whether adapting the VR meditation environment based on the predicted stress levels improves the user's stress experience.

7.5.5 Technical Limitations

The current backend implementation has several limitations that must be considered before wider deployment. One of the main concerns is communication security. The backend currently accepts unencrypted WebSocket connections and does not require client authentication. This means that physiological measurements and stress predictions could be exposed if the system were used on an untrusted network.

For this reason, the component is currently intended to operate only within a trusted local network. Wider deployment would require encrypted communication, authentication and appropriate restrictions on client access.

The backend also runs as a single server process, with a separate inference session maintained for each ingestion connection. Although this design keeps the physiological state of different sessions separate, the system has not been evaluated under conditions involving several simultaneous users. Its performance and resource requirements under higher workloads therefore remain unknown.

Another limitation concerns dataset licensing. The deployed models were trained using WESAD, which is available for scientific, non-commercial research use. As a

result, the current models cannot be used commercially without obtaining the necessary permission from the dataset owners. Alternatively, a commercial version would require retraining using data whose licensing terms permit commercial use.

These limitations do not prevent the component from operating within the current research environment. However, they identify areas that would need further development and evaluation before the system could be used more widely.

7.6 Implications

The findings of this research suggest three important practices for researchers working on wearable stress detection. First, studies using windowed physiological data should clearly explain how labels are assigned within each window. The experiments showed that this decision can affect not only the reported performance scores but also the conclusions drawn from model comparisons.

Second, when temporal detrending is applied to short physiological recordings, the resulting improvement should be interpreted as baseline normalisation unless the available data are sufficient to identify a circadian rhythm across different times of day. Third, when model configurations show no statistically significant differences, the final selection should consider practical factors such as causal safety, implementation simplicity and computational requirements. Small performance differences that fall within the variation across training seeds should not be the main reason for selecting one model over another.

For developers building real-time physiological monitoring systems, these findings show that the difference between offline and deployment performance can be measured through controlled experiments. The results also highlight that this difference is not always caused by the machine learning model itself. Evaluation choices, particularly the labelling convention and the use of future information, can have a greater influence on the reported accuracy.

The research also demonstrates the practical value of using causal baseline trackers initialised from the population mean. This allows the system to begin estimating stress once enough physiological data have been collected, without requiring a

separate calibration session for each new user. Another useful feature is the confidence-based output mechanism. Instead of always reporting a single stress level, the component can provide a band covering two possible levels when the prediction is uncertain. The experiments showed that this approach reduced the severe-error rate by approximately half at around 80% coverage, although fewer windows received single-level predictions.

For the complete VR meditation system, the developed component provides stress estimates that can be generated in real time without relying on future physiological measurements. It also communicates uncertainty through confidence values and two-level bands. This allows the VR environment to respond to changes in the user's estimated stress level while making more gradual adjustments when the prediction is uncertain.

However, several improvements are still required before the system can be used beyond the current research environment. Further validation involving actual users is needed to understand how well the system performs in practical situations and how users respond to the adaptive VR experience. The backend also requires stronger security measures, including encrypted communication and client authentication, before deployment on untrusted networks. Finally, any wider or commercial use must comply with the licensing conditions of the datasets used to train the models.

These considerations provide a direction for future development while showing how the findings of this research can support both wearable stress detection studies and the design of real-time adaptive systems.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

This research aimed to develop a multi-level stress inference component for an adaptive VR meditation system and investigate whether the performance achieved by HRV-based stress detection models during offline evaluation could be maintained under real-time conditions.

The research initially focused on the idea that removing circadian trends from HRV measurements could improve stress classification, as suggested by previous studies. The experiments confirmed that this type of processing improved model performance. However, they also revealed that the reason for the improvement was different from what was originally expected. Since the available recordings lasted approximately 1.5 hours, they were too short to reliably identify a 24-hour circadian rhythm. Instead, the improvement came mainly from normalising each person's physiological measurements against their own baseline within the recording session.

The next stage of the research examined how model performance changed when moving from offline evaluation to real-time deployment. The most significant factor was the choice of labelling convention. Midpoint labelling increased macro-F1 by approximately 0.08 compared with endpoint labelling, creating an overly optimistic estimate of performance. It also influenced the conclusions drawn about personalisation and baseline initialisation.

Other experiments provided useful findings for the final system design. Reducing the analysis window length by half improved responsiveness without reducing classification accuracy. Initialising the causal baseline using the population mean also allowed new users to begin a session without requiring a separate calibration period. In contrast, personalisation and the specially designed causal neural network produced no detectable improvement in prediction accuracy, and short-term forecasting performed worse than a persistence baseline. However, the causal architecture was retained because of its suitability for real-time operation.

The final component runs on a separate backend computer and receives raw PPG signals from the mobile application. It processes incoming physiological measurements, maintains causal baseline trackers and generates stress predictions after every five new heartbeats once the initial window is available. The output consists of four possible stress levels, along with confidence information. When the model is uncertain, it reports a band covering two possible levels instead of forcing a single prediction.

Under leave-one-subject-out evaluation with all real-time constraints applied, the deployed configuration achieved a macro-F1 score of 0.5923 and a quadratic weighted kappa (κ) of 0.7525. The implementation was also evaluated using an automated test suite containing 75 tests, which checked consistency with the research pipeline, causal processing and correct system behaviour. Performance testing showed that each 15-second PPG frame could be processed in less than half a second under the tested conditions, demonstrating that the backend could support continuous stress inference.

Overall, the research provides both a clearer understanding of why temporal baseline features improve HRV-based stress classification and a working component designed for real-time integration with an adaptive VR meditation system. Although the technical implementation and model performance have been evaluated, the component's effectiveness from the user's perspective has not yet been established. Conducting a validation study with participants is therefore an important next step to understand how users experience the stress predictions and whether the adaptive VR environment provides meaningful benefits.

8.2 Achievement of objectives

Table 8.1. sets out how each objective stated in Section 1.5 was addressed.

Table 8.1: Achievement of the research objectives

No.	Objective	Outcome	Status
1	Extract and analyse HRV information from RR intervals	Thirteen HRV features and five residual features computed from	Achieved

	for multi-level stress inference	RR intervals; four-level inference achieved (Sections 4.4, 6.1)	
2	Explore Cosinor-based temporal features	Cosinor features applied to HRV and skin temperature; +0.091 macro-F1 over the baseline (Table 6.1.)	Achieved
3	Determine whether the improvement is due to circadian correction	Shown to be within-session baseline normalisation by five converging analyses (Section 6.1)	Achieved
4	Design and compare machine learning and deep learning strategies	Trees, bidirectional and causal networks, two- and three-way ensembles compared over five seeds (Table 6.6.)	Achieved
5	Evaluate causal against non-causal processing	Causal baselines evaluated against the offline fit and against no baseline (Section 4.3, Table 6.7.)	Achieved
6	Study window length and labelling position	Labelling inflation measured in four configurations; 60 and 120-beat windows compared (Table 6.5.)	Achieved
7	Test the system under cold start, short history, confidence-aware output and short-term prediction	Cold start, personalisation, gating and forecasting evaluated (Tables 6.7–6.9)	Achieved
8	Select the method with the best trade-off for deployment	Two-way MS-CGCA ensemble selected under a stated decision rule (Section 4.3)	Achieved

9	Integrate the method into the Python backend	Backend implemented, tested with 75 automated tests and run end to end (Chapter 5, Section 6.4)	Achieved
10	Generate real-time predictions with a confidence level for the VR application	Four-level predictions with confidence and band published on /stream (Section 4.4)	Achieved

The approved project proposal set a target macro-F1 score of 0.85 for the four-level stress classification task. However, this target was not achieved. The offline research configuration obtained a macro-F1 of 0.646, while the final deployed configuration achieved 0.5923.

The target was originally established before the experiments revealed how certain offline evaluation methods could produce overly optimistic results. In particular, midpoint labelling and the use of information from complete recording sessions could make model performance appear higher than what would be possible during real-time operation. The labelling convention alone was found to increase macro-F1 by approximately 0.08.

These findings showed that the original performance target did not fully account for the limitations of real-time stress inference. Although the target was not reached, the deployed model was evaluated under conditions that more closely reflect actual operation, including endpoint labelling and causal processing. Therefore, the achieved macro-F1 of 0.5923 provides a more realistic estimate of the component's expected performance on previously unseen users than the higher offline score.

This also highlights the importance of defining performance targets based on the intended deployment conditions rather than relying only on results obtained from offline experiments.

8.3 Individual research and product contributions

The individual contributions of the author to the group project are described below.

8.3.1 Research Contributions

The author was responsible for designing and conducting the temporal analysis experiments. These included the temporal-basis audit, additive feature decomposition with the session-level control, timescale analysis, per-subject evaluation, cross-dataset replication and binary-label sensitivity testing.

The author also designed and carried out the experiments examining the effects of real-time deployment requirements on model performance. These covered the labelling convention, window length, model comparisons across five training seeds, personalisation, cold-start baseline initialisation, confidence-gated predictions and short-term stress forecasting.

In addition, the author designed and developed the Multi-Scale Circadian-Guided Cross-Attention (MS-CGCA) neural network and the causal multi-timescale baseline tracking mechanism. The author also prepared two conference papers based on the research findings.

8.3.2 Product Development Contributions

The author developed the stress inference package and Python backend server. This included implementing PPG heartbeat detection, causal feature extraction, the streaming inference engine, ensemble prediction and the confidence-gating mechanism. The author also implemented the HTTP and WebSocket interfaces used to communicate with the other system components.

The author was responsible for defining the data formats and communication interfaces used by the connected applications. An automated test suite was also developed to verify consistency between the deployed system and the research pipeline, including checks for causal processing and correct inference behaviour. The author additionally exported, verified using SHA-256 fingerprints and documented the trained model artefacts used in deployment.

8.3.3 Collaborative Contributions

The wearable sensing device was developed through the combined contributions of all group members, including the author. The mobile application, VR meditation application and voice-based component were developed by the other group members as their respective responsibilities.

The author also contributed to the integration of the stress inference backend with the other components to support physiological data transmission and the delivery of stress predictions within the complete adaptive VR meditation system.

8.4 Future directions

The most important next step is to conduct a validation study involving actual users. In this study, participants would use the complete system while wearing the sensor developed for the project and reporting their perceived stress levels. This would help determine whether the four predicted stress categories correspond to how users actually experience stress. It would also provide physiological data collected directly from the deployed wearable device, which could be used to further train and evaluate the models. In addition, the study could examine whether adapting the VR meditation environment based on stress predictions leads to measurable reductions in users' stress levels. Such research would require ethical approval and informed consent from participants. Depending on the permissions obtained, the collected data could also support future model development and wider use without relying entirely on datasets with restrictive licensing conditions.

Another important area for future research is investigating the effect of circadian rhythms using recordings collected across different times of day. The current research showed that recordings lasting approximately 1.5 hours were too short to reliably identify a 24-hour rhythm. Collecting physiological measurements throughout a full day or over several days would allow researchers to examine whether genuine circadian patterns contribute to stress classification when sufficient data is available. This could help determine whether the improvement continues to come mainly from baseline normalisation or whether circadian information provides an additional benefit. A related extension would be to develop a neural network

capable of learning an individual's baseline across multiple timescales directly from the physiological data, rather than relying on baseline values calculated during preprocessing.

Several statistical and experimental questions also require further investigation. Although some model configurations produced similar results, the current experiments did not establish that they were statistically equivalent. Future research could use two one-sided tests with a predefined equivalence margin to examine this more formally. The cold-start seeding, confidence-gating and forecasting experiments should also be repeated using multiple training seeds to determine whether their findings remain consistent across different runs.

The cross-dataset replication could be strengthened by including the previously excluded recording and treating the two files belonging to the same participant as a single subject during evaluation. This would improve the reliability of the subject-level comparisons. Another useful experiment would be to investigate why population-mean seeding performed better than initialisation using the subject's own session mean. One possible approach is to reduce the half-life of the slow baseline tracker or remove its baseline-level feature from the model input. This would help determine whether the tracker's long-term memory is responsible for the observed difference. In addition, analysis windows shorter than 60 beats should be evaluated to examine whether prediction delay can be reduced further without affecting classification performance.

The backend also requires several improvements before it can be deployed outside the current trusted local network. Communication should be secured using encrypted WebSocket connections (wss://) with TLS, and client authentication should be introduced to prevent unauthorised access. Cross-origin connections should be limited to trusted clients. The server should also be extended and tested to support multiple simultaneous users while keeping their physiological data and inference states separate. Another hardware-related improvement would be to replace the current non-contact infrared temperature sensor with a contact-based skin temperature sensor that more closely matches the type of measurements available in

the training datasets. This could help reduce differences between the physiological inputs used during training and those collected during deployment.

Finally, the prediction output could be improved further. The ordinal classification head investigated during the experiments reduced severe classification errors, although it did not produce a statistically significant improvement in macro-F1. Integrating this approach into the deployed MS-CGCA network could therefore be useful, particularly because the four stress levels follow a natural order. The confidence-gating mechanism could also be refined by selecting its threshold based on the actual needs of the VR meditation environment. This would allow the system to balance the number of single-level predictions against the risk of severe errors, ensuring that the prediction output supports appropriate and gradual changes to the virtual environment.

Together, these improvements would help strengthen the scientific validity, practical reliability and usability of the stress inference component. They would also provide a clearer understanding of how physiological stress estimates can be used effectively in real-time adaptive VR meditation systems.

REFERENCES

[1] G. Vos, K. Trinh, Z. Sarnyai, and M. Rahimi Azghadi, “Generalizable machine learning for stress monitoring from wearable devices: A systematic literature review,” *International Journal of Medical Informatics*, vol. 173, p. 105026, May 2023, doi: 10.1016/j.ijmedinf.2023.105026.

[2] P. Schmidt, A. Reiss, R. Duerichen, C. Marberger, and K. Van Laerhoven, “Introducing WESAD, a Multimodal Dataset for Wearable Stress and Affect Detection,” in *Proceedings of the 20th ACM International Conference on Multimodal Interaction*, Boulder CO USA: ACM, Oct. 2018, pp. 400–408. doi: 10.1145/3242969.3242985.

[3] J. A. Mortensen, M. E. Mollov, A. Chatterjee, D. Ghose, and F. Y. Li, “Multi-Class Stress Detection Through Heart Rate Variability: A Deep Neural Network Based Study,” *IEEE Access*, vol. 11, pp. 57470–57480, 2023, doi: 10.1109/ACCESS.2023.3274478.

[4] G. Cornelissen, “Cosinor-based rhythmometry,” *Theor Biol Med Model*, vol. 11, no. 1, p. 16, Dec. 2014, doi: 10.1186/1742-4682-11-16.

[5] Y. Shen, Z. Fang, T. Zhang, F. Yu, Y. Xu, and L. Yang, “Heart rate variability with circadian rhythm removed achieved high accuracy for stress assessment across all times throughout the day,” *Front. Physiol.*, vol. 16, p. 1535331, Apr. 2025, doi: 10.3389/fphys.2025.1535331.

[6] M. Kyrou, I. Kompatsiaris, and P. C. Petrantonakis, “Deep Learning Approaches for Stress Detection: A Survey,” *IEEE Trans. Affective Comput.*, vol. 16, no. 2, pp. 499–517, Apr. 2025, doi: 10.1109/TAFFC.2024.3455371.

[7] A. Hemakom, D. Atiwiwat, and P. Israsena, “ECG and EEG based detection and multilevel classification of stress using machine learning for specified genders: A preliminary study,” *PLoS ONE*, vol. 18, no. 9, p. e0291070, Sep. 2023, doi: 10.1371/journal.pone.0291070.

[8] M. Benchekroun, P. E. Velmovitsky, D. Istrate, V. Zalc, P. P. Morita, and D. Lenne, “Cross Dataset Analysis for Generalizability of HRV-Based Stress Detection Models,” *Sensors*, vol. 23, no. 4, p. 1807, Feb. 2023, doi: 10.3390/s23041807.

[9] Task Force of the European Society of Cardiology and the North American Society of Pacing and Electrophysiology, “Heart Rate Variability: Standards of Measurement, Physiological Interpretation, and Clinical Use,” *Circulation*, vol. 93, no. 5, pp. 1043–1065, Mar. 1996, doi: 10.1161/01.CIR.93.5.1043.

[10] J.-Z. Xiang, Q.-Y. Wang, Z.-B. Fang, J. A. Esquivel, and Z.-X. Su, “A multi-modal deep learning approach for stress detection using physiological signals: integrating time and frequency domain features,” *Front. Physiol.*, vol. 16, p. 1584299, Apr. 2025, doi: 10.3389/fphys.2025.1584299.

[11] S. Quoy, D. Istrate, M. Benchekroun, and V. Zalc, “HRV Based Stress Detection Using Convolutional Neural Networks (CNNs),” in *2024 International Conference on Content-Based Multimedia Indexing (CBMI)*, Reykjavik, Iceland: IEEE, Sep. 2024, pp. 1–5. doi: 10.1109/CBMI62980.2024.10859222.

[12] R. Hernandez *et al.*, “Resting Heart Rate Variability Measured by Consumer Wearables and Its Associations with Diverse Health Domains in Five Longitudinal Studies,” *Sensors*, vol. 25, no. 23, p. 7147, Nov. 2025, doi: 10.3390/s25237147.

[13] P.-J. Chen, W.-S. Chien, H.-Y. Chen, and C.-C. Lee, “Stress Detection Using HRV Features Augmentation Based on Heart Rate Signal Transformation,” in *2024 IEEE 20th International Conference on Body Sensor Networks (BSN)*, Chicago, IL, USA: IEEE, Oct. 2024, pp. 1–4. doi: 10.1109/BSN63547.2024.10780687.

[14] C. Kirschbaum, K.-M. Pirke, and D. H. Hellhammer, “The ‘Trier Social Stress Test’ – A Tool for Investigating Psychobiological Stress Responses in a Laboratory Setting,” *Neuropsychobiology*, pp. 76–81, Feb. 2008, doi: 10.1159/000119004.

[15] A. Hongn, F. Bosch, L. E. Prado, J. M. Ferrández, and M. P. Bonomini, “Wearable Physiological Signals under Acute Stress and Exercise Conditions,” *Sci Data*, vol. 12, no. 1, p. 520, Mar. 2025, doi: 10.1038/s41597-025-04845-9.

[16] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco California USA: ACM, Aug. 2016, pp. 785–794. doi: 10.1145/2939672.2939785.

[17] R. Tanwar, O. C. Phukan, G. Singh, P. K. Pal, and S. Tiwari, “Attention based hybrid deep learning model for wearable based stress recognition,” *Engineering Applications of Artificial Intelligence*, vol. 127, p. 107391, Jan. 2024, doi: 10.1016/j.engappai.2023.107391.

[18] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, “Focal Loss for Dense Object Detection,” in *2017 IEEE International Conference on Computer Vision (ICCV)*, Venice: IEEE, Oct. 2017, pp. 2999–3007. doi: 10.1109/ICCV.2017.324.

[19] M. Bahameish, T. Stockman, and J. Requena Carrión, “Strategies for Reliable Stress Recognition: A Machine Learning Approach Using Heart Rate Variability Features,” *Sensors*, vol. 24, no. 10, p. 3210, May 2024, doi: 10.3390/s24103210.

[20] E. Smets *et al.*, “Large-scale wearable data reveal digital phenotypes for daily-life stress detection,” *npj Digital Med*, vol. 1, no. 1, p. 67, Dec. 2018, doi: 10.1038/s41746-018-0074-9.

[21] S. Koldijk, M. Sappelli, S. Verberne, M. A. Neerincx, and W. Kraaij, “The SWELL Knowledge Work Dataset for Stress and User Modeling Research,” in *Proceedings of the 16th International Conference on Multimodal Interaction*, Istanbul Turkey: ACM, Nov. 2014, pp. 291–298. doi: 10.1145/2663204.2663257.

[22] K. Mundnich *et al.*, “TILES-2018, a longitudinal physiologic and behavioral data set of hospital workers,” *Sci Data*, vol. 7, no. 1, p. 354, Oct. 2020, doi: 10.1038/s41597-020-00655-3.

[23] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC Editor, RFC6455, Dec. 2011. doi: 10.17487/rfc6455.

[24] D. Makowski *et al.*, “NeuroKit2: A Python toolbox for neurophysiological signal processing,” *Behav Res*, vol. 53, no. 4, pp. 1689–1696, Aug. 2021, doi: 10.3758/s13428-020-01516-y.

[25] X. Shi, W. Cao, and S. Raschka, “Deep neural networks for rank-consistent ordinal regression based on conditional probabilities,” *Pattern Anal Applic*, vol. 26, no. 3, pp. 941–955, Aug. 2023, doi: 10.1007/s10044-023-01181-9.

[26] Y. S. Can, N. Chalabianloo, D. Ekiz, and C. Ersoy, “Continuous Stress Detection Using Wearable Sensors in Real Life: Algorithmic Programming Contest Case Study,” *Sensors*, vol. 19, no. 8, p. 1849, Apr. 2019, doi: 10.3390/s19081849.

[27] Ž. Stržinar, A. Sanchis, A. Ledezma, O. Sipele, B. Pregelj, and I. Škrjanc, “Stress Detection Using Frequency Spectrum Analysis of Wrist-Measured Electrodermal Activity,” *Sensors*, vol. 23, no. 2, p. 963, Jan. 2023, doi: 10.3390/s23020963.

GLOSSARY

Baseline normalisation - Expressing a physiological measurement relative to the same person's own recent level rather than in absolute terms.

Causal - A computation whose output at a given time depends only on data received up to that time.

Cold start - Operation for a new user for whom no previous data are available.

Confidence-gated band - An output that reports two adjacent stress levels instead of one when the model's two most probable levels are close.

Endpoint labelling - Assigning to a window the label found at its final beat.

Midpoint labelling - Assigning to a window the label found at its centre, so that half of the window postdates the label.

Nested estimate - A performance estimate in which every choice made from data, such as an ensemble weight, is made without the fold being scored.

Residual - The difference between an observed value and the baseline expected for it.

APPENDICES

Appendix A Research instruments or extended results

This appendix records the environment and procedure used to obtain the software verification and performance results reported in Sections 5.4, 6.2, and 6.4 so that the tests can be repeated.

Table A.1: Environment and procedure for verification and performance measurements

Item	Details
Source	Fresh clone of the public project repository, using the component-b directory
Environment	Linux; two virtual processor cores (Intel Xeon, 2.8 GHz); Python 3.13; TensorFlow 2.21 running on CPU; scikit-learn 1.6.1; XGBoost 3.x; NeuroKit2 0.2.13
Test procedure	pytest executed on the complete test directory with the package and server available on the Python path
Test result	75 tests collected; 74 passed; 1 skipped because the raw WESAD segment required by that test is not distributed with the repository
Timing procedure	40 consecutive 15-second frames from the project's PPG demonstration generator were streamed through the inference engine using the exported models
End-to-end backend procedure	The server was started in its deployed configuration. One WebSocket client sent 16 frames to /ingest, while a second client subscribed to /stream and received the generated prediction messages
End-to-end result	All 16 frames were accepted. After the initial 60-beat window was formed, 45 predictions were produced, consisting of 30

	single-level predictions and 15 band predictions
Message validation	All prediction messages received through /stream followed the output schema defined in Table 4.10
Network timing	Median time from sending a frame to receiving the final prediction triggered by that frame was 0.31 s; maximum was 0.46 s

Appendix B Academic integrity and AI use disclosure

AI tools were used in a limited and assistive manner in accordance with the IT4010 Research Project AI Usage Policy. Their use was restricted to activities such as concept clarification, debugging support, language refinement, and improving the clarity and structure of selected written content. The module policy allows AI to support learning, development, and writing, provided that it does not replace the student's own understanding or original contribution.

Tool	Purpose
ChatGPT	Concept clarification, language refinement, citation-order checking, and debugging support
Claude	Language editing and clarity improvement

AI tools were not used to replace the research problem formulation, research design, core implementation, experimentation, statistical analysis, or critical evaluation. These activities were completed by the researcher, as required by the module policy.

All AI-assisted material was reviewed before inclusion in the thesis. Technical statements were checked against the implementation, experimental outputs, and original research sources. References and DOI information were also verified before use. AI-generated suggestions were treated as supporting material rather than as authoritative research evidence.

The researcher remains fully responsible for the accuracy, validity, originality, and interpretation of all submitted work and is able to explain and justify the methodology, implementation, experimental results, and conclusions presented in the thesis.